Final Project Documentation



Federal Urdu University

Of Arts, Science & Technology, Gulshan-e-Iqbal Campus



CineMax Karachi: Online Movie Ticket Booking System

Federal Urdu University of Arts, Science & Technology
Gulshan-e-Iqbal Campus, Karachi
Department of Computer Science

FINAL YEAR PROJECT REPORT

CineMax Karachi: An Online Movie Ticket Booking System

Project Thesis and Technical Report

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Computer Science

PROJECT MEMBERS

Name	Roll number	Contribution
Umair	19252071	Administration panel and customer account features
Abdul Rafay	19252064	Database, booking engine and integrations
Muhammad Arqam	19252054	User interface design

Supervisor
Muhammad Sadiq
Head of Department, Department of Computer Science

October 2026

Table of Contents

Declaration iv

Certificate of Approval v

Acknowledgements vi

Abstract vii

Chapter 1: Introduction 1
- 1.1 Background 1
- 1.2 The starting point: a legacy academic system 2
- 1.3 Problem statement 2
- 1.4 Motivation 3
- 1.5 Aims and objectives 3
- 1.6 Scope 4
- 1.7 Target users 4
- 1.8 Contributions 5
- 1.9 Team and responsibilities 5
- 1.10 Organisation of the report 6

Chapter 2: Literature Review and Background 7
- 2.1 Online ticketing as a problem domain 7
- 2.2 Existing solutions 8
- 2.3 Web application architecture 9
- 2.4 Technologies 11
- 2.5 Theoretical framework: concurrency control 13
- 2.6 Theoretical framework: web application security 14
- 2.7 QR codes for ticket verification 15
- 2.8 Usability considerations 15
- 2.9 Summary and research gap 16

Chapter 3: Requirements Analysis 17
- 3.1 Stakeholders and actors 17
- 3.2 Functional requirements 18
- 3.3 Non-functional requirements 21
- 3.4 Use case model 23
- 3.5 Business rules 26
- 3.6 Feasibility study 27

Chapter 4: System Architecture and Design 28
- 4.1 Architectural overview 28
- 4.2 Layered view 30
- 4.3 Request lifecycle 30
- 4.4 Database design 31
- 4.5 Seat model 36
- 4.6 Booking workflow 36
- 4.7 Booking and refund state machines 38
- 4.8 Component interactions 39
- 4.9 User interface design 40
- 4.10 Key design decisions 41

Chapter 5: Methodology and Implementation 43
- 5.1 Development methodology 43
- 5.2 Tools and environment 45
- 5.3 Project structure 45

5.4 The helper layer 46
5.5 Shows and the seat map 48
5.6 Seat holds 52
5.7 Checkout 53
5.8 Payment simulation 55
5.9 Booking finalisation 57
5.10 E-ticket and QR code 59
5.11 Booking history and cancellation 62
5.12 Ticket scanner 64
5.13 Administration panel 66
5.14 Dashboard and reports 70
5.15 Customer account features 73
5.16 TMDB integration 76
5.17 Deployment 77

Chapter 6: Risk Management and Mitigation 79
6.1 Approach 79
6.2 Project management risks 79
6.3 Technical risks 80
6.4 Security risks 82
6.5 Data protection and privacy 85
6.6 Residual-risk register 85

Chapter 7: Testing, Evaluation and Results 86
7.1 Test strategy 86
7.2 Test environment 87
7.3 Unit testing of business rules 87
7.4 Integration and end-to-end testing 88
7.5 Concurrency testing 90
7.6 Security testing 91
7.7 Performance evaluation 93
7.8 Usability and accessibility review 95
7.9 Evaluation against objectives 96

Chapter 8: Future Scope and Conclusion 98
8.1 Immediate improvements (before any production use) 98
8.2 Functional extensions 99
8.3 Data-model clean-up 100
8.4 Scalability 100
8.5 Mobile and API 101
8.6 Engineering practice 101
8.7 Lessons learned 101
8.8 Conclusion 102

References 104

Appendix A: Installation and Configuration 107

Appendix B: AJAX Endpoint Reference 109

Appendix C: Requirement-to-Test Traceability 111

Appendix D: Demonstration Script (Summary) 113

Appendix E: Glossary 114

Appendix F: Additional Administration Screens 115

Declaration

We, the undersigned, declare that this report and the software it describes are our own work, carried out as our Final Year Project at the Federal Urdu University of Arts, Science and Technology, Karachi. Where we used the work of others, including open-source libraries, templates and the TMDB data service, it is acknowledged in the text and in the References. This work has not been submitted for any other degree or qualification.

Name	Roll number	Signature
Umair	19252071	
Abdul Rafay	19252064	
Muhammad Arqam	19252054	

Certificate of Approval

This is to certify that the Final Year Project titled *CineMax Karachi: An Online Movie Ticket Booking System*, submitted by the students named above, has been examined and approved in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science.

Role	Name	Signature	Date
Supervisor	Muhammad Sadiq		
Internal examiner			
External examiner			
Chairperson, Department of Computer Science	Muhammad Sadiq		

Acknowledgements

We thank our supervisor for guidance throughout the project, and the faculty of the Department of Computer Science at FUUAST for the knowledge on which this work is built. We also thank The Movie Database (TMDB) for its free developer API, and the maintainers of the open-source tools this system depends on: PHP, MariaDB, Apache, Bootstrap, Chart.js, SweetAlert2, qrcode.js and html5-qrcode. Finally, we thank our families for their patience and support.

Abstract

CineMax Karachi is a web-based cinema ticketing system that lets customers in Karachi browse current films, choose an exact seat on a live seat map, pay, and receive a QR-coded e-ticket, while cinema staff run the whole operation from an administrative panel. The project began as a minimal academic PHP application with a flat booking table, string-built SQL queries and no notion of individual seats. We re-engineered it into a show-based, seat-level booking platform without changing its core technology: plain PHP, MySQL-compatible MariaDB 10.4 and server-rendered HTML.

The central technical problem in any ticketing system is preventing two customers from buying the same seat. We solve it with a three-layer design. First, selected seats are placed in a `seat_holds` table for five minutes. Second, a composite `UNIQUE (show_id, seat_label)` key on both `seat_holds` and `booking_seats` makes the database itself refuse a duplicate claim, so correctness does not depend on application timing. Third, the booking is written inside a single InnoDB transaction that either commits the booking, its seats, snacks, promotion usage and loyalty points together, or rolls all of them back.

Around this core the system provides show scheduling with clash detection, three seat classes priced per film, snacks, promotional codes, a simulated payment step for card, JazzCash and Easypaisa with a one-time password, e-tickets carrying a QR code, a camera-based ticket scanner for check-in, cancellation with a time-based refund policy (100% at least 24 hours before the show, 50% after that), loyalty points, reviews, favourites and an analytics dashboard with CSV reports. The film catalogue is populated from The Movie Database (TMDB) API by a command-line seeder that runs inside a single transaction. Security controls include prepared statements, output escaping, CSRF tokens, session-gated administration and validated image uploads.

We evaluated the system on the project's Docker environment (PHP 7.4, Apache, MariaDB 10.4.34). Twenty-two unit checks of the business rules passed. In a concurrency test, ten authenticated users requested the same seat at the same instant for 20 rounds (200 requests), and every round produced exactly one successful hold. End-to-end booking, check-in, re-scan rejection, cancellation and authorisation tests behaved as specified. With ten concurrent clients, public pages served 130 to 176 requests per second with 95th-percentile latency between 70 and 112 ms. The evaluation also documents known limitations, chiefly plain-text password storage and a simulated payment gateway, and the report sets out a roadmap to address them.

Keywords: online ticket booking, seat reservation, concurrency control, database transactions, PHP, MariaDB, TMDB API, QR code, web application security

CHAPTER 1

Introduction

This chapter sets out why CineMax Karachi was built, the problem it solves, the objectives we committed to, the boundaries of the work and how the rest of the report is organised.

1.1 Background

Cinema-going in Pakistan has recovered strongly since the revival of multiplex cinemas in the major cities, and Karachi, as the country's largest city, hosts the densest concentration of screens. Chains such as Nueplex, Cinepax, Atrium and Capri operate halls across districts that range from Saddar and Clifton to DHA and Bahria Town. A typical cinema shows a mix of Hollywood releases, Bollywood films and local Lollywood productions, often in several halls and at five or more show times per day.

For the customer, buying a ticket has traditionally meant either queuing at the box office or using a cinema chain's own website. Box-office sales give the customer no view of seat availability before arrival, cause long queues on release weekends and opening nights, and tie up staff who must hand-write or print tickets. Online sales solve these problems only when they are built on a reliable model of individual seats, shows and payments. A system that merely records "three tickets for the evening show" cannot tell a customer which seats remain, cannot stop two customers from receiving the same seat, and cannot be checked at the door.

Web technology has made this kind of system accessible to small teams. A relational database such as MySQL or MariaDB provides transactions and integrity constraints. A server-side language such as PHP renders pages and enforces business rules. Small JavaScript libraries supply interactive seat maps, charts and QR codes in the browser. Public APIs such as The Movie Database (TMDB) supply posters, synopses, trailers and release dates that would otherwise have to be typed in by hand. The project's thesis is that these widely taught, low-cost technologies are sufficient to build a ticketing system whose core guarantee, that no seat is ever sold twice, holds under concurrent load.

1.2 The starting point: a legacy academic system

The project did not start from an empty repository. It started from an existing "Online Movie Booking System", a small PHP and MySQL application of the kind commonly circulated as a student template. Its initial commit (5 October 2026 in our repository) contained a public site with movie listings by industry (Hollywood, Bollywood, Lollywood) and an admin panel for managing lookup tables such as cinemas, genres, languages and dates.

An audit of that baseline found the following properties, which shaped every later design decision:

- **No shows or seats.** A booking was a single row in `booking_details` that stored the movie name, cinema, date and price tier as copied text, plus a seat *count*. There was no concept of a scheduled show or of an individual seat.
- **No concurrency control.** Because seats did not exist, nothing could prevent overselling a hall.
- **SQL built by string interpolation.** Values from `$_GET`, `$_POST` and `$_SESSION` were inserted directly into SQL strings, which exposes the application to SQL injection.
- **Sensitive data in plain text.** Passwords and full card numbers were stored as entered.
- **Weak administration security.** Only some admin pages checked for a logged-in administrator, and several delete handlers could be called directly by URL.
- **Functional defects.** For example, the seat-price edit form issued an `UPDATE` without a `WHERE` clause, so editing one film's prices overwrote every film's prices.

We kept the baseline's technology (plain PHP, MySQL-compatible database, Bootstrap templates) and its original database dump (`dbmo.sql`) intact, and evolved the system through numbered, re-runnable migrations. This decision is discussed in Chapter 4.

1.3 Problem statement

Cinema customers in Karachi need a way to see real seat availability for a specific show, reserve exact seats, pay and receive a ticket that can be verified at the entrance, without visiting the box office. Cinema operators need a way to schedule shows, set prices, sell add-ons, process cancellations and refunds fairly, verify tickets at the door and measure sales. The baseline system could do none of these reliably: it had no seat model, could not prevent double booking, could not issue verifiable tickets, and was insecure against common web attacks.

Stated as a single problem: **design and implement a secure, seat-level online cinema booking system on a conventional PHP and MySQL stack that guarantees no seat is sold twice under concurrent access, and that supports the full ticket lifecycle from discovery to check-in and refund.**

1.4 Motivation

Three considerations motivated the project.

1. **A real, local need.** Seat-level online booking is still not universal among Karachi's cinemas, especially single-screen and smaller venues. A system that runs on inexpensive shared hosting or a single Docker host lowers the barrier for such venues.
2. **A concrete computer-science problem.** Double booking is a textbook instance of a race condition on a shared resource. Solving it correctly exercises database constraints, transactions, isolation and lock behaviour, which are central topics in the Database Systems and Operating Systems courses of the degree.
3. **Software-engineering practice.** Working on an inherited codebase as a team of developers is closer to industry reality than a greenfield exercise. It required us to audit legacy code, introduce migrations, document a shared schema contract and divide file ownership.

1.5 Aims and objectives

The aim of the project is to deliver a working, demonstrable cinema booking platform that a single Karachi cinema operator could adopt.

The specific objectives were:

1. **O1, Show model.** Introduce scheduled shows (film, cinema, hall, date, time, hall size) and let administrators create, edit and cancel them, with automatic detection of hall clashes.
2. **O2, Seat-level booking.** Present an interactive seat map with three seat classes (Economy rows A to C, Business rows D to F, First rows G to H) and per-film prices.
3. **O3, Double-booking prevention.** Guarantee at the database level that a seat cannot be held or sold to two customers for the same show, and verify the guarantee under concurrent requests.
4. **O4, Checkout and payment.** Support snacks, promotional codes and a demonstration payment step for card and the two dominant Pakistani mobile wallets, JazzCash and Easypaisa, storing only the last four digits of any account number.

5. **O5, Verifiable tickets.** Issue an e-ticket with a unique reference and QR code, and provide a scanner that admits each ticket exactly once.
6. **O6, Cancellation and refunds.** Implement a published refund policy with administrator approval or rejection, and release cancelled seats for resale.
7. **O7, Administration and analytics.** Provide a protected admin panel covering catalogue, prices, shows, snacks, promotions, users, reviews, messages, refunds, an activity log, a dashboard and exportable reports.
8. **O8, Real catalogue data.** Populate the catalogue with real current and upcoming films from TMDB.
9. **O9, Security baseline.** Remove SQL injection and cross-site scripting from all new and modified code, add CSRF protection to state-changing forms and harden file uploads.
10. **O10, Reproducible deployment.** Make the system runnable by any team member, or an examiner, with a single command.

Chapter 7 evaluates each objective against measured results.

1.6 Scope

In scope. A customer-facing website (browse, search, filter, film details, show selection, seat map, checkout, payment simulation, e-ticket, booking history, cancellation, profile, favourites, reviews, contact). An administration panel for one cinema operator managing several cinemas and halls. A command-line catalogue seeder using TMDB. A Docker-based development and demonstration environment, with XAMPP as an alternative.

Out of scope. Real money movement through a payment gateway; email or SMS delivery; native mobile applications; multi-tenant operation by competing cinema chains; seat layouts other than rectangular halls; dynamic pricing; and integration with cinema projection or point-of-sale hardware. Chapter 8 discusses how several of these could be added.

1.7 Target users

User group	What they need from the system
Cinema customers in Karachi	Find a film, see real seat availability, book exact seats, pay with a familiar method, carry a ticket on the phone, cancel with a clear refund rule
Box-office and door staff	Verify tickets quickly by camera or by typing a reference, and see whether a ticket is valid, already used or cancelled

User group	What they need from the system
Cinema managers	Schedule shows, set prices, run promotions, approve refunds, moderate reviews, and track revenue, occupancy and refunds
System administrators	Deploy and seed the system, apply database migrations, back up data

1.8 Contributions

The principal contributions of this project are:

- A seat-level data model (`shows`, `seat_holds`, `booking_seats`) added to a legacy schema through re-runnable migrations, without breaking existing bookings.
- A database-enforced, transaction-based double-booking prevention scheme, validated by a repeatable concurrency test.
- A complete ticket lifecycle: hold, pay, issue, scan, cancel, refund, with a documented state machine.
- A small, reusable PHP helper layer (`includes/functions.php`, `includes/booking.php`) that brings prepared statements, escaping, CSRF tokens and flash messages to a framework-less codebase.
- An idempotent, transactional TMDB catalogue seeder.
- A shared schema contract and ownership model that let the developers work on the same codebase in parallel.

1.9 Team and responsibilities

Member	Roll number	Responsibilities
Abdul Rafay	19252064	Database design and migrations, shows, seat map and seat holds, checkout, payment simulation, e-tickets and QR scanner, cancellations and refunds, dashboard, reports, live search, TMDB integration, Docker environment
Umair	19252071	Admin management pages (movies, prices, snacks, promo codes, coming soon, users, reviews, messages, activity log), admin authentication across pages, sign-up, profile, favourites, reviews, contact, navigation, site pages, end-to-end test passes
Muhammad Arqam	19252054	User interface design: the look and layout of the customer website and the admin panel

1.10 Organisation of the report

Chapter 2 reviews existing booking platforms and the technologies and theory the system relies on. Chapter 3 analyses requirements and use cases. Chapter 4 presents the architecture, database schema and key workflows. Chapter 5 describes the implementation module by module with code excerpts. Chapter 6 covers project, technical and security risks and their mitigation. Chapter 7 reports the testing strategy and measured results. Chapter 8 discusses future work and concludes. Appendices contain installation steps, the API reference, the full test-case catalogue and the demonstration script.

CHAPTER 2

Literature Review and Background

This chapter reviews the kinds of cinema booking systems already in use, the architectural patterns and technologies available to build one, and the theory of concurrency control and web security on which our design rests. It closes by identifying the gap that CineMax Karachi addresses.

2.1 Online ticketing as a problem domain

An online ticketing system sells access to a perishable, capacity-limited resource. A seat for the 18:30 show has value only until 18:30, and each seat can be sold exactly once. These two properties distinguish ticketing from ordinary e-commerce. In a typical online shop, stock is fungible (any unit of a product will do) and oversell can be fixed later by back-ordering. In ticketing, the customer chooses a *specific* unit (seat E6), and an oversold seat cannot be fixed after the fact without disappointing a customer in front of the screen.

The domain therefore has a well-known structure, described in practitioner literature on reservation systems (Kleppmann, 2017; Fowler, 2002):

- **Inventory** is a set of discrete, identifiable units (seats) attached to a time-bound event (a show).
- **Selection** is interactive and slow: customers look at a seat map, change their minds and compare prices.
- **Commitment** is fast but must be atomic: once payment succeeds, the seats, the order and the payment record must all be saved, or none of them.
- **Contention** peaks sharply on release days, when many customers compete for the same central seats at the same moment.

The standard response to slow selection followed by fast commitment is a **temporary hold** (also called a lease or soft reservation): the system sets the selected seats aside for a short time so the customer can pay without losing them, and returns them to sale automatically if the customer

abandons the purchase. Every major consumer ticketing platform exposes this as a visible countdown during checkout. CineMax Karachi adopts the same pattern with a five-minute hold.

2.2 Existing solutions

We surveyed the categories of booking systems available to a Karachi cinema-goer and to a small cinema operator. We describe categories rather than ranking named commercial products, because their feature sets change frequently and their internals are not public.

Global aggregators. Platforms such as BookMyShow in South Asia and Fandango in North America aggregate listings from many cinema chains. They offer seat maps, wallets, offers and reviews. They are closed-source, integrate with each chain's ticketing back end, and charge per-ticket convenience fees. They are not available to a small Karachi venue as software it can run.

Cinema-chain websites. Large local chains run their own booking sites, typically built on commercial cinema management software that also drives the box office and the concession stand. These systems are comprehensive but proprietary and licensed per site, which places them out of reach of single-screen or small multi-screen cinemas.

Local ticket marketplaces. Pakistani marketplaces sell cinema tickets alongside bus, event and travel tickets, and support local payment methods such as JazzCash and Easypaisa. They depend on the participating cinema exposing its inventory.

Academic and open-source projects. Many freely available "online movie booking" projects written in PHP and MySQL are used as student templates. The baseline of this project belongs to this category. In our review of the baseline and of the pattern it represents, such projects usually record a booking as a single row with a seat count, build SQL by concatenating request values, store passwords and card numbers in plain text, and do not model shows or individual seats. They demonstrate CRUD operations but not the essential correctness property of a ticketing system.

Table 2.1 summarises the comparison.

Table 2.1: Capabilities by category of booking system

Capability	Global aggregator	Chain website	Local marketplace	Typical academic PHP project	CineMax Karachi
Seat-level selection on a live map	Yes	Yes	Usually	No	Yes

Capability	Global aggregator	Chain website	Local marketplace	Typical academic PHP project	CineMax Karachi
Database-enforced double-booking prevention	Yes (internal)	Yes (internal)	Depends on cinema	No	Yes
Timed seat hold with countdown	Yes	Yes	Varies	No	Yes, 5 minutes
Local wallets (JazzCash, Easypaisa)	Varies by region	Varies	Yes	No	Simulated
QR e-ticket and door scanning	Yes	Yes	Varies	No	Yes
Self-service cancellation with refund policy	Varies	Varies	Varies	No	Yes
Operator analytics and reports	Internal	Yes	Partner portal	Minimal	Yes, with CSV export
Source available to a small operator	No	No	No	Yes	Yes
Hosting cost for an operator	Commission	Licence	Commission	Shared hosting	Shared hosting or one Docker host

The gap is visible in the last two rows. Systems with the correct seat model are proprietary, while the systems a small operator could run lack it. CineMax Karachi aims to occupy the empty cell: an open, low-cost system with the correctness properties of the commercial platforms.

2.3 Web application architecture

2.3.1 Client-server and three-tier architecture

Web applications conventionally follow a three-tier architecture: a **presentation tier** (the browser rendering HTML, CSS and JavaScript), an **application tier** (server code applying business rules) and a **data tier** (a database management system). The tiers communicate over HTTP and over a database protocol respectively. The separation allows each tier to be scaled and secured independently (Sommerville, 2016).

2.3.2 Server-side rendering and single-page applications

Two styles dominate the presentation tier. In **server-side rendering (SSR)**, each URL is handled by server code that queries the database and returns a complete HTML page. In a **single-page application (SPA)**, the browser downloads a JavaScript application once and exchanges JSON with a web API. SPAs give richer interaction at the cost of a separate API layer, a JavaScript build toolchain and more complex state management. SSR remains simpler to develop, to secure (no API surface beyond the pages) and to host.

CineMax Karachi uses SSR for all pages and adds small JSON endpoints only where interactivity requires it: seat holds, seat status polling, live search, promo validation and OTP issue. This hybrid keeps the benefits of SSR while providing an interactive seat map.

2.3.3 Enterprise application patterns

Fowler (2002) catalogues patterns for organising web application logic. Three of them describe CineMax Karachi precisely:

- **Page Controller.** Each page is its own controller: `seat_map.php` handles the request for the seat map, `checkout.php` for checkout, and so on. This is the natural structure of a framework-less PHP site.
- **Transaction Script.** Business operations are organised as procedures, each handling one request end to end, for example `finalize_booking()` and `cancel_booking()` in `includes/booking.php`. Fowler recommends Transaction Script for applications with modest domain logic, where a full Domain Model would add more structure than value.
- **Template View.** Pages embed PHP in HTML markup to render data, as in `partials/ticket_card.php`.

Fowler also describes the **Post/Redirect/Get** discipline, in which a form submission is processed and then answered with an HTTP redirect, so that refreshing the result page does not resubmit the form. Our checkout, payment, cancellation and admin forms follow it.

The alternative would be a Model-View-Controller framework such as Laravel or Symfony. Section 2.4.1 explains why we did not migrate to one.

2.4 Technologies

2.4.1 PHP

PHP is a server-side scripting language designed for the web, with built-in session handling, form parsing and database drivers. It remains one of the most widely deployed server-side languages and is supported by nearly every shared hosting provider. The baseline system was written in procedural PHP, and the target deployment for a small cinema is inexpensive shared hosting. We therefore kept plain PHP rather than adopting a framework. A framework migration would have meant rewriting every existing page while delivering none of the missing booking features, and it would have raised the hosting requirements.

The system targets PHP 7.4 (the Docker image `php:7.4-apache`) and was also run end to end by the team on PHP 8.2 under XAMPP. The relevant language features are the `mysqli` extension with prepared statements, `random_bytes()` and `random_int()` for cryptographically secure tokens and OTPs, `hash_equals()` for constant-time comparison, and `htmlspecialchars()` for output encoding.

2.4.2 MariaDB and the InnoDB storage engine

MariaDB is a community-developed fork of MySQL, wire-compatible with it and used interchangeably in most hosting environments. Our environment runs MariaDB 10.4.34. All application tables use the **InnoDB** storage engine, which provides:

- **ACID transactions** with `BEGIN`, `COMMIT` and `ROLLBACK`.
- **Row-level locking**, including `SELECT ... FOR UPDATE` to lock rows read inside a transaction.
- **Unique and foreign-key constraints** that are checked at write time regardless of the application code.
- A default isolation level of **REPEATABLE READ**, with next-key locking to prevent phantoms in locking reads.

MariaDB-specific features we rely on are `ADD COLUMN IF NOT EXISTS` and `ADD KEY IF NOT EXISTS`, which make schema migrations safe to re-run.

2.4.3 Apache HTTP Server

Apache serves PHP through the `mod_php` module in the official `php:7.4-apache` image. We use its per-directory `.htaccess` support to deny script execution in the profile-photo upload directory, a standard defence against upload-based code execution.

2.4.4 Front-end libraries

Library	Version	Purpose in CineMax Karachi
Bootstrap	5.1.3 (customer site), 4.1.3 (admin)	Responsive grid and components
jQuery	3.3.1 / 3.6.0	Template interactions (carousel, scrollbars)
Chart.js	2.7.2	Dashboard charts
SweetAlert2	11	Confirmation and error dialogs
Font Awesome	5.15.3 with v4 shims	Icons
qrcode.js	1.0.0	Renders the ticket QR code in the browser
html5-qrcode	2.3.8	Camera-based QR scanning in the admin scanner

The customer pages are built on the FlixGo cinema template and the admin panel on the Bootstrapious dashboard template. Both templates ship with dark colour schemes; Section 4.9 describes how the project converted them to a light theme without editing the vendored template files. Generating the QR code in the browser means no image is stored or transmitted, and the code carries only the booking reference and its signature (Section 5.10.1).

2.4.5 Docker

Docker packages an application and its dependencies into containers that run identically on any host (Merkel, 2014). The project's `docker-compose.yml` defines two services: `web` (PHP 7.4 with Apache and the `mysqli` extension, with the repository bind-mounted so edits are live) and `db` (MariaDB 10.4, initialised from `dbmo.sql` on first start and persisted in a named volume). A single `docker compose up -d --build` reproduces the environment on any team member's machine or an examiner's laptop, which addressed objective O10.

2.4.6 The Movie Database (TMDB) API

TMDB is a community-maintained film database with a free REST API for non-commercial use. It provides lists such as *now playing* and *upcoming* by region, a *discover* endpoint with filters such as original language and country of origin, detailed film records (title, overview, runtime, genres, spoken languages, ratings, popularity) and associated videos such as YouTube trailers. Images are served from a separate content delivery host at several resolutions. Authentication accepts either a v3 API key as a query parameter or a v4 read-access token as a bearer header. TMDB applies rate limits and asks clients to back off on HTTP 429 responses. Section 5.9 describes how our seeder uses these endpoints.

2.5 Theoretical framework: concurrency control

2.5.1 Race conditions and the double-booking anomaly

A race condition occurs when the correctness of a computation depends on the relative timing of concurrent operations. The naive booking algorithm is a check-then-act sequence:

```
if seat E6 is free:        -- read
    record E6 as sold      -- write
```

If two requests execute the read before either executes the write, both see the seat as free and both sell it. This is the *lost update* or *write skew* family of anomalies described by Berenson et al. (1995). On a busy release night, the window between read and write, typically a few milliseconds, is wide enough for collisions to occur.

2.5.2 ACID transactions and isolation levels

A transaction groups operations so that they are **Atomic** (all or nothing), **Consistent** (constraints hold before and after), **Isolated** (concurrent transactions do not see each other's partial work) and **Durable** (committed work survives failure) (Gray and Reuter, 1992; Silberschatz, Korth and Sudarshan, 2019). The SQL standard defines four isolation levels, from READ UNCOMMITTED to SERIALIZABLE, by the anomalies each permits. Importantly, a plain read inside a REPEATABLE READ transaction does not lock the row it reads, so wrapping the naive check-then-act sequence in a transaction does **not** by itself prevent double booking. Additional measures are needed.

2.5.3 Pessimistic and optimistic control

There are three standard ways to make check-then-act safe (Elmasri and Navathe, 2016; Kleppmann, 2017):

1. **Pessimistic locking.** Lock the resource before checking it, for example with `SELECT ... FOR UPDATE`, so a second transaction waits until the first commits.
2. **Optimistic concurrency control.** Proceed without locks and detect conflicts at commit time, for example with a version column, retrying on conflict.
3. **Constraint-based exclusion.** Let the database enforce the invariant directly through a `UNIQUE` constraint, so that the second insert of the same `(show, seat)` pair fails atomically. This collapses check and act into one indivisible operation.

The third approach is the most robust for seat booking because the invariant, "at most one sale per seat per show", maps exactly onto a unique key, and because it remains correct even if a future developer adds a new code path that forgets to lock. CineMax Karachi uses constraint-based exclusion as its primary guarantee and pessimistic locking (`FOR UPDATE` on the user's holds and on the promo code row) as a secondary measure inside the payment transaction.

2.5.4 Leases and expiring reservations

A hold is a **lease**: a grant of exclusive access that expires after a fixed time unless renewed or converted (Gray and Cheriton, 1989). Leases are a standard way to tolerate clients that disappear, such as a customer who closes the browser during checkout, without a separate clean-up protocol. Our holds carry an `expires_at` timestamp; expired holds are treated as free and are deleted lazily whenever seat state is read or written.

2.6 Theoretical framework: web application security

The OWASP Top 10 (OWASP Foundation, 2021) ranks the most critical web application security risks. The risks most relevant to this project are:

OWASP 2021 category	Manifestation in a booking system	Standard control
A01 Broken Access Control	Viewing or cancelling another customer's ticket; opening admin pages without logging in	Server-side ownership checks; session-gated admin pages
A02 Cryptographic Failures	Plain-text passwords; storing full card numbers	Adaptive password hashing (bcrypt or Argon2); never storing card data
A03 Injection	SQL injection through login or search fields; cross-site scripting through reviews	Prepared statements; contextual output encoding
A04 Insecure Design	Double booking; refund abuse	Database constraints; explicit business rules
A05 Security Misconfiguration	Scripts executable in upload folders; CLI tools reachable over HTTP	Deny execution; restrict entry points
A07 Identification and Authentication Failures	No login throttling; weak passwords	Rate limiting; password policy

SQL injection arises when user input is concatenated into SQL text and changes the query's structure (Halfond, Viegas and Orso, 2006). Prepared statements send the query structure and the parameter values separately, so input can never be parsed as SQL.

Cross-site scripting (XSS) arises when user-controlled text is emitted into HTML without encoding and is interpreted as markup or script. The standard defence is to encode every dynamic value for the context it is written into; for HTML text and attributes this means `htmlspecialchars()` with `ENT_QUOTES`.

Cross-site request forgery (CSRF) occurs when a malicious site causes a victim's browser, which carries the victim's session cookie, to submit a state-changing request. The **synchroniser token** pattern defeats it: the server places a secret, unpredictable token in each form and rejects submissions that do not echo it, since the attacking site cannot read the token (Barth, Jackson and Mitchell, 2008).

Unrestricted file upload can lead to code execution if an uploaded file with a script extension is later served by the interpreter. Defences are allow-listing extensions, verifying content, renaming files and disabling execution in the upload directory.

2.7 QR codes for ticket verification

A QR code is a two-dimensional matrix barcode standardised in ISO/IEC 18004. It encodes text with Reed-Solomon error correction at four levels (L, M, Q, H), recovering roughly 7%, 15%, 25% and 30% of damaged codewords respectively. Our tickets use level M, a balance between density and tolerance for screen glare or a creased printout. A QR ticket should carry an opaque identifier rather than personal data, since anyone who photographs the ticket can read it. Validity must be decided by the server at scan time, so that a copied QR code is accepted only once. Signing the identifier with a key that only the server holds (a message authentication code such as HMAC) additionally lets anyone, not only the door scanner, recognise a code that the cinema did not issue.

2.8 Usability considerations

Nielsen's ten usability heuristics (Nielsen, 1994) informed the customer flow. *Visibility of system status* motivated the live seat-hold countdown and the ten-second seat-map refresh. *Error prevention* motivated disabling sold and held seats rather than reporting an error after the click, and the client-side card-format checks. *Match between system and the real world* motivated a seat map laid out like a hall, with the screen at the top and an aisle in the middle. *Help users recognise, diagnose and recover from errors* motivated messages such as "Sorry, E6 was just taken by someone else. Please choose other seats."

2.9 Summary and research gap

Commercial platforms solve double booking but are proprietary and costly. Freely available academic systems are affordable but omit the seat model and are insecure. The literature offers a clear set of techniques for closing that gap: database constraints and transactions for correctness, leases for abandoned checkouts, synchroniser tokens and prepared statements for security, and server-validated QR identifiers for admission. CineMax Karachi applies these techniques to a conventional, framework-less PHP and MariaDB stack, and evaluates whether the correctness guarantee holds under concurrent load.

CHAPTER 3

Requirements Analysis

This chapter identifies the system's stakeholders, specifies its functional and non-functional requirements, models its use cases and records the business rules that the implementation must enforce. Requirements were elicited from three sources: an audit of the baseline system, observation of how Karachi cinemas sell tickets, and the feature sets of established platforms reviewed in Chapter 2.

3.1 Stakeholders and actors

Actor	Type	Description
Guest	Primary, human	An unauthenticated visitor who browses films, searches, reads reviews and views show times
Customer	Primary, human	A registered, active user who books, pays, views tickets, cancels, reviews and manages a profile
Administrator	Primary, human	Cinema staff with access to the admin panel: catalogue, shows, prices, snacks, promotions, users, refunds, scanning and reports
Door staff	Primary, human	An administrator role in practice, using only the ticket scanner at the hall entrance
TMDB	Secondary, system	External film-data service consulted by the catalogue seeder
System operator	Supporting, human	Deploys the containers, applies migrations, runs the seeder, takes backups
Scheduler (time)	Supporting, implicit	The passage of time expires seat holds and moves bookings from upcoming to past

3.2 Functional requirements

Requirements are numbered for traceability to the test cases in Chapter 7 and Appendix C. Priority uses the MoSCoW scale: Must, Should, Could.

3.2.1 Catalogue and discovery

ID	Requirement	Priority
FR-01	The system shall list films that are now showing, with poster, genre, language, rating and next show time	Must
FR-02	The system shall list films by industry (Hollywood, Bollywood, Lollywood) on separate pages	Must
FR-03	The system shall filter the film list by genre, language and cinema; the cinema filter shall show only films with upcoming active shows at that cinema	Should
FR-04	The system shall provide live search suggestions after two typed characters, returning at most six matches with posters, and support keyboard navigation	Should
FR-05	The system shall show a film detail page with synopsis, runtime, release date, TMDB rating, audience rating, trailer and upcoming shows	Must
FR-06	The system shall show a Coming Soon list with release dates and a countdown in days	Could
FR-07	The home page shall show a slider of featured films using TMDB backdrop images	Could

3.2.2 Accounts

ID	Requirement	Priority
FR-08	A guest shall be able to register with name, a unique valid email and a password of at least six characters	Must
FR-09	A customer shall be able to sign in and sign out; after sign-in the customer shall return to the page that required it	Must
FR-10	A blocked customer shall not be able to sign in, and an already signed-in customer who is blocked shall be signed out on the next page load	Must
FR-11	A customer shall be able to edit name and phone, upload a profile photo and change password	Should
FR-12	A customer shall be able to mark films as favourites and view the list	Could

3.2.3 Shows and seats

ID	Requirement	Priority
FR-13	The system shall list upcoming active shows for a film for the next seven days, grouped by date, with cinema, hall, time and seats left	Must
FR-14	The system shall show a seat map for a show with rows lettered from A and seats numbered from 1, an aisle at the midpoint, and the screen at the top	Must
FR-15	Seats shall be coloured by class (Economy, Business, First) and by state (free, selected, on hold by another customer, sold)	Must
FR-16	The seat map shall refresh seat states every ten seconds without reloading the page, and shall deselect a selected seat that another customer has taken	Should
FR-17	A customer shall be able to select at most ten seats per booking	Must
FR-18	On continuing, the system shall hold the selected seats for five minutes for that customer, replacing any earlier holds by the same customer	Must
FR-19	If any selected seat is already held or sold, the hold shall fail as a whole and the conflicting seats shall be named	Must

3.2.4 Checkout and payment

ID	Requirement	Priority
FR-20	Checkout shall show the held seats with class prices, a countdown of the hold, and active snacks with quantity controls (0 to 10 each)	Must
FR-21	A customer shall be able to apply one promotional code; the system shall reject unknown, inactive, expired and exhausted codes with a specific message	Should
FR-22	The system shall compute totals as tickets plus snacks minus a floored percentage discount, in whole rupees	Must
FR-23	Payment shall support Card (16-digit number, name, future expiry, 3 or 4 digit CVV) and JazzCash or Easypaisa (Pakistani mobile number and a six-digit OTP valid for five minutes)	Must
FR-24	Only the last four digits of the card or mobile number shall be stored	Must
FR-25	Confirming payment shall create the booking, its seats and snacks, increment promo usage, award loyalty points and release the holds as one atomic operation	Must
FR-26	If the hold has expired or a seat was taken at payment time, no booking shall be created and the customer shall be returned to the seat map with an explanation	Must

3.2.5 Tickets, history and cancellation

ID	Requirement	Priority
FR-27	Each booking shall receive a unique reference of the form `CNX-YYYY-NNNNN`	Must
FR-28	The e-ticket shall show film, cinema, hall, date, time, seats, snacks, discount, total, masked payment details and a QR code holding a signed verification link that any phone can check	Must
FR-29	The e-ticket shall be printable on its own (print and save as PDF)	Should
FR-30	A customer shall see bookings in Upcoming, Past and Cancelled tabs, with loyalty points	Must
FR-31	A customer shall be able to view only their own tickets	Must
FR-32	A customer shall be able to cancel a booked, not-yet-started show; the refund quote (100% or 50%) shall be shown before confirmation	Must
FR-33	Cancellation shall release the seats for resale and deduct the loyalty points earned	Must

3.2.6 Reviews and contact

ID	Requirement	Priority
FR-34	A customer shall be able to post one rating (1 to 5) and comment per film, and edit or delete it	Should
FR-35	The film's average rating shall count only visible reviews	Should
FR-36	Any visitor shall be able to send a contact message	Could

3.2.7 Administration

ID	Requirement	Priority
FR-37	Every admin page and action handler shall require an authenticated administrator	Must
FR-38	Administrators shall manage cinemas, genres, industries, languages, release dates and films, including poster upload and description	Must
FR-39	Administrators shall set Economy, Business and First prices per film	Must

ID	Requirement	Priority
FR-40	Administrators shall create, edit, cancel and delete shows; the system shall reject a second show in the same hall at the same date and time, and refuse to delete a show that has bookings	Must
FR-41	Administrators shall manage snacks and promotional codes (percentage, usage limit, expiry, active flag)	Should
FR-42	Administrators shall view and filter all bookings by status, date range, film and reference, and open booking details	Must
FR-43	Administrators shall approve or reject pending refunds, giving a reason when rejecting	Must
FR-44	Administrators shall check in tickets by camera or by typing a reference; results shall be VALID, ALREADY_USED, CANCELLED, WRONG_DAY, FORGED or NOT_FOUND, and only VALID checks the ticket in	Must
FR-45	Administrators shall view a dashboard of revenue, tickets, bookings, cancellations, refunds, users and today's bookings, with charts	Should
FR-46	Administrators shall produce reports filtered by date range, film and cinema, export them as CSV and print them	Should
FR-47	Administrators shall block and unblock customers, moderate reviews (Visible or Hidden), and read contact messages with an unread count	Should
FR-48	Every administrative add, edit and delete shall be recorded in an activity log with administrator, action, details and time	Should
FR-49	Administrators shall manage the Coming Soon list	Could
FR-50	An operator shall be able to replace the catalogue with current TMDB data from the command line, atomically	Should

3.3 Non-functional requirements

ID	Category (ISO/IEC 25010)	Requirement	Measure
NFR-01	Functional suitability: correctness	No seat may be sold twice for the same show under any interleaving of requests	Zero duplicate `(show_id, seat_label)` rows; concurrency test in Section 7.5
NFR-02	Reliability: fault tolerance	A failure during booking or seeding shall leave the database unchanged	All multi-row writes in one transaction with rollback

ID	Category (ISO/IEC 25010)	Requirement	Measure
NFR-03	Performance efficiency: time behaviour	Public pages shall respond in under 200 ms at the 95th percentile with ten concurrent users on a single host	ApacheBench, Section 7.6
NFR-04	Security: confidentiality	No full card number or full wallet number shall be stored	Schema and code review
NFR-05	Security: integrity	All new and modified queries that use request data shall use prepared statements; all dynamic output shall be HTML-escaped	Code review; injection tests
NFR-06	Security: integrity	State-changing admin actions and customer cancellations shall require a valid CSRF token	Token tests
NFR-07	Security: accountability	Administrative changes shall be attributable to an administrator	Activity log
NFR-08	Usability	The booking flow from film page to e-ticket shall take no more than five screens	Flow inspection: film, shows, seat map, checkout, payment, ticket
NFR-09	Usability: accessibility	Seats shall be buttons with labels and `aria-pressed` state; the hold timer shall be an ARIA live region	Markup review
NFR-10	Compatibility	The system shall run on PHP 7.4 and PHP 8.x, on MariaDB 10.4 or MySQL, and in current Chrome, Edge and Firefox, including mobile widths	Test runs on Docker (PHP 7.4) and XAMPP (PHP 8.2)
NFR-11	Maintainability	Schema changes shall be delivered as numbered, re-runnable migrations documented in a schema contract	Repository review
NFR-12	Portability	A new developer shall be able to run the system with one command	`docker compose up -d --build`
NFR-13	Localisation	Money shall be in whole Pakistani rupees; times shall use the Asia/Karachi time zone	`money()`; time zone set in PHP and in each MySQL session

3.4 Use case model

3.4.1 Use case catalogue

ID	Use case	Primary actor	Related requirements
UC-01	Browse and filter films	Guest	FR-01 to FR-03, FR-06, FR-07
UC-02	Search films	Guest	FR-04
UC-03	View film details and shows	Guest	FR-05, FR-13
UC-04	Register	Guest	FR-08
UC-05	Sign in / sign out	Customer	FR-09, FR-10
UC-06	Select seats and hold them	Customer	FR-14 to FR-19
UC-07	Check out and pay	Customer	FR-20 to FR-26
UC-08	View and print e-ticket	Customer	FR-27 to FR-29, FR-31
UC-09	View booking history	Customer	FR-30
UC-10	Cancel booking	Customer	FR-32, FR-33
UC-11	Review a film / manage favourites / profile	Customer	FR-11, FR-12, FR-34, FR-35
UC-12	Manage catalogue and prices	Administrator	FR-38, FR-39, FR-49
UC-13	Schedule shows	Administrator	FR-40
UC-14	Manage snacks and promotions	Administrator	FR-41
UC-15	Process refunds	Administrator	FR-43
UC-16	Scan tickets	Door staff	FR-44
UC-17	Monitor dashboard and reports	Administrator	FR-42, FR-45, FR-46
UC-18	Manage users, reviews and messages	Administrator	FR-47, FR-48
UC-19	Seed catalogue from TMDB	System operator	FR-50

3.4.2 UC-06 and UC-07: Book tickets (fully dressed)

Primary actor: Customer. **Stakeholders and interests:** the customer wants the chosen seats at the displayed price; the cinema wants each seat sold once and paid for. **Preconditions:** the show is Active and has not started; prices are set for the film. **Trigger:** the customer chooses a show time.

Main success scenario

1. The system displays the seat map with current seat states and class prices.
2. The customer selects one to ten free seats; the system shows a running total per class.
3. The customer presses Continue.
4. The system validates that every seat label exists in the hall, releases any earlier holds of this customer, and inserts a hold for each seat, expiring in five minutes.
5. The system opens checkout with a hold countdown.
6. The customer optionally adds snacks and applies a promo code; the system recomputes the total.
7. The customer proceeds to payment, chooses a method and enters details.
8. The system validates the details and, in one transaction, re-reads the customer's unexpired holds with a lock, re-validates the promo code with a lock, inserts the booking, assigns its reference, inserts one `booking_seats` row per seat, inserts snack lines, increments promo usage, adds loyalty points and deletes the holds.
9. The system commits and shows the e-ticket with its QR code.

Extensions

- 2a. The customer tries to select an eleventh seat: the system refuses and explains the limit.
- 2b. During selection, polling reveals that a selected seat was taken: the system deselects it and names it.
- 4a. A seat is already held or sold: the whole hold is rolled back and the conflicting seats are named; the customer chooses again.
- 4b. The show has started or been cancelled: the system reports that the show is no longer available.
- 5a. The countdown reaches zero: the system returns the customer to the seat map.
- 6a. The promo code is unknown, inactive, expired or exhausted: a specific message is shown and no discount applies.
- 8a. Payment details are invalid: the form is shown again with the error; the hold continues.
- 8b. The hold expired before payment: the transaction is rolled back and the customer is returned to the seat map.

- 8c. A seat insert violates the unique key: the transaction is rolled back and the customer is told which seat was taken.

Postconditions: exactly one booking exists with status Booked and refund status None; its seats are no longer selectable by anyone; the customer's holds are gone.

3.4.3 UC-10: Cancel booking

Preconditions: the booking belongs to the signed-in customer, has status Booked, and its show has not started.

1. The customer presses Cancel on an upcoming booking.
2. The system shows the refund quote: 100% if the show starts in 24 hours or more, otherwise 50%, floored to whole rupees.
3. The customer confirms.
4. The system verifies the CSRF token and ownership, then in one transaction sets status Cancelled, refund status Pending, the refund amount and the cancellation time (only if the status is still Booked), deletes the booking's seat rows and deducts the loyalty points, never below zero.
5. The system shows the Cancelled tab with the pending refund.

Extensions: 4a. The status is no longer Booked (for example the ticket was scanned in the meantime): nothing changes and the customer is told the booking can no longer be cancelled.

3.4.4 UC-16: Scan ticket

1. Door staff open the scanner and start the camera, or type a reference.
2. The scanner reads the QR code and submits its content, a signed link, with the session's CSRF token.
3. The system atomically changes status from Booked to Used and records the check-in time, and reports VALID with booking details.
4. If no row changed, the system reports ALREADY_USED, CANCELLED or NOT_FOUND according to the booking's state. A code whose signature does not match its reference is reported as FORGED, and a genuine unused ticket for another day as WRONG_DAY; neither is checked in.

3.5 Business rules

The following rules are recorded in the project's schema contract (`database/SCHEMA_CONTRACT.md`), which both developers agreed would take precedence over code if the two ever disagreed.

ID	Rule
BR-01	Rows A to C are Economy, D to F are Business, G and beyond are First. A standard hall has 8 rows of 12 seats (96 seats)
BR-02	Prices come from `movie_price` (`economy`, `business`, `first_class`) for the film
BR-03	A hold lasts 5 minutes; expired holds count as free and are deleted on read
BR-04	At most 10 seats per booking
BR-05	Total = tickets + snacks − discount; discount = floor((tickets + snacks) × percent / 100); all money is whole rupees stored as integers
BR-06	A promo code is valid if it exists, is active, has not passed `valid_until`, and `used_count < max_uses` (with `max_uses = 0` meaning unlimited); usage is incremented inside the booking transaction
BR-07	Loyalty points = floor(total / 100), added on booking and removed on cancellation, never below zero
BR-08	Booking reference = `CNX-` + year + `-` + booking id padded to five digits; the QR code holds a verification link with the reference and its HMAC-SHA256 signature
BR-09	Cancellation at least 24 hours before the show refunds 100%; less than 24 hours refunds 50%; started shows cannot be cancelled
BR-10	Refunds start Pending and are set to Refunded or Rejected by an administrator; rejection records a reason
BR-11	Check-in is allowed only for status Booked and sets status Used; Used and Cancelled tickets are invalid at the door
BR-12	A film or cinema that has shows cannot be deleted; a snack that has been sold cannot be deleted, only deactivated
BR-13	Booking rows keep copies of film, cinema, date and price text so that history does not change when the catalogue changes

3.6 Feasibility study

Technical feasibility. Every requirement maps onto standard features of PHP 7.4+ and InnoDB: sessions, prepared statements, transactions, unique keys and row locks. No component requires specialised infrastructure. The highest-risk requirement, NFR-01, can be met with a unique key, which the database enforces regardless of load.

Economic feasibility. All software used is free and open source, and TMDB is free for non-commercial use. A production deployment needs only a PHP and MySQL host, available in Pakistan at low monthly cost, plus a payment-gateway merchant account at the operator's expense.

Operational feasibility. Cinema staff already understand shows, halls and seat classes, so the admin panel uses the same vocabulary. The scanner runs in a phone or laptop browser and needs no dedicated hardware.

Schedule feasibility. The work was divided into a shared foundation step followed by five feature phases for the core booking engine, with the admin and customer-account pages developed in parallel under a file-ownership model (Section 5.1). The repository history shows the foundation, the five core phases, four administration and account tasks, the TMDB integration and the final test fixes all completed within the project timeline.

CHAPTER 4

System Architecture and Design

CineMax Karachi is a server-rendered, three-tier web application: browsers talk HTTP to Apache and PHP, PHP talks to a single MariaDB database, and an offline command-line seeder talks to the TMDB API. This chapter describes the architecture, the database schema and its evolution, the key workflows and state machines, and the design decisions behind them.

4.1 Architectural overview

Figure 4.1 shows the runtime components and how they call each other.

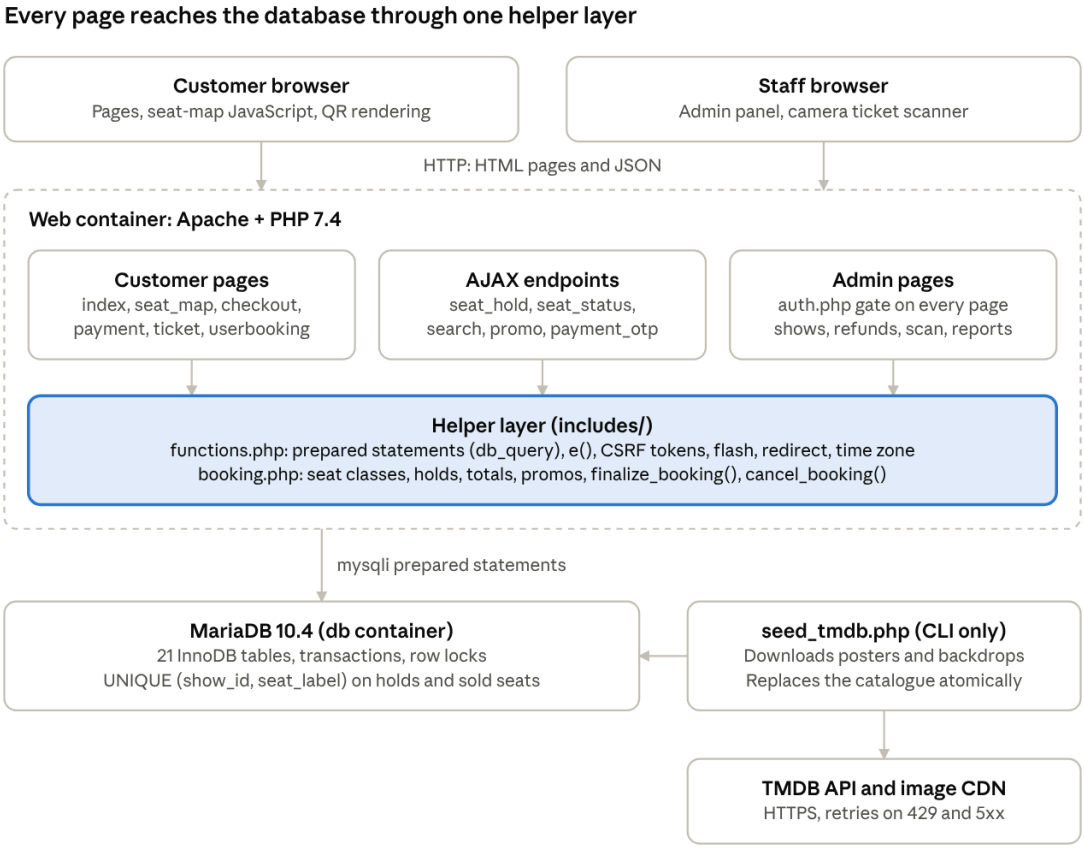


Figure 4.1: Runtime architecture. Browsers call page controllers and AJAX endpoints over HTTP; every one of them goes through the helper layer to MariaDB, and only the command-line seeder talks to TMDB.

The system has two web applications that share one database and one helper layer:

- **The customer site** (repository root) consists of page controllers such as `index.php`, `moviedetail.php`, `select_show.php`, `seat_map.php`, `checkout.php`, `payment.php`, `ticket.php`, `userbooking.php` and `cancel_booking.php`. Each page includes `header.php` and `footer.php` for the shared layout and navigation.
- **The admin panel** (`Admin/`) consists of one page per feature. Every page begins with `include('auth.php')`, which redirects to `login.php` unless an administrator is signed in, and then includes `admin_header.php`, which renders the sidebar and loads the helper layer.
- **The helper layer** (`includes/`) holds all shared logic. `functions.php` provides database access, escaping, CSRF tokens, flash messages, redirects and the current user. `booking.php` holds the booking domain: seat classes, holds, prices, totals, promotions, booking, cancellation and refunds. `admin_functions.php` provides the activity log. `tmdb.php` is the TMDB HTTP client.

- **AJAX endpoints** (`ajax/`) return JSON for the interactive parts of the customer site: `seat_hold.php`, `seat_status.php`, `search.php`, `promo.php` and `payment_otp.php`.
- **Partials** (`partials/`) are reusable view fragments: the movie browser, show list, ticket card, hold timer, home slider and coming-soon strip.
- **Database tooling** (`database/`) holds the numbered migrations, the schema contract, the TMDB seeder and a demo-data seeder. The seeders refuse to run over HTTP and run only from the command line.

4.2 Layered view

Although no framework is used, the code follows a consistent layering:

Layer	Responsibility	Where it lives
Presentation	HTML, CSS, client-side JavaScript (seat map, timers, charts, QR)	Page files below the PHP header block, `partials/`, `assets/booking.css`, `css/customer.css`
Controller	Read the request, check authentication, CSRF and ownership, call domain functions, redirect or render	Top of each page file; `ajax/*.php`
Domain	Business rules and transactions	`includes/booking.php`, `customer_functions.php`, `Admin/admin_upload.php`
Data access	Prepared statements and result fetching	`db_query()`, `db_one()`, `db_all()` in `includes/functions.php`
Persistence	Tables, constraints, transactions, time zone	MariaDB 10.4, InnoDB

A controller block at the top of each page performs all processing *before* any HTML is output. This allows real HTTP redirects (`Location` headers) after form submissions, which the Post/Redirect/Get discipline requires.

4.3 Request lifecycle

A typical state-changing request, such as a customer cancelling a booking, flows as follows:

1. The browser POSTs the form, including the hidden `csrf` field, to `cancel_booking.php`.

2. The page includes `includes/booking.php`, which includes `functions.php`, which includes `config.php` (opening the `mysqli` connection), starts the session if needed, sets the PHP time zone to Asia/Karachi and issues `SET time_zone = '+05:00'` on the connection.
3. `require_login()` redirects to `signin.php` if no customer is signed in, remembering the requested URL.
4. The controller rejects the request unless it is a POST with a valid CSRF token.
5. `load_booking()` fetches the booking by reference with a prepared statement; `owns_booking()` confirms that it belongs to the current user.
6. `cancel_booking()` runs the transaction.
7. The controller stores a flash message in the session and redirects to `userbooking.php?tab=cancelled`.
8. The browser follows the redirect with a GET; the page renders and displays the flash message once.

4.4 Database design

4.4.1 Overview

The database `dbmo` contains 21 InnoDB tables in the `utf8mb4` character set: 10 inherited from the baseline dump and 11 added by migration 001, together with new columns on three of the inherited tables. Figure 4.2 shows the entities and relationships.

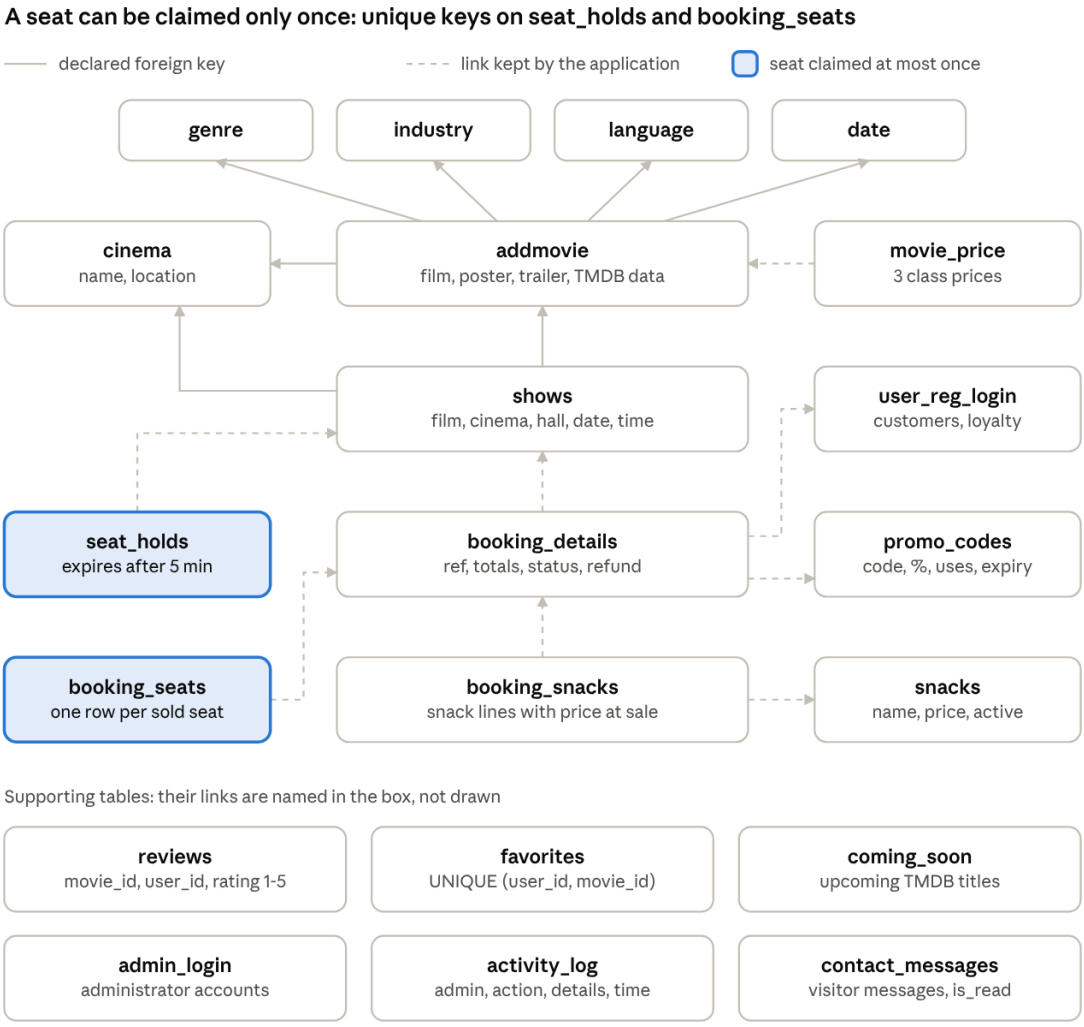


Figure 4.2: Entity-relationship diagram. `addmovie` is the hub of the catalogue and `shows` the hub of inventory; the two highlighted tables carry the `UNIQUE (show_id, seat_label)` keys that prevent double booking. Links between the booking tables and their parents are maintained by the application rather than declared as foreign keys, a choice inherited from the baseline schema.

4.4.2 Catalogue tables

Table	Key columns	Notes
`addmovie`	`movie_id` PK; FKs `movie_cin_id`, `movie_genre_id`, `movie_indus_id`, `movie_lang_id`, `movie_date_id`; `movie_name`, `movie_img`, `movie_trl_url`, `movie_duration`, `description`; TMDB columns `tmdb_id` (unique), `backdrop`, `release_date`, `rating` DECIMAL(3,1), `runtime`, `popularity`	The hub of the catalogue. Five foreign keys from the baseline link it to lookup tables
`cinema`	`cin_id` PK, `cinema_name`, `cinema_location`	Referenced by films and shows

Table	Key columns	Notes
`genre`, `industry`, `language`, `date`	Surrogate PK plus a name (or `date_name` DATE)	Lookup tables; the seeder creates missing values and removes unused ones
`movie_price`	`price_id` PK, `movie_name_price`, `economy`, `business`, `first_class`	Inherited design: linked to films by name, prices stored as text and cast to integers in PHP
`coming_soon`	`id` PK, `title`, `poster`, `trailer_url`, `release_date`, `description`, `tmdb_id` (unique)	Independent of `addmovie`

4.4.3 Scheduling and inventory tables

Table	Columns	Constraints and indexes
`shows`	`show_id` PK, `movie_id`, `cin_id`, `show_date` DATE, `show_time` VARCHAR(30) as 24-hour `HH:MM`, `hall_name`, `total_rows` (default 8), `seats_per_row` (default 12), `status` (Active or Cancelled)	FK to `addmovie` and `cinema`; index on `show_date`
`seat_holds`	`id` PK, `show_id`, `seat_label` (e.g. `D7`), `user_id`, `expires_at`	**UNIQUE (`show_id`, `seat_label`)**; index on `expires_at`
`booking_seats`	`id` PK, `booking_id`, `show_id`, `seat_label`, `seat_class`, `price`	**UNIQUE (`show_id`, `seat_label`)**; index on `booking_id`

The two unique keys are the heart of the design. They state the system's main invariant, at most one claim per seat per show, in a form the database enforces on every insert.

4.4.4 Transaction tables

`booking_details` began as the baseline's flat booking table and was extended by migrations 001 and 002. Its columns fall into three groups:

Group	Columns
Denormalised snapshot (baseline)	`user_name`, `user_email`, `user_movie_name`, `user_movie_cinema`, `user_date`, `user_movie_dur` (show time), `user_price` (seat-class summary such as "Business x2, First x1"), `user_seat` (seat labels), `user_card` (last four digits only), `user_movie_img`

Group	Columns
Links and money (001)	`user_id`, `show_id`, `booking_ref` (unique), `total_amount`, `snacks_amount`, `discount_amount`, `promo_code`, `payment_method`
Lifecycle (001, 002)	`status` (Booked, Used, Cancelled), `refund_status` (None, Pending, Refunded, Rejected), `refund_amount`, `refund_reason`, `refund_processed_at`, `cancelled_at`, `checked_in_at`, `booked_at`

Indexes exist on `user_id`, `show_id`, `status` and `booking_ref`. `booking_snacks` stores one line per snack (`booking_id`, `snack_id`, `qty`, `price` at the time of sale).

4.4.5 Customer, marketing and administration tables

Table	Purpose
`user_reg_login`	Customers: `id`, `name`, `email`, `password`, plus `phone`, `profile_pic`, `status` (Active or Blocked), `loyalty_points`, `created_at`
`admin_login`	Administrators: `id`, `email`, `password`
`snacks`	`snack_id`, `name`, `price`, `image`, `is_active`
`promo_codes`	`promo_id`, `code` (unique), `discount_percent`, `max_uses`, `used_count`, `valid_until`, `is_active`
`reviews`	`review_id`, `movie_id`, `user_id`, `rating` 1 to 5, `comment`, `status` (Visible or Hidden), `created_at`
`favorites`	`id`, `user_id`, `movie_id`; UNIQUE (`user_id`, `movie_id`)
`contact_messages`	`id`, `name`, `email`, `subject`, `message`, `is_read`, `created_at`
`activity_log`	`id`, `admin_email`, `action`, `details`, `created_at` (indexed)

4.4.6 Normalisation and deliberate denormalisation

The new tables are in third normal form: each non-key attribute depends only on its table's key. Two departures are deliberate.

1. **Snapshot columns on `booking_details`.** A booking records the film name, cinema, date, time and seat-class summary as text at the time of sale. If an administrator later renames a film, changes a price or deletes a past show, the customer's ticket and the revenue reports

still show what was actually sold. This is the standard treatment of transactional records, comparable to an invoice that copies the product description and price. `booking_seats.price` and `booking_snacks.price` follow the same principle.

2. **Derived totals.** `total_amount`, `snacks_amount` and `discount_amount` could be recomputed from the line tables, but they are stored so that legacy bookings, which have no line rows, report consistently.

One inherited weakness was kept for compatibility and is listed as future work: `movie_price` is joined to films by name rather than by `movie_id`, and stores prices as text.

4.4.7 Schema evolution through migrations

The original dump `dbmo.sql` is never edited. All changes are delivered as numbered SQL files in `database/migrations/` and applied in order:

Migration	Author	Change
`001_shared_schema.sql`	Abdul Rafay	New columns on `booking_details`, `user_reg_login` and `addmovie`; 11 new tables; backfill of totals, references and user ids for legacy bookings; seed shows, snacks and promo codes
`002_rafay_refund_reason.sql`	Abdul Rafay	`refund_reason`, `refund_processed_at`
`003_rafay_admin_email.sql`	Abdul Rafay	Seed administrator renamed
`004_umair_sample_data.sql`	Umair	Sample reviews and messages
`005_rafay_tmdb.sql`	Abdul Rafay	TMDB columns on `addmovie` and `coming_soon` with unique `tmdb_id`
`005_umair_demo_data.sql`	Umair	Demo customers, bookings, refunds, reviews and messages

Every statement is idempotent. Schema changes use `ADD COLUMN IF NOT EXISTS`, `ADD KEY IF NOT EXISTS` and `CREATE TABLE IF NOT EXISTS`; data inserts are guarded by `WHERE NOT EXISTS` or `INSERT IGNORE`. A developer who is unsure whether a migration was applied can simply run it again. Migration 001 also backfills legacy data: it computes `total_amount` from the old `"Economy : 1100"` price text multiplied by the seat count, generates `booking_ref` values from each booking's id and year, and links old bookings to users by email.

4.5 Seat model

A hall is a rectangle of `total_rows` × `seats_per_row` seats, 8 × 12 = 96 by default. Rows are lettered from A and seats numbered from 1, so labels run from `A1` to `H12`. The class of a seat is a pure function of its row letter:

```
index = ord(row) - ord('A')
class = Economy   if index < 3      (rows A-C)
        Business  if index < 6      (rows D-F)
        First     otherwise         (rows G onward)
```

The seat map inserts an aisle after seat `ceil(seats_per_row / 2)`. Validation (`valid_seat_label()`) accepts a label only if it matches `^[A-Z][0-9]{1,2}$`, its row is within the hall's rows and its number is between 1 and `seats_per_row`.

A seat's state for a given show is derived, not stored: it is **sold** if a `booking_seats` row exists, **held by me** or **held by another** if an unexpired `seat_holds` row exists, and **free** otherwise. Deriving the state from the two tables at read time means there is no separate status column that could drift out of agreement with them.

4.6 Booking workflow

Figure 4.3 traces the interactions from seat selection to e-ticket.

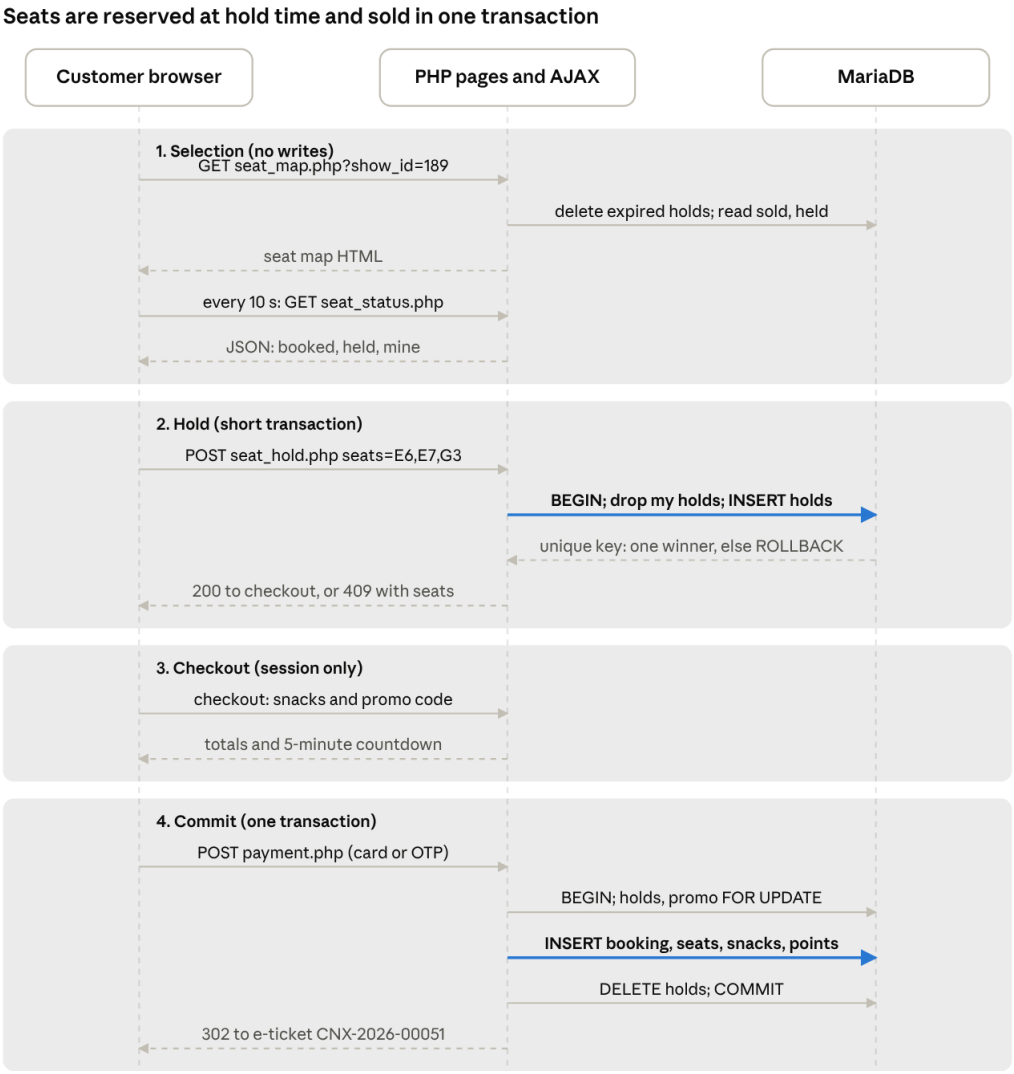


Figure 4.3: Booking sequence. Solid arrows are requests, dashed arrows replies; the two highlighted writes are the points where a `UNIQUE (show_id, seat_label)` key can refuse a seat. Selection and checkout write nothing to the seat tables.

The workflow has three phases with different consistency needs:

1. **Selection (no writes).** The customer selects seats in the browser. The page polls `seat_status.php` every ten seconds and disables seats that others have taken, but nothing is reserved. This keeps browsing cheap and avoids locking seats for customers who are only looking.

2. **Hold (short transaction).** Continue sends the selection to `seat_hold.php`. In one transaction the server deletes the customer's previous holds, checks each seat against `booking_seats`, and inserts a hold. Any failed insert, including a unique-key violation

caused by a concurrent customer, rolls back the whole hold. The customer therefore gets all the requested seats or none.

3. **Commit (one transaction).** On payment, `finalize_booking()` re-reads the customer's unexpired holds `FOR UPDATE`, re-validates the promo code `FOR UPDATE`, and writes the booking, reference, seat rows, snack rows, promo usage and loyalty points, then deletes the holds and commits. The `booking_seats` unique key is the final guard: even if a hold had somehow been lost, a duplicate sale would fail and roll back.

4.7 Booking and refund state machines

A booking has two orthogonal status fields. Figure 4.4 shows their transitions.

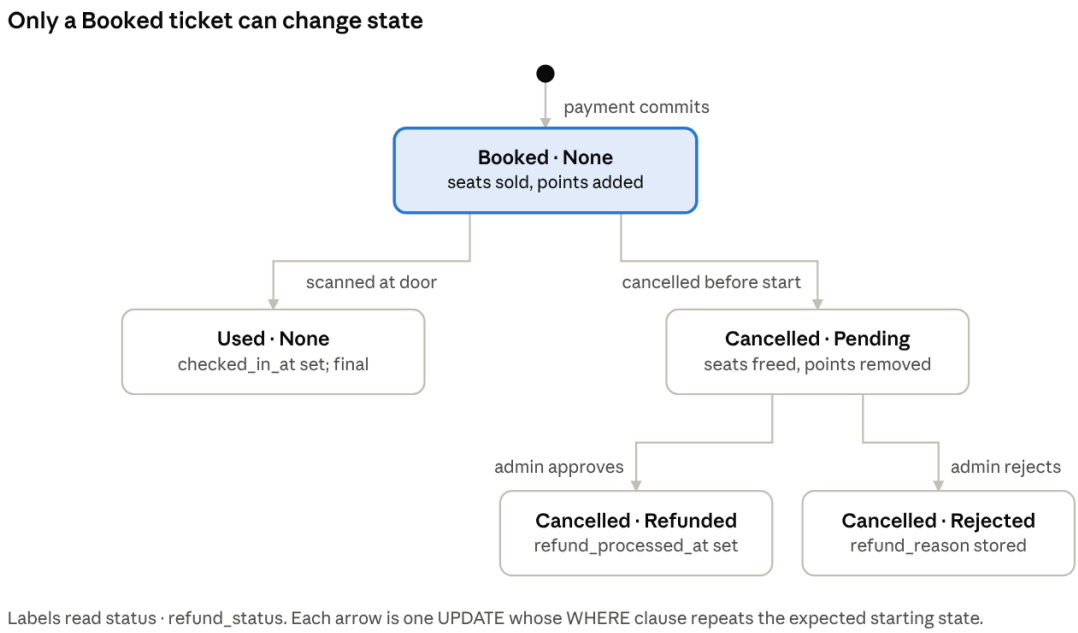


Figure 4.4: Booking state machine. A booking leaves the Booked state exactly once, either at the door or by cancellation; a cancelled booking then waits for an administrator's refund decision.

From	Event	Guard	To	Side effects
(none)	Payment confirmed	Unexpired holds; seats unsold	Booked / None	Seats, snacks, promo usage, loyalty points written
Booked	Ticket scanned	Status is Booked	Used	`checked_in_at` set
Booked	Customer cancels	Show not started	Cancelled / Pending	Refund 100% or 50%; seats released; points deducted

From	Event	Guard	To	Side effects
Cancelled / Pending	Admin approves	Refund Pending	Cancelled / Refunded	`refund_processed_at` set
Cancelled / Pending	Admin rejects	Refund Pending; reason given	Cancelled / Rejected	Reason and time recorded

Every transition is implemented as a conditional `UPDATE` whose `WHERE` clause repeats the guard, for example `WHERE booking_ref = ? AND status = 'Booked'`. The number of affected rows tells the code whether the transition happened. Two concurrent attempts, such as a scan and a cancellation of the same ticket, therefore cannot both succeed.

4.8 Component interactions

Interaction	Caller	Callee	Protocol and format
Page request	Browser	Page controller	HTTP GET, HTML response
Form submission	Browser	Page controller	HTTP POST (form-encoded), 302 redirect
Seat hold / release	Seat map JavaScript	`ajax/seat_hold.php`	`fetch` POST, JSON; 200, 401, 404, 409, 422 status codes
Seat status poll	Seat map JavaScript	`ajax/seat_status.php`	`fetch` GET every 10 s, JSON lists `booked`, `held`, `mine`
Live search	Movie browser JavaScript	`ajax/search.php`	GET `?q=`, JSON (max 6 results)
Promo check	Checkout JavaScript	`ajax/promo.php`	POST, JSON
OTP request	Payment JavaScript	`ajax/payment_otp.php`	POST, JSON (demo returns the code)
Ticket check-in	Scanner JavaScript	`Admin/scan_ticket.php`	POST with CSRF token, JSON result
Data access	All PHP	MariaDB	`mysqli` prepared statements over TCP
Catalogue import	`seed_tmdb.php` (CLI)	TMDB API and image CDN	HTTPS GET with cURL, JSON and JPEG

4.9 User interface design

The customer site uses a clean light theme: white header and cards with thin grey borders and soft shadows on a light grey page (`#f5f6fa`), dark text (`#1f2230`), and the pink-to-coral gradient of the original brand for buttons, links and highlights. The booking pages share a visual vocabulary of *cards*, *chips* and *badges*: seat-class chips with prices, status badges in green (Booked, Refunded), blue (Used), orange (Pending) and red (Cancelled, Rejected), each a dark label on a pale tint so that it stays readable on white.

The header groups the camera logo, on a pink gradient tile, with the name "CineMax Karachi" as one link, so the brand stays together at every screen width, and the home page's typed heading scales its font size with the viewport so the whole sentence fits on a phone. Banners built from film images (the home slider, the blurred backdrop above the seat map, the trailer) are deliberately kept dark, because white text over a still frame reads better than dark text.

The seat map renders each seat as a `<button>` with a `data-label`, a `data-class` and an `aria-pressed` attribute. Economy, Business and First seats use blue, purple and gold tints; selected seats turn green; seats held by others are orange and disabled; sold seats are greyed out and disabled. A sticky summary bar shows the selected seats as removable chips, a per-class breakdown and the total. The checkout and payment pages carry a fixed countdown badge that turns red in the final minute.

The admin panel follows the same light palette: a white navbar and sidebar, white content cards on a grey page, and the template's coral accent for the active menu item, primary buttons and focus rings. The sidebar is grouped under three headings: "Main" (Dashboard, All Bookings, Cinema, Industry, Genre, Language, Cinema Date, Add & View Movies, Seat Price Details, Users), "Shows & Tickets" (Shows, Scan Ticket, Refunds, Reports) and "Customers & Content" (Snacks, Promo Codes, Reviews, Messages, Coming Soon, Activity Log). Dashboard charts use dark axis labels and faint grid lines to match.

4.9.1 How the light theme is implemented

Both templates were dark, and their stylesheets (about 3,700 and 3,500 lines) are vendored files that the project does not edit. The light theme is therefore applied as a set of override layers loaded after the template CSS:

Layer	File	What it does
Shared colour tokens	`assets/booking.css`	CSS custom properties (`--cx-bg` , `--cx-card` , `--cx-text` , `--cx-text-soft` , `--cx-muted` , `--cx-line` , `--cx-shadow`) now hold light values; every booking component reads them, so changing the tokens re-themes the seat map, checkout, payment, tickets and history pages at once
Template overrides (customer)	`assets/light.css`	Re-declares only the colour properties of the FlixGo rules that used the dark palette, mapping each dark value to its light counterpart (for example page background `#2b2b31` to `#f5f6fa` , card `#28282d` to white, white text to `#1f2230`) while leaving white text on pink buttons untouched; photo backgrounds on page-title bands are replaced by a light gradient
Template overrides (admin)	`Admin/css/custom.css`	The template's own slot for local changes, loaded after `style.default.css` ; re-colours the navbar, sidebar, cards, tables, forms, modals, tabs and pagination
Page-level styles	Individual pages	Inline styles that hard-coded white text or dark panels (seat map, movie page, movie grid, checkout, payment, e-ticket, admin login) were converted to the shared tokens

The override sheet for the customer template was produced by a small script that walks the template's CSS rules, keeps every rule whose background, text, border or shadow colour belonged to the dark palette, and emits only those properties with mapped values, inside the same media queries. This keeps the override to about 470 lines and guarantees that no template rule using a dark colour is missed. Colours that carry meaning were also adjusted for contrast on white: the First-class seat colour was darkened from `#f5c542` to `#e0a800` , and sold seats became light grey with a cross rather than dark grey.

Finally, both override stylesheets are linked with a version parameter taken from the file's modification time (`light.css?v=<filemtime>`), so browsers fetch the new CSS as soon as it changes, without a manual cache refresh.

4.10 Key design decisions

Decision	Alternatives considered	Rationale
Keep plain PHP; add a small helper layer	Migrate to Laravel or another MVC framework	Preserves the existing pages and shared-hosting deployability; the helper layer supplies the safety features a framework would

Decision	Alternatives considered	Rationale
Evolve the schema with re-runnable migrations; never edit `dbmo.sql`	Edit the dump; recreate the database	Keeps legacy data, lets developers apply changes in any state, documents history
Enforce seat exclusivity with unique keys	Application-level checks only; table locks	Correct under any interleaving and any future code path; no global lock contention
Five-minute expiring holds stored in a table	Holds in PHP sessions; no holds	Holds must be visible to other customers' seat maps and enforced by the database
Lazy expiry on read	Cron job or database event	No scheduler to deploy; expired rows are deleted whenever seats are read or held
Poll seat status every 10 s	WebSockets or server-sent events	Works on any PHP host; the unique key, not the display, guarantees correctness
QR holds a link with the booking reference and an HMAC signature	Plain reference only; personal data in QR	No personal data in the code; any phone can confirm the cinema issued it; entry is still decided by the server at scan time
Store only last four digits of payment accounts	Store full card numbers (baseline)	Removes card data from scope entirely
Snapshot film and price text on bookings	Join to catalogue at read time	Tickets and reports remain accurate after catalogue edits
Seed catalogue from TMDB by CLI in one transaction	Manual entry; live API calls per page view	Real data without per-request latency or API dependence; all-or-nothing replacement

CHAPTER 5

Methodology and Implementation

This chapter explains how the system was built: the development process and team workflow, the tools, the project structure, and then each module in turn with the code that implements it. Code excerpts are taken from the repository and shortened where marked with `...`.

Screenshots of each screen appear in the section that describes it. They were taken on 13 October 2026 from the Docker deployment described in Section 5.17, using the TMDB catalogue and demonstration data; the booking shown, `CNX-2026-00059`, was made through the normal booking flow for these screenshots. Further administration screens are in Appendix F.

5.1 Development methodology

5.1.1 Incremental, phase-based delivery

We followed an incremental model in which each increment delivered a usable, demonstrable slice of the booking flow on top of the previous one. This suited an inherited codebase: every increment left the site working, so the team could demonstrate progress at any point and a defect could be traced to a small change.

Increment	Content	Owner
Step 0: Foundation	Schema contract, migration 001, helper layer (`functions.php` , `booking.php`), seed data	Abdul Rafay
Phase 1	Shows, seat map, seat holds, admin shows page	Abdul Rafay
Phase 2	Checkout, promo codes, simulated payment, one-transaction booking	Abdul Rafay
Phase 3	E-ticket with QR, print layout, admin ticket scanner	Abdul Rafay
Phase 4	My Bookings tabs, cancellation with refund policy, admin refunds	Abdul Rafay

Increment	Content	Owner
Phase 5	Real dashboard, booking filters, reports, movie filters and live search	Abdul Rafay
Task 1	Admin authentication on every page, sidebar, price-edit fix, movie descriptions, activity log	Umair
Task 2	Sign-up fix, admin management pages (snacks, promo codes, coming soon, users, reviews, messages)	Umair
Task 3	Reviews, favourites, profile, contact, navigation bar	Umair
Task 4	Polish, About and FAQ pages, full end-to-end test, demonstration and viva documents	Umair
TMDB integration	API client, catalogue seeder, dynamic home slider, catalogue replacement	Abdul Rafay
Hardening	Seat map and admin login redesign, test fixes, demo-data seeder, booking date filter	Both

5.1.2 Team workflow

Two practices let the developers change the same codebase in parallel without conflicts.

1. **File ownership.** `TEAM_NOTES.md` assigns every file to one owner. A developer who needs a change in a file they do not own writes a numbered *request* in the notes instead of editing the file. The owner implements it and marks it done. Most requests were resolved this way, including the sign-up and Add Movie fixes needed after migration 001, admin authentication across all pages and the price-edit `WHERE` clause. Three requests remain open at the time of writing and are discussed in Chapter 6.
2. **Schema contract.** `database/SCHEMA_CONTRACT.md` documents every table, column, allowed value and business rule. It states that "if code and this file disagree, this file wins", and it may be changed only by agreement. The contract let one developer write the checkout while the other wrote the admin promo-code page, both relying on the same definition of `promo_codes`.

Version control used Git with a shared GitHub repository. The agreed loop was: commit after each piece of work, `git pull --rebase origin main`, then `git push origin main`. Database changes were only ever delivered as new migration files.

5.2 Tools and environment

Tool	Use
Docker and Docker Compose	Reproducible PHP 7.4 + Apache and MariaDB 10.4 environment
XAMPP (PHP 8.2, MariaDB)	Alternative local environment used for team testing on Windows
phpMyAdmin and the `mysql` client	Inspecting data, importing the dump and migrations
Git and GitHub	Version control and collaboration
Browser developer tools	Network and console inspection of AJAX endpoints
Playwright	Browser automation during end-to-end checks
ApacheBench (`ab`)	Load and latency measurements (Chapter 7)
cURL	Scripted integration and security tests (Chapter 7)

5.3 Project structure

```
FYP/
├── index.php, bollywood.php, hollywood.php, lollywood.php   film listings
├── moviedetail.php, select_show.php, seat_map.php           film page, shows, seat
map
├── checkout.php, payment.php, booking_success.php, ticket.php
├── userbooking.php, cancel_booking.php                      history, cancellation
├── signin.php, signup.php, profile.php, favorites.php, contact.php
├── about.php, faq.php, pricing.php, header.php, footer.php
├── customer_functions.php                                    customer helpers
├── config.php                                                DB connection
├── includes/
│   ├── functions.php        db_query, e(), CSRF, flash, redirect, current user
│   ├── booking.php          seats, holds, totals, promos, booking, refunds
│   ├── admin_functions.php  log_activity()
│   ├── tmdb.php             TMDB HTTP client
│   └── secrets.php          TMDB key (private repository only)
├── ajax/                    seat_hold, seat_status, search, promo, payment_otp
├── partials/                movie_browser, show_list, ticket_card, hold_timer, ...
├── Admin/                   39 PHP files: auth, dashboard, CRUD pages, shows,
│                            refunds, scanner, reports, uploads, activity log
├── database/
│   ├── migrations/          001 ... 005
│   ├── SCHEMA_CONTRACT.md
│   ├── seed_tmdb.php        CLI catalogue seeder
│   └── seed_demo.php        CLI demo-data seeder
├── assets/booking.css       booking UI styles
├── uploads/profile/         customer photos (.htaccess denies scripts)
```

```
├── dbmo.sql                original database dump (never edited)
├── Dockerfile, docker-compose.yml
└── README.md, TEAM_NOTES.md, VIVA_PREP.md, DEMO_SCRIPT*.md
```

The application code amounts to roughly 9,000 lines of PHP: 27 customer pages (about 3,000 lines), 39 admin files (about 4,700 lines), 840 lines in the helper layer, 580 lines of partials and AJAX endpoints, and 1,020 lines of migrations and seeders. Vendored front-end libraries and templates are excluded from these counts.

5.4 The helper layer

The baseline had no shared code beyond the connection file. The first increment introduced `includes/functions.php`, which every new page includes. Including it has three side effects that establish a consistent runtime: it opens the database connection, starts the session if one is not already active, and aligns PHP and MySQL on the Asia/Karachi time zone.

```
require_once __DIR__ . '/../config.php';

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

date_default_timezone_set('Asia/Karachi');
mysqli_set_charset($con, 'utf8mb4');
mysqli_query($con, "SET time_zone = '+05:00'");
```

Aligning both clocks matters because business rules mix PHP time and SQL time: holds expire by SQL `NOW()`, while refund quotes compare a show's start with PHP `time()`. In the Docker container the server clock is UTC, so without these lines a show at 18:30 Karachi time would be judged five hours wrong.

5.4.1 Prepared-statement wrapper

`db_query()` is the single entry point for parameterised SQL. It prepares the statement, binds parameters by type string (`i` integer, `s` string, `d` double) and returns a result set for reads, the affected-row count for writes, or `false` on any failure.

```
function db_query($sql, $types = '', $params = [])
{
    $stmt = mysqli_prepare(db(), $sql);
    if (!$stmt) {
        return false;
    }
```

```
    if ($types !== '') {
        mysqli_stmt_bind_param($stmt, $types, ...$params);
    }
    if (!mysqli_stmt_execute($stmt)) {
        mysqli_stmt_close($stmt);
        return false;
    }
    $result = mysqli_stmt_get_result($stmt);
    if ($result === false) {
        $affected = mysqli_stmt_affected_rows($stmt);
        mysqli_stmt_close($stmt);
        return $affected;
    }
    mysqli_stmt_close($stmt);
    return $result;
}
```

`db_one()` and `db_all()` wrap it to return one associative row or all rows. Returning the affected-row count is what makes the conditional state transitions of Section 4.7 possible: callers test `=== 1` to learn whether their guarded `UPDATE` took effect. Across the codebase there are 223 calls to these three helpers, against 22 remaining raw `mysqli_query()` calls in legacy admin listing pages, none of which interpolate request input.

5.4.2 Output escaping, CSRF tokens and messages

```
function e($value)
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}

function csrf_token()
{
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(16));
    }
    return $_SESSION['csrf'];
}

function csrf_ok()
{
    return isset($_POST['csrf']) && hash_equals(csrf_token(), (string)
$_POST['csrf']);
}
```

The CSRF token is 128 bits from the operating system's cryptographic random source, stored in the session and rendered into forms by `csrf_field()`. `hash_equals()` compares in constant time, so response timing does not leak how many leading characters matched. `flash()` stores a

one-time message in the session and `render_flash()` prints and clears it, which is how a redirect can carry "Booking CNX-2026-00051 cancelled" to the next page.

5.4.3 Current user and login gate

`current_user_id()` returns the signed-in customer's id from the session, falling back to a lookup by email for sessions created by legacy pages. `require_login()` remembers the requested URL in `$_SESSION['after_login']` and redirects to sign-in. After a successful sign-in, `signin.php` returns the customer to that URL only if it is a local path (it must match `^/[^/\\]`), which prevents an open redirect to another site.

5.5 Shows and the seat map

5.5.1 Listing shows

`upcoming_shows()` returns a film's active shows for the next seven days, with the number of seats left computed in the same query:

```
SELECT s.*, c.cinema_name, c.cinema_location,
       s.total_rows * s.seats_per_row
         - (SELECT COUNT(*) FROM booking_seats b WHERE b.show_id = s.show_id) AS
seats_left
FROM shows s
JOIN cinema c ON c.cin_id = s.cin_id
WHERE s.movie_id = ? AND s.status = 'Active'
  AND TIMESTAMP(s.show_date, s.show_time) > NOW()
  AND s.show_date <= CURDATE() + INTERVAL ? DAY
ORDER BY s.show_date, s.show_time, c.cinema_name
```

`TIMESTAMP(show_date, show_time)` combines the date and the `HH:MM` text into a comparable instant, so a show at 13:00 today disappears from the list at 13:00. `bookable_show()` applies the same test to a single show and is called by every booking step, so a stale link to a past or cancelled show is rejected at each stage.

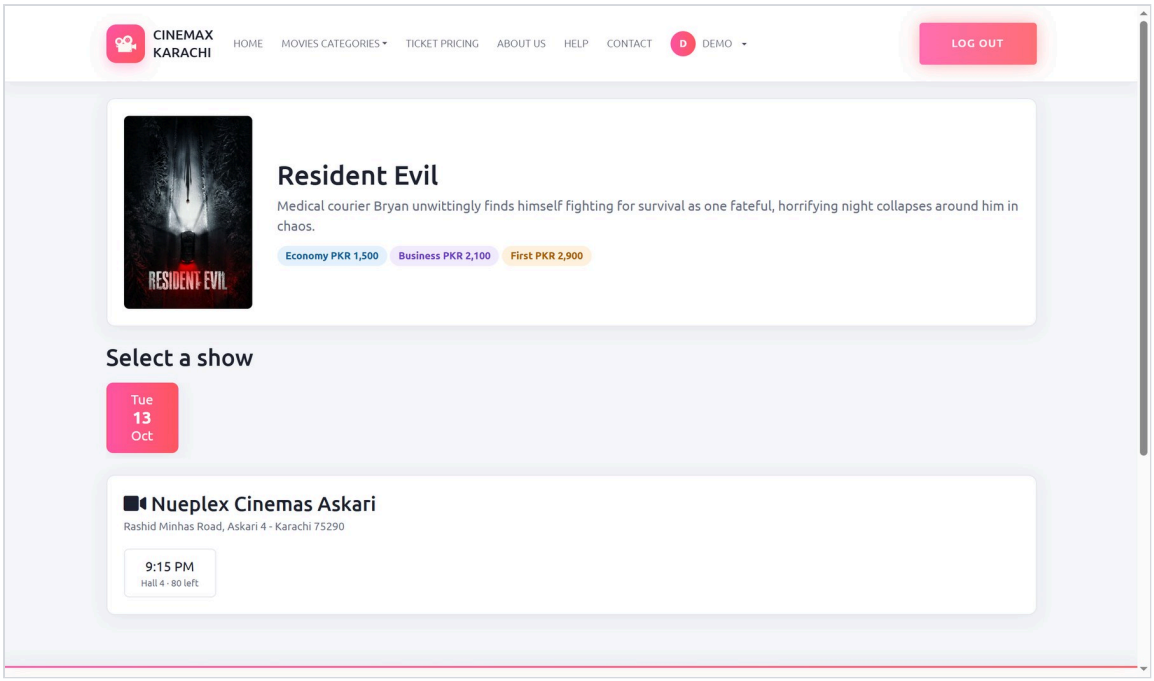


Figure 5.1: Show selection. Shows grouped by date and cinema, each with its hall and the number of seats left; the three class prices are shown at the top (FR-13).

5.5.2 Rendering the seat map

`seat_map.php` loads the show, the film's prices and the current seat states, then renders the hall row by row. Each seat is a button:

```
<button type="button" class="cx-seat c-<?php echo $class; ?> <?php echo $state; ?>"
        data-label="<?php echo $label; ?>" data-class="<?php echo $class; ?>"
        aria-pressed="false" <?php echo $disabled ? 'disabled' : ''; ?>>
  <?php echo $number; ?>
</button>
```

Seat state comes from `seat_states()`, which first deletes expired holds, then returns three lists: `booked` (labels in `booking_seats`), `held` (other customers' holds) and `mine` (this customer's holds, pre-selected on return from checkout).

```
function seat_states($show_id, $user_id)
{
    release_expired_holds();
    $booked = array_column(db_all('SELECT seat_label FROM booking_seats WHERE
show_id = ?', 'i', [(int) $show_id]), 'seat_label');
    $held = [];
    $mine = [];
    foreach (db_all('SELECT seat_label, user_id FROM seat_holds WHERE show_id = ?',
'i', [(int) $show_id]) as $hold) {
        if ((int) $hold['user_id'] === (int) $user_id) {
```

```
            $mine[] = $hold['seat_label'];
        } else {
            $held[] = $hold['seat_label'];
        }
    }
    return ['booked' => $booked, 'held' => $held, 'mine' => $mine];
}
```

5.5.3 Client-side selection and polling

The seat map's JavaScript keeps the selection in the DOM (the `is-selected` class) and recomputes the per-class breakdown and total on every change, using the prices embedded by PHP as JSON. It enforces the ten-seat limit before the server does. Every ten seconds it fetches `ajax/seat_status.php` and applies the result:

```
function applyStates(data) {
  var lost = [];
  seats().forEach(function (s) {
    var label = s.dataset.label;
    var taken = data.booked.indexOf(label) !== -1 ? 'is-booked'
              : (data.held.indexOf(label) !== -1 ? 'is-held' : '');
    if (taken && s.classList.contains('is-selected')) lost.push(label);
    s.classList.remove('is-booked', 'is-held');
    if (taken) { setSelected(s, false); s.classList.add(taken); }
    s.disabled = !!taken;
  });
  if (lost.length) notify('Seat ' + lost.join(', ') + ' was just taken by someone
else. Please choose another seat.');
  refresh();
}
```

Polling is a usability feature, not a correctness mechanism. A customer may still press Continue on a seat taken in the last ten seconds; the server then refuses the hold and names the seat.

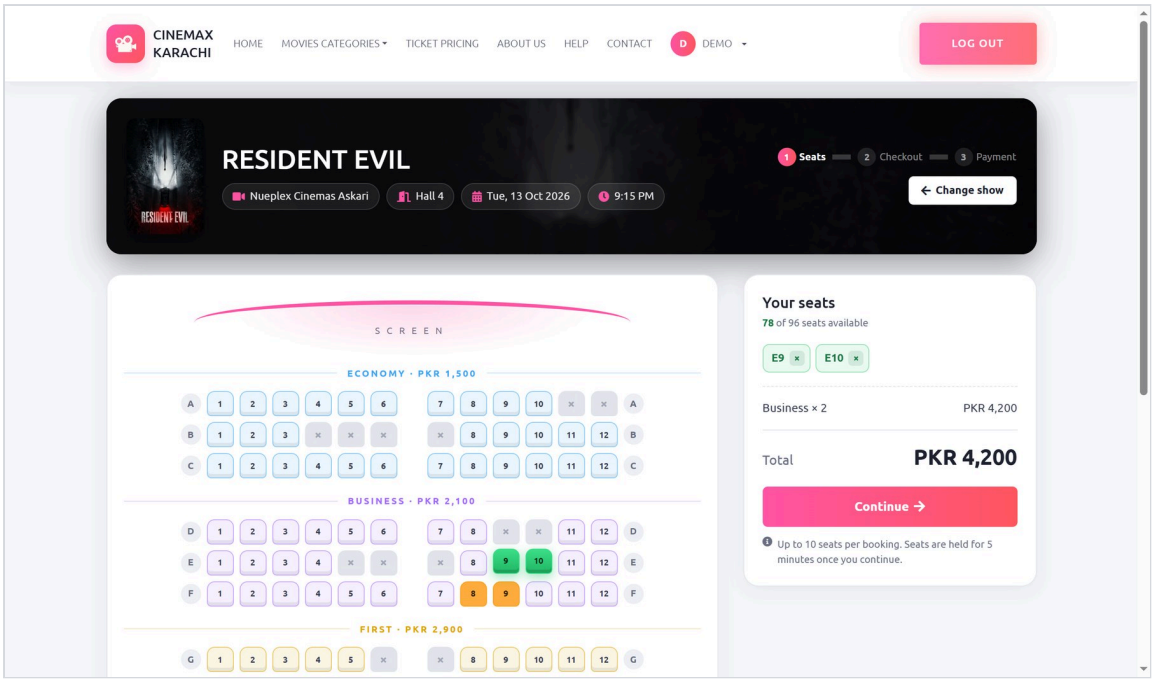


Figure 5.2: Seat map. Economy, Business and First rows with their prices. Sold seats are greyed out with a cross, F8 and F9 are orange because another customer is holding them, and E9 and E10 are green because this customer has selected them. The summary panel totals the selection before the seats are held (FR-14 to FR-19).

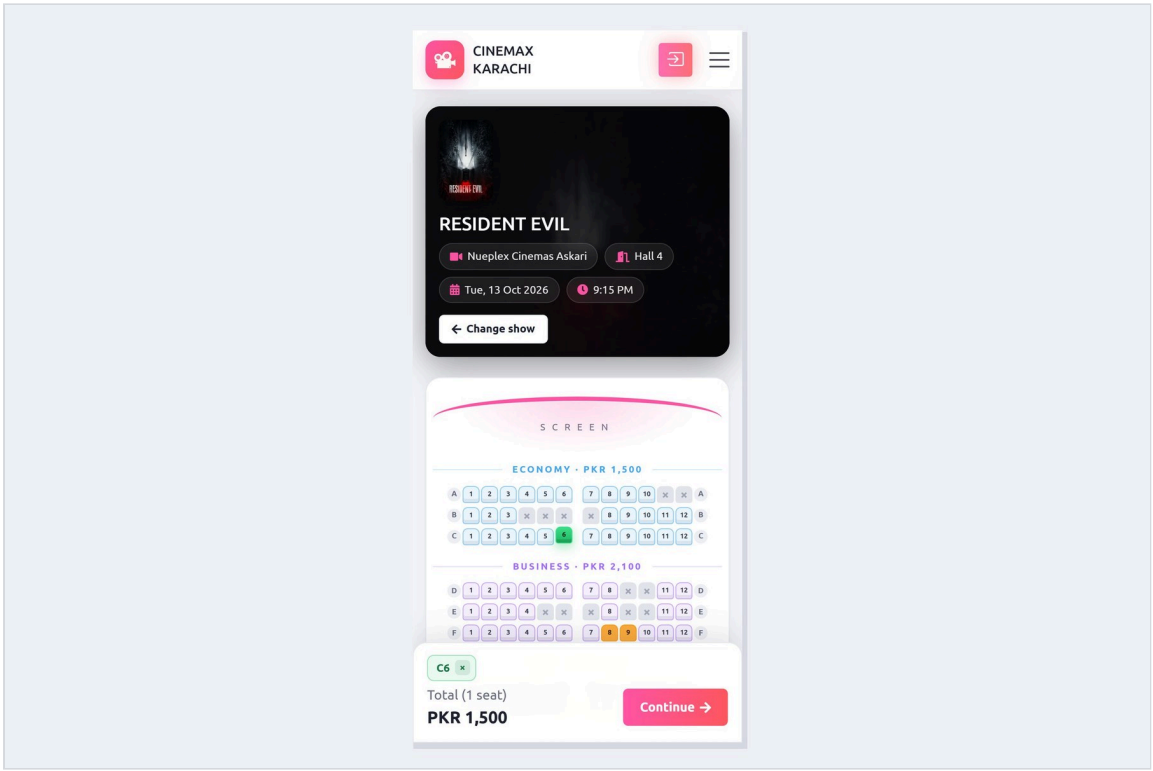


Figure 5.3: Seat map on a phone. On a 390-pixel-wide screen the seats shrink to fit and the selected seats, total and Continue button move to a bar fixed at the bottom of the screen.

5.6 Seat holds

5.6.1 The hold endpoint

`ajax/seat_hold.php` accepts a POST with `show_id` and a comma-separated `seats` list. It validates in order and stops at the first failure, returning an HTTP status that describes the problem:

Check	Failure response
Customer signed in	401 "Your session has ended. Please sign in again."
Method is POST	405
Show exists, is Active and has not started	404 "This show is no longer available for booking."
Prices are set for the film	422
At least one seat; at most `MAX_SEATS` (10)	422
Every label is valid for this hall	422 "Seat Z99 does not exist in this hall."
All seats can be held	409 with the list of conflicting seats

On success it returns `{"ok": true, "expires_in": 300, "redirect": "checkout.php?show_id=..."}`. The same endpoint with `action=release` drops the customer's holds.

5.6.2 The hold transaction

```
function hold_seats($show_id, $user_id, $labels)
{
    release_expired_holds();
    mysqli_begin_transaction(db());
    db_query('DELETE FROM seat_holds WHERE user_id = ?', 'i', [(int) $user_id]);
    $conflicts = [];
    foreach ($labels as $label) {
        $booked = db_one('SELECT 1 FROM booking_seats WHERE show_id = ? AND
seat_label = ?', 'is', [(int) $show_id, $label]);
        $inserted = $booked ? false : db_query(
            'INSERT INTO seat_holds (show_id, seat_label, user_id, expires_at)
VALUES (?, ?, ?, NOW() + INTERVAL ' . HOLD_MINUTES . ' MINUTE)',
            'isi',
            [(int) $show_id, $label, (int) $user_id]
        );
        if (!$inserted) {
            $conflicts[] = $label;
        }
    }
    if ($conflicts) {
```

```
            mysqli_rollback(db());
            return $conflicts;
        }
        mysqli_commit(db());
        return [];
    }
```

Three properties of this function matter.

1. **One active selection per customer.** Deleting the customer's existing holds first means a customer who goes back and picks different seats does not keep the old ones locked.
2. **The unique key decides contention.** If two customers race for seat H12, both may pass the `booking_seats` check, but only one `INSERT INTO seat_holds` can succeed, because of `UNIQUE (show_id, seat_label)`. The loser's insert fails, `db_query()` returns `false`, and the seat is reported as a conflict.
3. **All or nothing.** Any conflict rolls back the whole transaction, including the deletion of the customer's earlier holds, so a failed attempt leaves the customer exactly where they were.

5.6.3 Expiry

Holds are never extended. `release_expired_holds()` runs `DELETE FROM seat_holds WHERE expires_at <= NOW()` at the start of every seat read and every hold attempt, using the `idx_hold_expiry` index. A hold that expires therefore frees its seat for the very next reader, without a scheduler. The checkout and payment pages show the remaining time, computed by SQL as `TIMESTAMPDIFF(SECOND, NOW(), expires_at)`, through the `partials/hold_timer.php` countdown, which returns the customer to the seat map when it reaches zero.

5.7 Checkout

5.7.1 Checkout state

`load_checkout()` assembles everything checkout needs and returns `null` if the customer has no live holds, in which case the page redirects back to the seat map. The customer's snack quantities and promo code are kept in `$_SESSION['checkout']`, keyed to the show, so they survive a round trip between checkout and payment but are discarded if the customer switches to another show.

5.7.2 Totals

All prices are recomputed on the server from the database on every request; the browser's totals are only a preview.

```
function checkout_totals($prices, $seat_labels, $snack_qty, $promo)
{
    $seats = priced_seats($seat_labels, $prices);
    $tickets = array_sum(array_column($seats, 'price'));
    ...
    $subtotal = $tickets + $snacks_amount;
    $discount = $promo ? (int) floor($subtotal * (int) $promo['discount_percent'] /
100) : 0;
    return [
        'seats' => $seats,
        'tickets_amount' => $tickets,
        'snacks' => $snacks,
        'snacks_amount' => $snacks_amount,
        'subtotal' => $subtotal,
        'discount' => $discount,
        ...
        'total' => $subtotal - $discount,
    ];
}
```

Snack quantities pass through `clean_snack_qty()`, which clamps each quantity to 0 to 10 and discards ids that are not active snacks, so a tampered form cannot order an inactive snack or a negative quantity. Flooring the discount keeps the total a whole number of rupees; any fractional rupee is rounded against the discount, so the cinema never gives away more than the advertised percentage.

A worked example from the end-to-end test in Chapter 7: seats E6 and E7 (Business, PKR 1,400 each) and G3 (First, PKR 2,000) give tickets of PKR 4,800; one Soft Drink adds PKR 250; the subtotal is PKR 5,050; `EID20` takes floor(5,050 × 0.20) = PKR 1,010; the total is PKR 4,040.

5.7.3 Promotional codes

```
function find_promo($code, $lock = false)
{
    $code = strtoupper(trim((string) $code));
    if ($code === '') {
        return [null, 'Please enter a promo code.'];
    }
    $promo = db_one('SELECT * FROM promo_codes WHERE code = ?' . ($lock ? ' FOR
UPDATE' : ''), 's', [$code]);
    if (!$promo || !(int) $promo['is_active']) {
        return [null, 'This promo code is not valid.'];
```

```
    }
    if ($promo['valid_until'] !== null && $promo['valid_until'] < date('Y-m-d')) {
        return [null, 'This promo code has expired.'];
    }
    if ((int) $promo['max_uses'] > 0 && (int) $promo['used_count'] >= (int)
$promo['max_uses']) {
        return [null, 'This promo code has reached its usage limit.'];
    }
    return [$promo, null];
}
```

Codes are stored and compared in upper case, so `eid20` and `EID20` are the same code. The `$lock` parameter is used at payment time: inside the booking transaction the promo row is read `FOR UPDATE`, so two customers redeeming the last remaining use of a limited code are serialised, and the second sees the exhausted count.

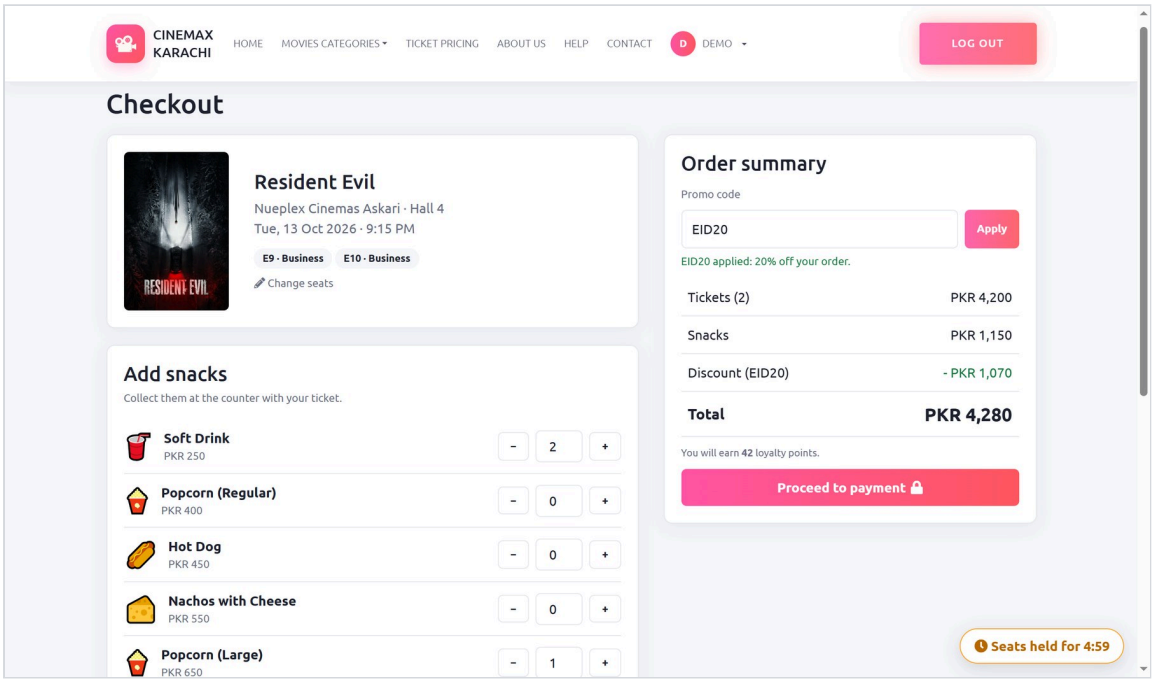


Figure 5.4: Checkout. Snacks are added with quantity buttons and promo code EID20 takes 20% off; the badge at the bottom right counts down the remaining hold time (FR-20 to FR-22).

5.8 Payment simulation

Real payment processing was out of scope, but the payment step was built to behave like one so that the booking transaction is exercised realistically and the integration point for a real gateway is clear.

Card. The browser formats the card number in groups of four and the expiry as `MM/YY`, and checks the format before submitting. The server repeats every check: the number must have 16 digits, the name must be present, the expiry must match `^(0[1-9]|1[0-2])/([0-9]{2})$` and be the current month or later, and the CVV must be 3 or 4 digits. Only `substr($number, -4)` is passed on to the booking. The CVV field is a password input and the CVV is never stored.

JazzCash and Easypaisa. The customer enters a Pakistani mobile number, which must match `^03[0-9]{9}$`, and requests a one-time password. `ajax/payment_otp.php` generates a six-digit code with `random_int(100000, 999999)` and stores it in the session with the mobile number and a five-minute expiry. In this demonstration build the endpoint also returns the code in a "demo SMS" message, standing in for an SMS gateway. At payment the server checks that an OTP was issued for the same number, has not expired, and matches using `hash_equals()`.

```
if (!$sent || $sent['mobile'] !== $mobile || $sent['expires'] < time()) {
    return [null, 'Please request an OTP for this mobile number first.'];
}
if (!hash_equals($sent['code'], $otp)) {
    return [null, 'The OTP you entered is incorrect.'];
}
return [substr($mobile, -4), null];
```

Submission. On submit the page disables the button, shows a "Processing payment..." overlay, and posts the form after 2.5 seconds, imitating a gateway round trip. A JavaScript flag prevents a double submission; on the server, a second submission would find no holds and fail safely, because the first one deleted them.

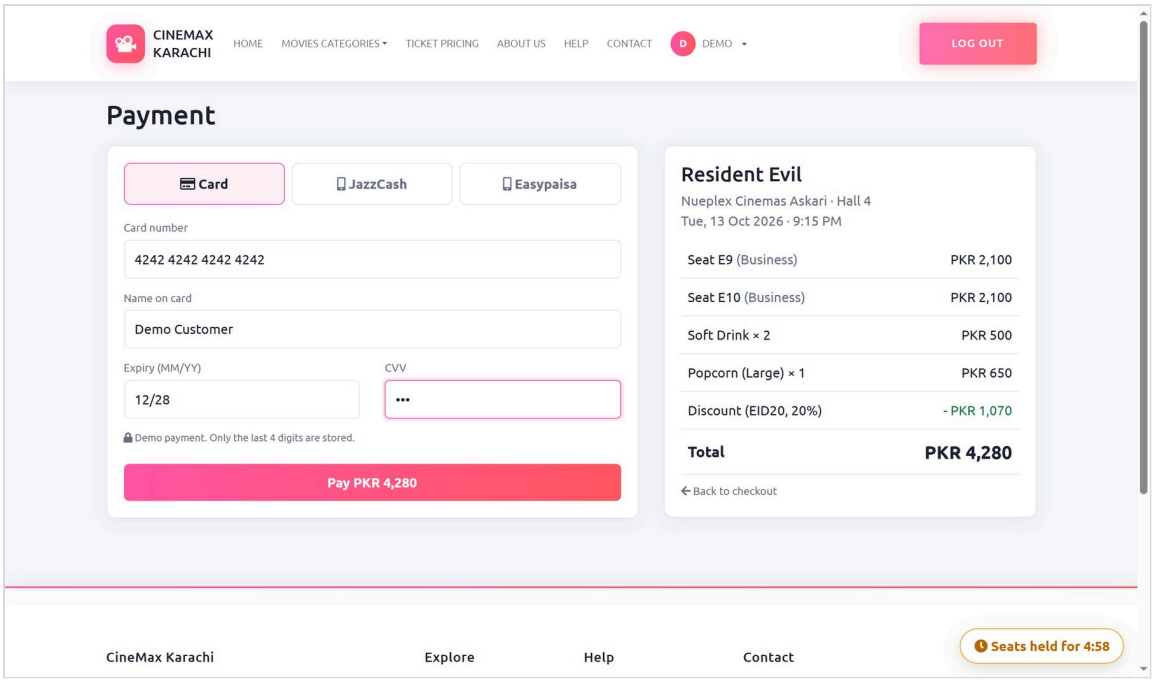


Figure 5.5: Payment. Card, JazzCash and Easypaisa tabs with live formatting and the order summary; only the last four digits are stored (FR-23, FR-24).

5.9 Booking finalisation

`finalize_booking()` is the most important function in the system. It turns a customer's holds into a confirmed booking in a single transaction, and every failure path rolls back.

```
function finalize_booking($user, $show, $snack_qty, $promo_code, $method,
$account_last4)
{
    $db = db();
    mysqli_begin_transaction($db);
    $fail = function ($message) use ($db) {
        mysqli_rollback($db);
        return ['ok' => false, 'message' => $message];
    };

    $holds = db_all(
        'SELECT seat_label FROM seat_holds WHERE user_id = ? AND show_id = ? AND
expires_at > NOW() FOR UPDATE',
        'ii',
        [(int) $user['id'], (int) $show['show_id']]
    );
    if (!$holds) {
        return $fail('Your seat hold has expired. Please select your seats again.');
    }
    $prices = movie_prices($show['movie_name']);
    ...
```

```
    if ($promo_code) {
        list($promo, $promo_error) = find_promo($promo_code, true);
        if (!$promo) {
            return $fail($promo_error . ' Please go back to checkout and remove
it.');
        }
    }

    $labels = array_column($holds, 'seat_label');
    sort($labels, SORT_NATURAL);
    $t = checkout_totals($prices, $labels, $snack_qty, $promo);

    $inserted = db_query("INSERT INTO booking_details (...) VALUES (..., 'Booked',
'None')", ...);
    if (!$inserted) {
        return $fail('Could not save your booking. Please try again.');
    }
    $booking_id = mysqli_insert_id($db);
    $ref = generate_booking_ref($booking_id);
    db_query('UPDATE booking_details SET booking_ref = ? WHERE booking_id = ?',
'si', [$ref, $booking_id]);

    foreach ($t['seats'] as $seat) {
        $ok = db_query(
            'INSERT INTO booking_seats (booking_id, show_id, seat_label, seat_class,
price) VALUES (?, ?, ?, ?, ?)',
            'iissi',
            [$booking_id, (int) $show['show_id'], $seat['label'], $seat['class'],
$seat['price']]
        );
        if (!$ok) {
            return $fail('Seat ' . $seat['label'] . ' was just booked by someone
else. Please select your seats again.');
        }
    }
    ...
    if ($promo) {
        db_query('UPDATE promo_codes SET used_count = used_count + 1 WHERE promo_id
= ?', 'i', [(int) $promo['promo_id']]);
    }
    db_query('UPDATE user_reg_login SET loyalty_points = loyalty_points + ? WHERE id
= ?', 'ii', [loyalty_points_for($t['total']), (int) $user['id']]);
    db_query('DELETE FROM seat_holds WHERE user_id = ?', 'i', [(int) $user['id']]);

    if (!mysqli_commit($db)) {
        return $fail('Could not complete your booking. Please try again.');
    }
    return ['ok' => true, 'booking_id' => $booking_id, 'booking_ref' => $ref];
}
```

The design choices in this function are:

- **Holds are the source of truth for which seats are being bought.** The seats are re-read from `seat_holds` inside the transaction, not taken from the form. A customer cannot add seats to the order by editing the request.
- **Prices and totals are recomputed inside the transaction.** If an administrator changes a price between checkout and payment, the customer pays the price at the moment of commit, and the ticket shows that price.
- **Two locks, one invariant.** `FOR UPDATE` on the holds and on the promo row serialises concurrent attempts by the same customer and concurrent redemptions of a limited code. The `booking_seats` unique key remains the final guarantee that no seat is sold twice.
- **The reference is derived from the auto-increment id** inside the same transaction, so it is unique and gap-free per booking without a separate sequence. `generate_booking_ref(51)` in 2026 yields `CNX-2026-00051`.
- **The booking row stores snapshots** of the customer name and email, film, cinema, date, time, seat labels and class summary, as described in Section 4.4.6.

`payment.php` calls `finalize_booking()` after validating the payment details. On success it clears the checkout state and OTP from the session and redirects to `booking_success.php?ref=...`. On failure it re-checks whether the holds still exist: if so the payment form is shown again with the error, otherwise the customer is sent back to the seat map.

5.10 E-ticket and QR code

`ticket.php` and `booking_success.php` share one implementation (the latter sets a flag that adds a "Booking confirmed!" header and the loyalty points earned). Both load the booking with `load_booking()`, which joins the show and cinema and attaches the seat and snack lines, and both refuse to render unless `owns_booking()` confirms that the booking belongs to the signed-in customer. A customer who guesses another booking's reference sees "Ticket not found", the same message as for a reference that does not exist, so the response does not reveal which references are valid.

The ticket card is a partial (`partials/ticket_card.php`) that renders the film poster, reference, cinema, hall, date, time, seats with classes, snacks, discount, total, masked payment details (for example `Card ****4242`) and the QR code. The QR code is drawn in the browser:

```
<div class="tk-qr" id="ticket-qr" data-code="<?php echo
e(ticket_verify_url($ticket['booking_ref'])); ?>"></div>
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js">
</script>
<script>
  new QRCode(box, { text: box.dataset.code, width: 170, height: 170, correctLevel:
QRCode.CorrectLevel.M });
</script>
```

A print stylesheet (`@media print`) in the same partial hides the site navigation, so "Print / Save as PDF" produces a ticket on its own. A scanned or cancelled ticket carries a translucent USED or CANCELLED stamp across the card.

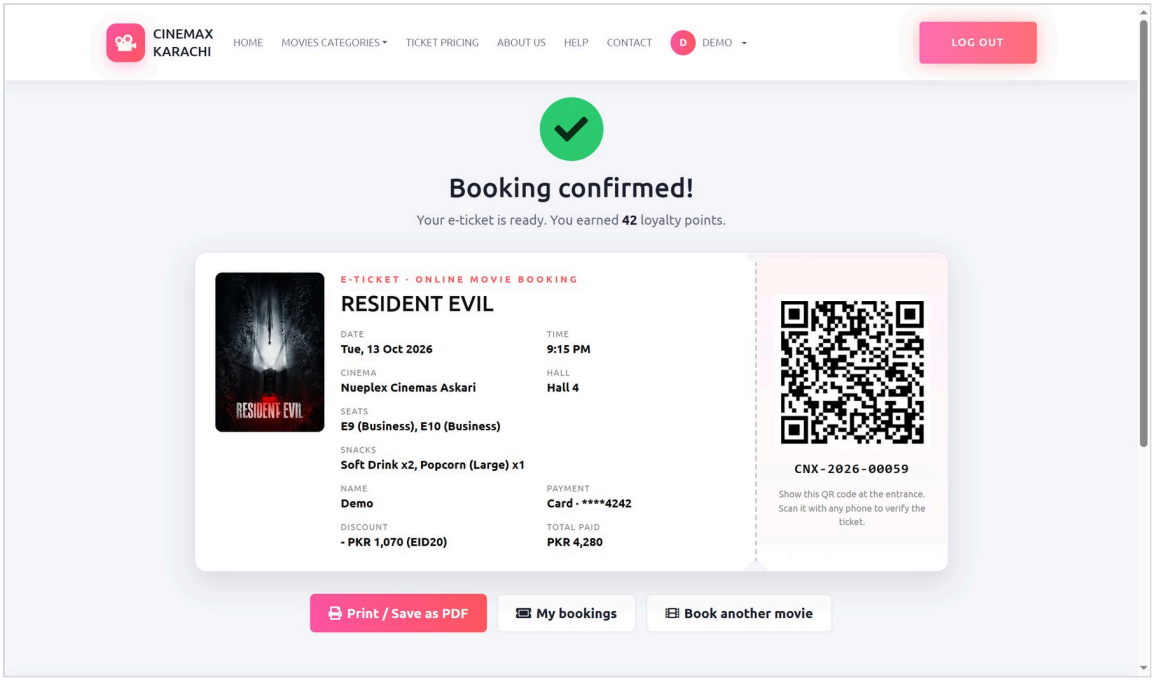


Figure 5.6: Booking confirmation and e-ticket. The ticket lists the show, hall, seats, snacks, discount and payment, and carries the signed QR code described in Section 5.10.1 (FR-27 to FR-29).

5.10.1 Signed QR codes and public verification

An early version of the ticket encoded only the booking reference, for example `CNX-2026-00051` . The door scanner could look that reference up, but any other phone showed just the text, and nothing on the ticket proved that the cinema had issued it: anyone could print a QR code containing a made-up reference. The final version makes the QR code itself verifiable.

The QR code now encodes a link to the public verification page, carrying the reference and a signature:

```
https://<site>/verify.php?ref=CNX-2026-00059&sig=f20c1c621501b1df8fe48130
```

The signature is a keyed hash (HMAC-SHA256) of the reference, computed with a 64-character secret that exists only on the server (`ticket_secret` in `includes/secrets.php`, or the `TICKET_SECRET` environment variable), and shortened to 24 hexadecimal characters (96 bits):

```
function ticket_signature($ref)
{
    return substr(hash_hmac('sha256', strtoupper(trim((string) $ref)),
ticket_secret()), 0, 24);
}

function ticket_signature_ok($ref, $sig)
{
    return is_string($sig) && $sig !== '' && hash_equals(ticket_signature($ref),
strtolower($sig));
}
```

Without the secret, nobody can produce a valid signature for a reference, and changing any character of the reference invalidates the signature. `hash_equals()` compares in constant time, so response timing reveals nothing about how close a guess was.

Scanning the QR code with any phone camera opens `verify.php`, which shows one of five verdicts: **Genuine ticket**, **Already used** (with the check-in time), **Cancelled ticket**, **Show date has passed**, or **Not a valid CineMax ticket**. For a genuine ticket it shows the movie, cinema, hall, date, time, booking reference and seats with their classes. The customer's name is masked ("Ayesha Khan" becomes "A***** K***") and no email address, phone number or price is shown. The page reveals nothing when the signature is missing or wrong, so booking references cannot be enumerated by guessing. Verification is read-only: only the admin scanner can admit a ticket.

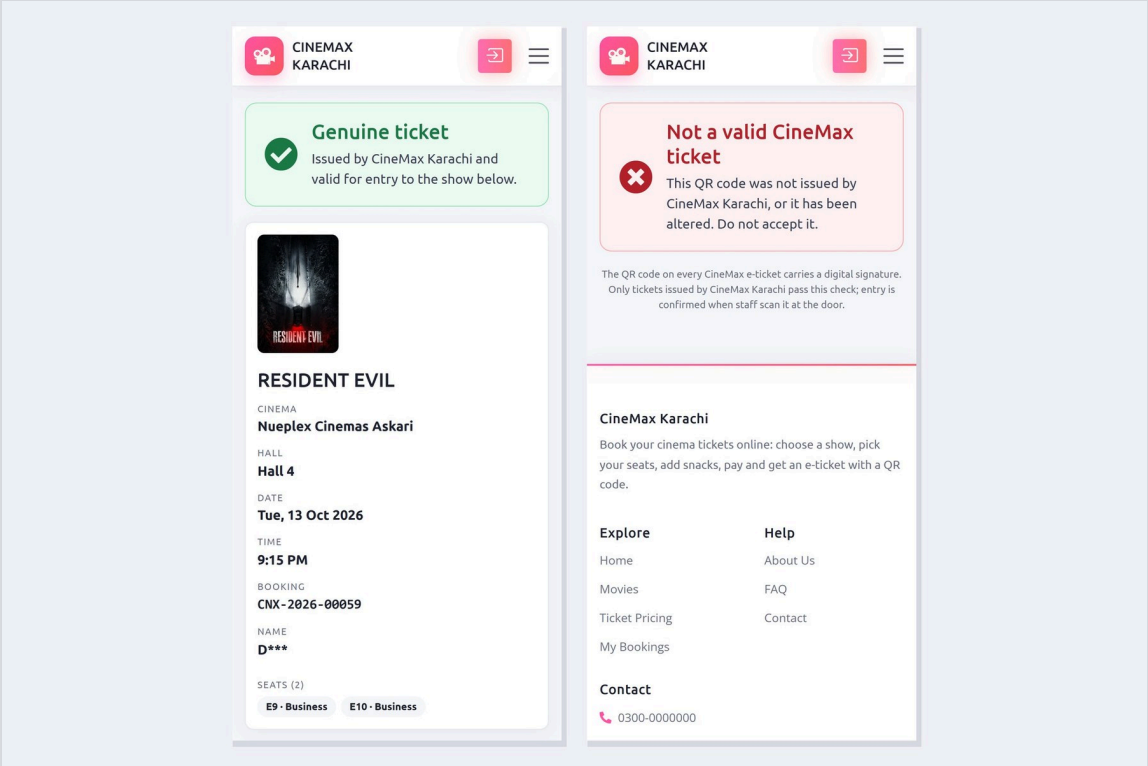


Figure 5.7: Verifying a ticket with an ordinary phone camera. Left: the QR code of a genuine ticket opens the verification page, with the customer's name masked. Right: the same link with one character of the signature changed is rejected and reveals nothing about the booking.

5.11 Booking history and cancellation

`userbooking.php` lists the customer's bookings in three tabs. *Upcoming* contains Booked bookings whose show has not started, *Past* contains Used bookings and bookings whose show has started, and *Cancelled* contains cancelled bookings with their refund status and, for rejected refunds, the administrator's reason. The customer's loyalty-point balance is shown above the tabs.

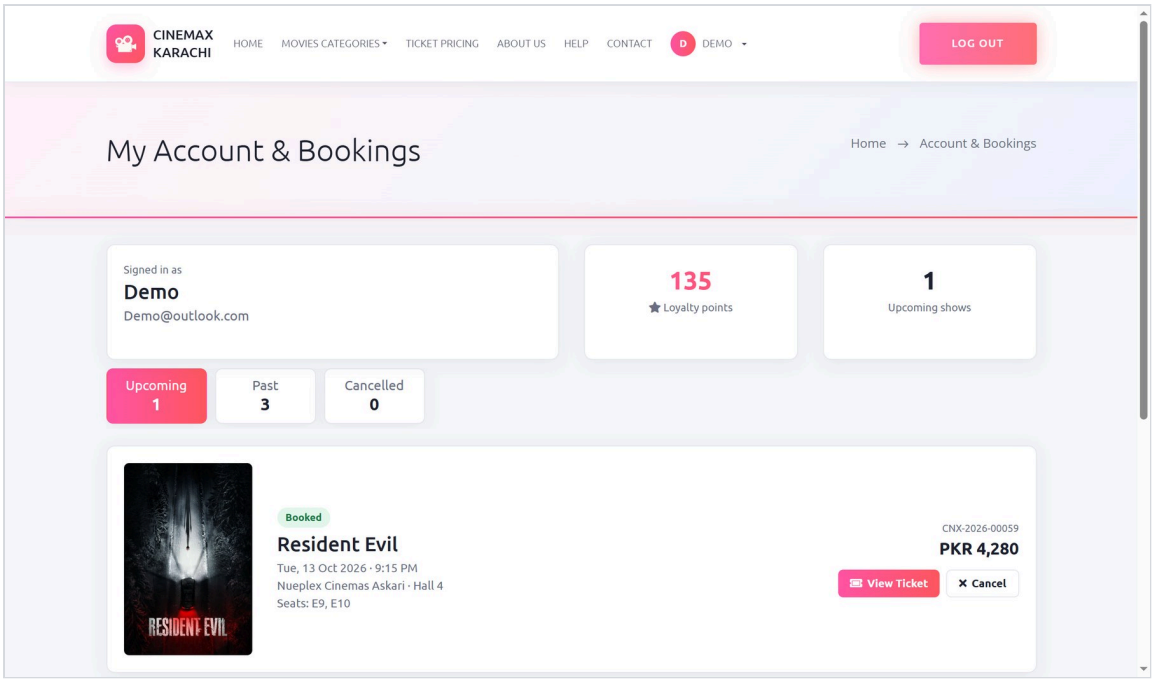


Figure 5.8: My Bookings. Loyalty points, Upcoming, Past and Cancelled tabs, and View Ticket and Cancel actions for each booking (FR-30, FR-32).

The refund rule is a pure function of the booking and the current time:

```
function refund_quote($booking)
{
    if ($booking['status'] !== 'Booked' || !$booking['starts_at']) {
        return null;
    }
    $seconds = strtotime($booking['starts_at']) - time();
    if ($seconds <= 0) {
        return null;
    }
    $percent = $seconds >= 86400 ? 100 : 50;
    return ['percent' => $percent, 'amount' => (int) floor((int)
$booking['total_amount'] * $percent / 100)];
}
```

The Cancel button opens a SweetAlert2 dialog showing the quote (for example "Refund: PKR 500 (50%)"). Confirming posts the reference and CSRF token to `cancel_booking.php`, which re-checks the token and ownership and then calls `cancel_booking()`:

```
mysqli_begin_transaction($db);
$updated = db_query(
    "UPDATE booking_details SET status = 'Cancelled', refund_status = 'Pending',
refund_amount = ?, cancelled_at = NOW()
     WHERE booking_id = ? AND status = 'Booked'",
```

```
    'ii',
    [$quote['amount'], (int) $booking['booking_id']]
);
if ($updated !== 1) {
    mysqli_rollback($db);
    return ['ok' => false, 'message' => 'This booking can no longer be cancelled.'];
}
db_query('DELETE FROM booking_seats WHERE booking_id = ?', 'i', [(int)
$booking['booking_id']]);
db_query('UPDATE user_reg_login SET loyalty_points = GREATEST(loyalty_points - ?, 0)
WHERE id = ?', ...);
mysqli_commit($db);
```

Deleting the `booking_seats` rows returns the seats to sale immediately; the next seat-map poll shows them as free. The booking row itself is kept, with its snapshot, for the customer's history and for the reports.

5.12 Ticket scanner

The admin scanner (`Admin/scan_ticket.php`) uses the html5-qrcode library to read QR codes from the device camera, preferring the rear camera (`facingMode: 'environment'`) at 10 frames per second with a 220-pixel scanning box. A manual-entry field accepts a typed reference or a pasted link for damaged codes. Both paths POST the raw scanned text with the CSRF token. The server parses it with `parse_ticket_code()`, which extracts the reference and, when present, the signature, and then runs `check_in_ticket()`:

```
list($ref, $sig) = parse_ticket_code($raw);
if ($sig !== null && !ticket_signature_ok($ref, $sig)) {
    log_activity('Rejected ticket with invalid QR signature', $ref);
    return ['result' => 'FORGED', 'ref' => $ref];
}
$b = load_booking($ref);
if (!$b) {
    return ['result' => 'NOT_FOUND', 'ref' => $ref];
}
$show_today = !$b['show_date'] || $b['show_date'] === date('Y-m-d');
if ($b['status'] === 'Booked' && !$show_today) {
    $result = 'WRONG_DAY';
} else {
    $updated = db_query(
        "UPDATE booking_details SET status = 'Used', checked_in_at = NOW() WHERE
booking_ref = ? AND status = 'Booked'",
        's',
        [$ref]
    );
    $b = load_booking($ref);
```

```
    $result = $updated === 1 ? 'VALID' : ($b['status'] === 'Cancelled' ? 'CANCELLED'
: 'ALREADY_USED');
}
```

The scanner therefore distinguishes six outcomes:

Result	Meaning	Ticket changed?
VALID	Genuine ticket for a show today; checked in now	Booked → Used
ALREADY USED	Genuine ticket that was checked in before	No
CANCELLED	The booking was cancelled	No
WRONG DAY	Genuine, unused ticket for a show on another day	No
NOT GENUINE	The QR signature does not match the reference (forged or altered code); the attempt is written to the activity log	No
NOT FOUND	No booking with this reference	No

The check-in itself is a single conditional `UPDATE`, so the decision and the state change are atomic: two door staff scanning the same ticket at the same moment cannot both receive VALID. Refusing wrong-day tickets without checking them in means a customer who arrives on the wrong day keeps a usable ticket. The result panel lists the movie, cinema, hall, show time and seats (with a count), so staff can confirm that the customer is at the right hall, and a green note confirms when the QR signature was verified. A typed reference, or a ticket issued before signing was introduced, is still accepted, flagged as "QR signature not checked". Browsers allow camera access only in a secure context, so the camera works on `https://` or `localhost` origins; manual entry works everywhere.

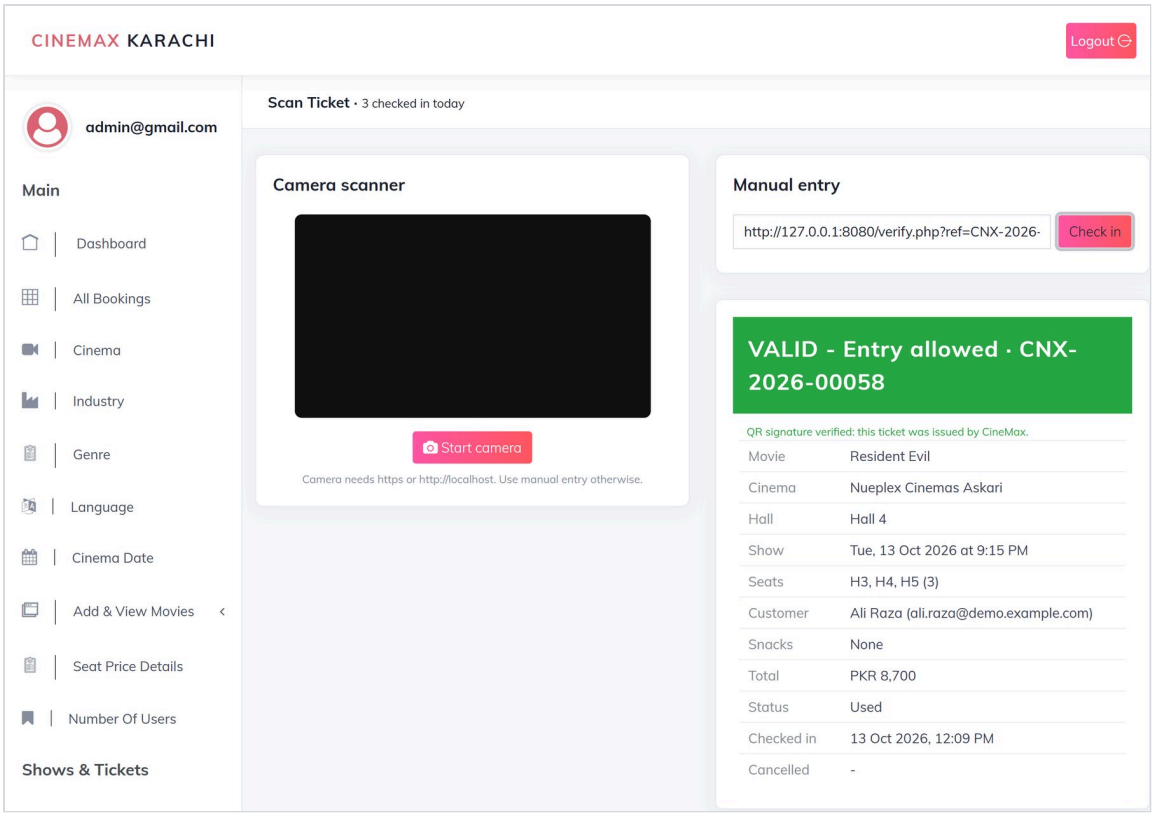


Figure 5.9: Ticket scanner, valid ticket. The signed link is verified, the ticket is checked in, and the panel shows the movie, hall, show time and seats so staff can confirm the customer is at the right door (FR-44).

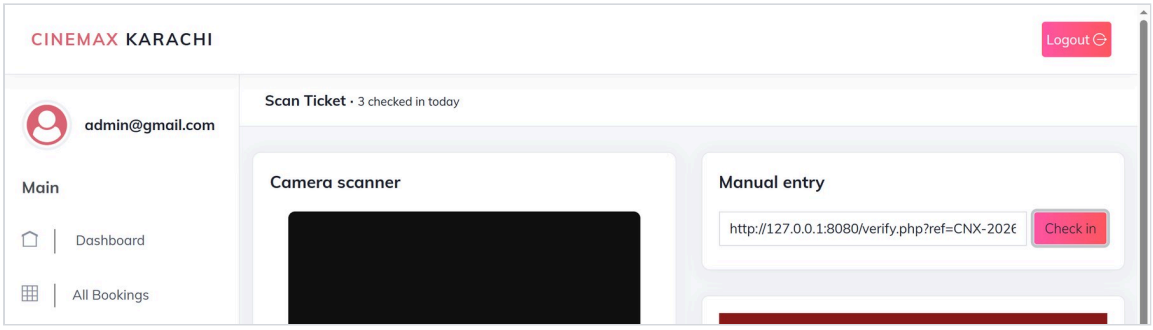


Figure 5.10: Ticket scanner, forged ticket. A QR code whose signature does not match its reference is rejected and recorded in the activity log.

5.13 Administration panel

5.13.1 Authentication and access control

`Admin/login.php` validates the administrator's email and password with a prepared statement before any output, stores the email in `$_SESSION['admin']` and redirects to the dashboard. Every other admin page, including every `*_del.php` handler, starts with:

```
<?php
include('auth.php');
```

`auth.php` starts the session and, if `$_SESSION['admin']` is empty, redirects to `login.php` and calls `exit` before the page can read or change anything.

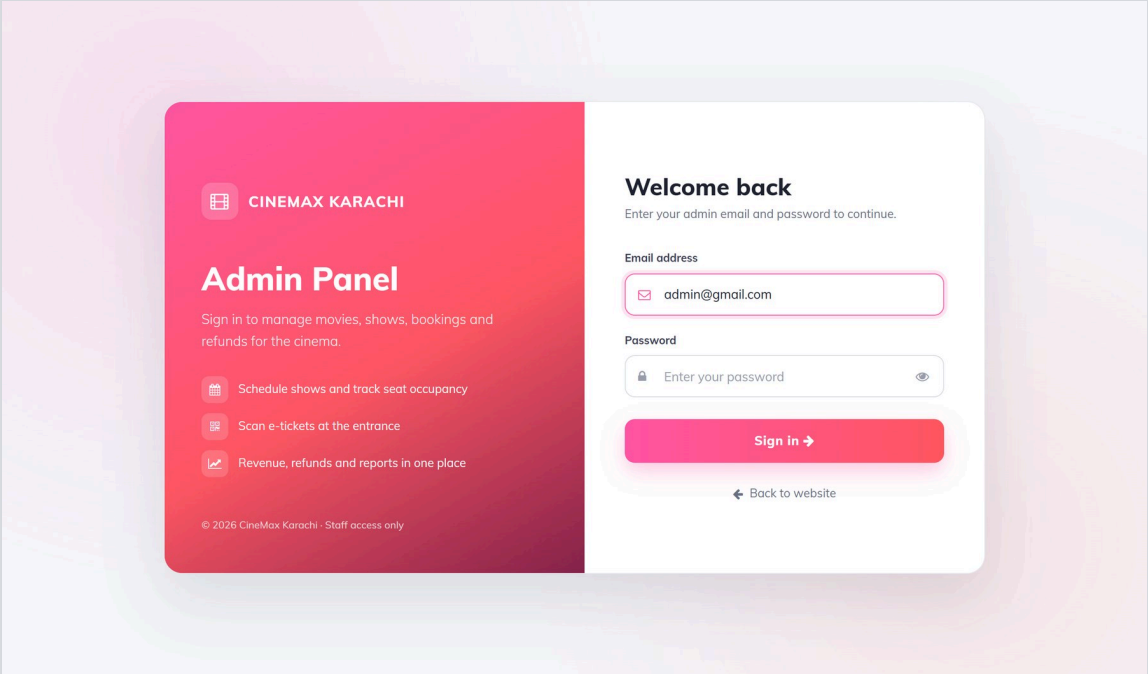


Figure 5.11: Admin login. A separate sign-in for cinema staff, with a show/hide password control (FR-37).

5.13.2 Catalogue management

The catalogue pages (cinema, genre, industry, language, release date, films, seat prices, snacks, promo codes, coming soon) follow a common pattern: a form to add, a table to list with edit and delete actions, CSRF-protected submissions, flash messages, and an activity-log entry for each change. Two integrity rules are handled explicitly:

- **Referenced rows cannot be deleted.** Deleting a cinema or film that still has shows violates a foreign key. Each delete handler catches the failure and explains it ("still used by ... Delete those first") instead of showing a database error. A snack that has been sold is still needed by old tickets, so the admin is told to deactivate it instead.
- **Price edits are scoped.** The baseline's price-edit `UPDATE` had no `WHERE` clause and overwrote every film's prices; it now updates only the selected `price_id`, and validates that prices are whole numbers.

5.13.3 Show scheduling

`Admin/shows.php` validates that the film and cinema exist, the date is valid, the time is `HH:MM`, the hall has 1 to 12 rows of 1 to 20 seats, and a new show lies in the future. Before saving, it checks for a clash:

```
SELECT show_id FROM shows
WHERE cin_id = ? AND hall_name = ? AND show_date = ? AND show_time = ? AND show_id
<> ?
```

A show with booked seats cannot be deleted ("Set its status to Cancelled instead") and its seat layout cannot be changed, because existing seat labels would no longer match the hall. The list shows each show's occupancy as a percentage bar.

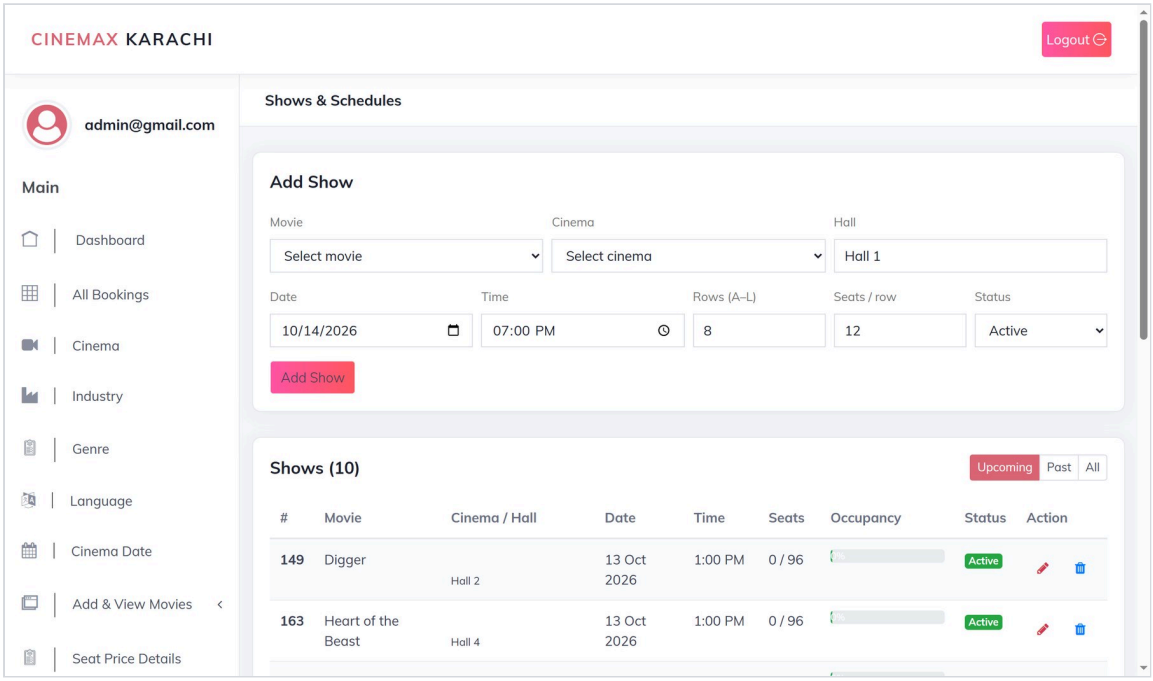


Figure 5.12: Shows and schedules. The Add Show form, which checks for hall clashes, and the list of upcoming shows with occupancy bars (FR-40).

5.13.4 Image uploads

All image uploads, including film posters, snack images, coming-soon posters and customer profile photos, pass through one function, `admin_upload_image()`:

1. Reject upload errors and files larger than 2 MB.
2. Allow only the extensions `jpg`, `jpeg`, `png` and `webp`.

3. Call `getimagesize()` on the uploaded file and require that the detected image type matches the extension, so a PHP script renamed to `.jpg` is rejected.
4. Store the file under a random name, `bin2hex(random_bytes(8))` plus the extension, so the uploader cannot choose or overwrite a file name.

The companion `admin_delete_upload()` deletes a file only if its name matches the random-name pattern, so TMDB posters and template images are never removed by accident. The profile-photo directory additionally contains an `.htaccess` file that denies execution of `.php`, `.phtml`, `.phar`, `.pl`, `.py`, `.cgi` and `.sh` files and disables directory listing.

5.13.5 Users, reviews, messages and the activity log

Administrators can block or unblock customers. A blocked customer cannot sign in ("Your account has been blocked. Please contact support for help."), and a customer who is blocked while signed in is signed out on the next page load, because `header.php` calls `customer()`, which clears the session for blocked accounts. Reviews can be hidden or shown; hidden reviews are excluded from the film's average rating. Contact messages show an unread badge in the admin header.

`log_activity()` records the administrator's email, an action and details for every add, edit and delete across the catalogue, shows, refunds, check-ins and seeding. The activity-log page lists entries newest first, 20 per page, with search.

5.13.6 Refund processing

`Admin/refunds.php` lists refunds in Pending, Refunded and Rejected tabs. Approval and rejection are conditional updates guarded by `refund_status = 'Pending'`, so a refund cannot be processed twice, even from two browser tabs. A rejection requires a reason, stored in `refund_reason` and shown to the customer.

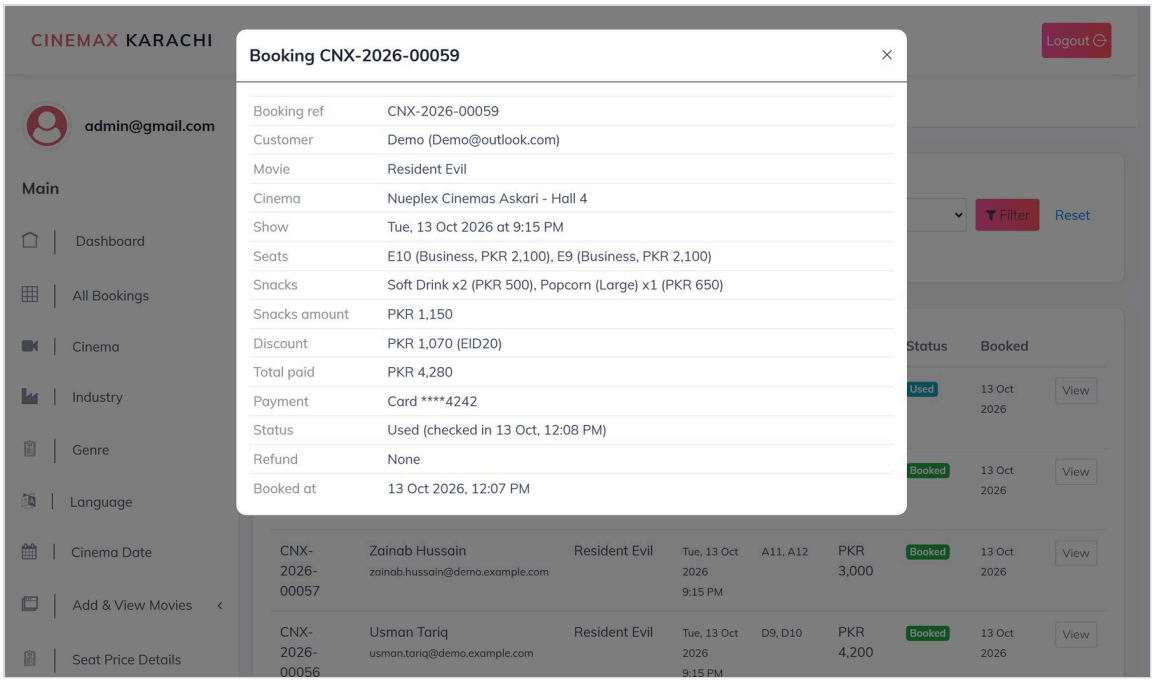


Figure 5.13: Booking details. Opening a booking from All Bookings shows its seats with prices, snacks, discount, payment, status and refund state (FR-42).

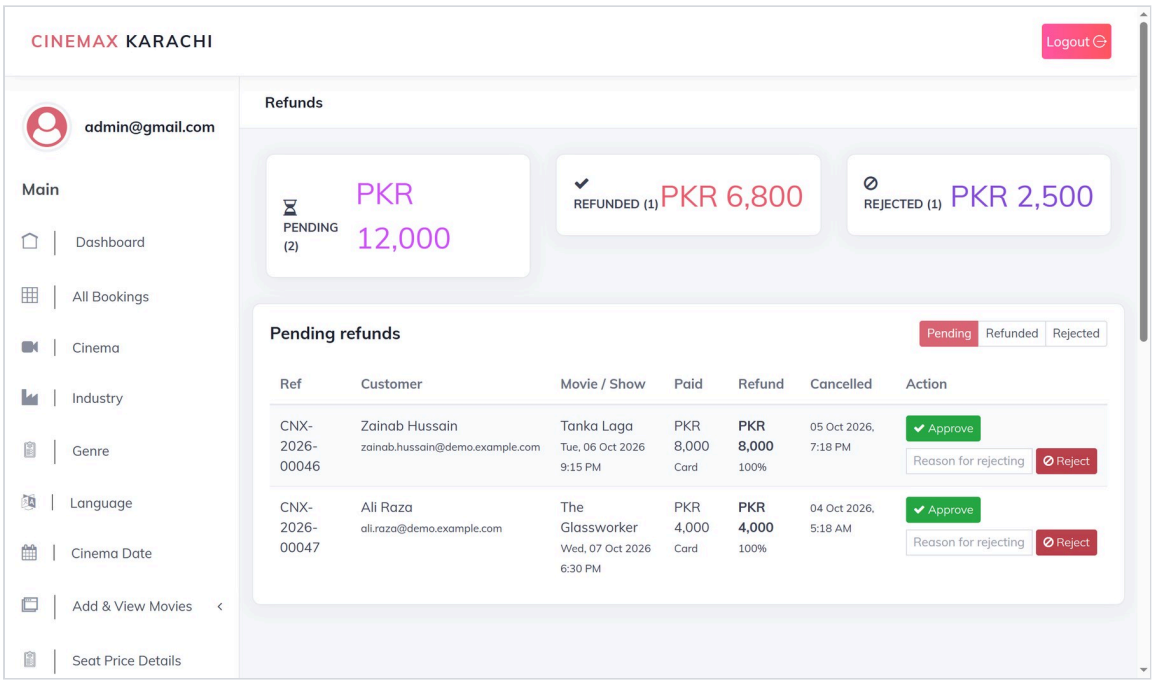


Figure 5.14: Refunds. Pending refunds with the amount due under the refund policy; each is approved, or rejected with a reason (FR-43).

5.14 Dashboard and reports

The dashboard computes its tiles in a single aggregate query over `booking_details`:

```
SELECT COUNT(*) AS bookings,
       COALESCE(SUM(b.status = 'Cancelled'), 0) AS cancelled,
       COALESCE(SUM(b.total_amount), 0) AS gross,
       COALESCE(SUM(CASE WHEN b.refund_status = 'Refunded' THEN b.refund_amount ELSE
0 END), 0) AS refunded,
       COALESCE(SUM(CASE WHEN b.refund_status = 'Pending' THEN b.refund_amount ELSE
0 END), 0) AS pending_amount,
       COALESCE(SUM(b.refund_status = 'Pending'), 0) AS pending_count,
       COALESCE(SUM(CASE WHEN b.status <> 'Cancelled' THEN <tickets> ELSE 0 END), 0)
AS tickets,
       COALESCE(SUM(DATE(b.booked_at) = CURDATE()), 0) AS today
FROM booking_details b
```

Counting tickets must work for both new and legacy bookings. New bookings store seat labels ("E6, E7, G3"), so the ticket count is the number of commas plus one; legacy bookings store a count in the same column. The shared SQL expression `TICKETS_SQL` handles both:

```
CASE WHEN b.show_id IS NULL
     THEN IF(b.user_seat REGEXP '^[0-9]+$', CAST(b.user_seat AS UNSIGNED), 0)
     ELSE LENGTH(b.user_seat) - LENGTH(REPLACE(b.user_seat, ',', '')) + 1
END
```

Net revenue subtracts only refunds that were actually paid (`refund_status = 'Refunded'`). Four Chart.js charts show net revenue for the last seven days (line), the top five films by revenue (horizontal bar), the seat-class split (doughnut) and the booking-status split (pie). A table lists the eight most recent bookings.

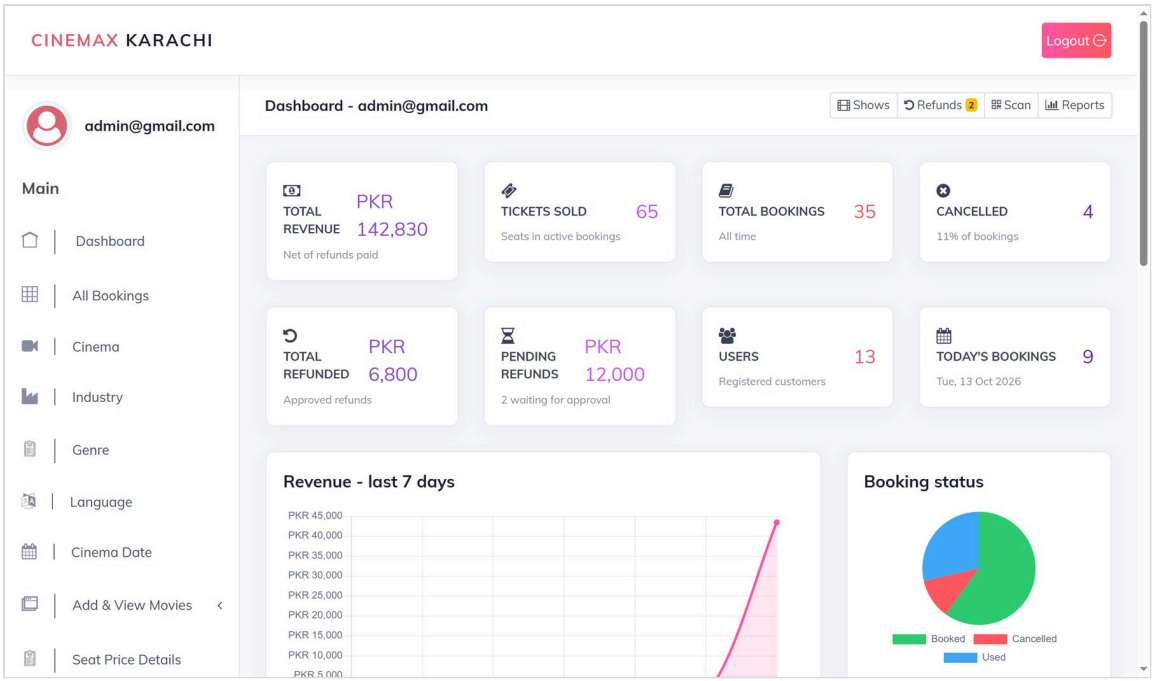


Figure 5.15: Dashboard. Revenue, tickets, bookings, cancellations, refunds, users and today's bookings, with quick links to shows, refunds, the scanner and reports (FR-45).

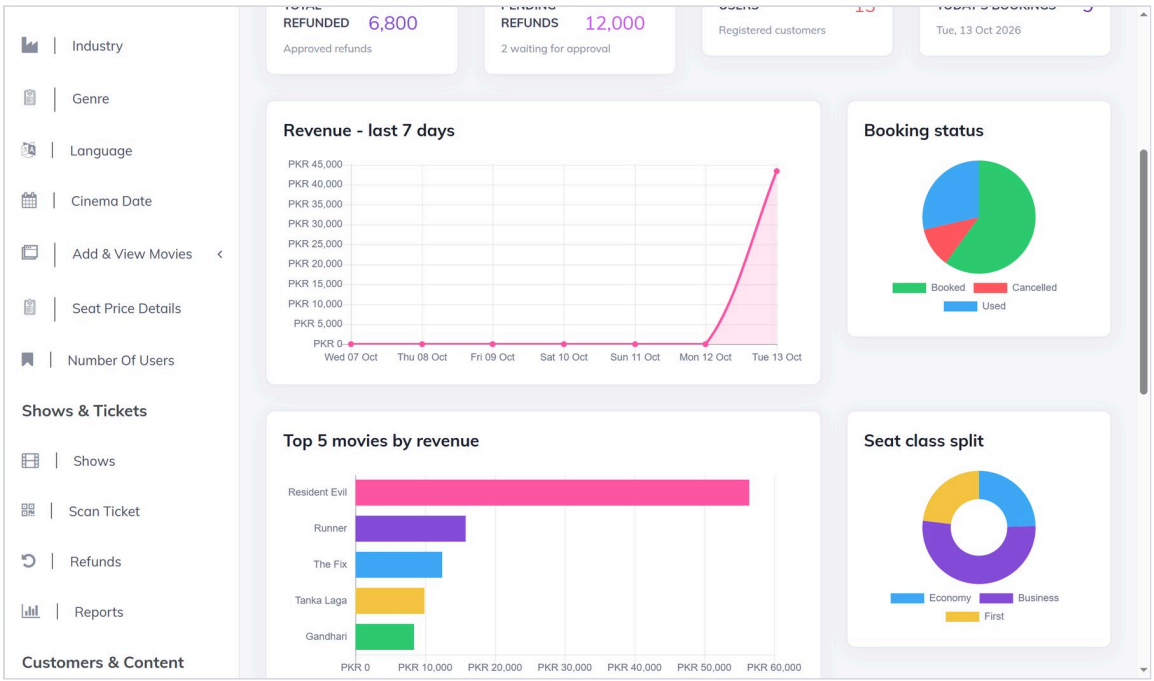


Figure 5.16: Dashboard charts. Net revenue for the last seven days, top five films by revenue, booking status and seat-class split (FR-45).

The reports page filters by date range (defaulting to the current month, with quick ranges such as last 7 days and this year), film and cinema, and shows net revenue, gross revenue, tickets, refunds, snacks and discounts, with per-film and per-cinema breakdowns. **Export CSV** streams

the filtered bookings with `fputcsv()`. Each cell passes through `csv_cell()`, which prefixes a quote to any value beginning with `=`, `+`, `-`, `@`, tab or carriage return, so a customer name such as `=HYPERLINK(...)` cannot execute as a formula when the report is opened in Excel. A print stylesheet produces a clean paper report.

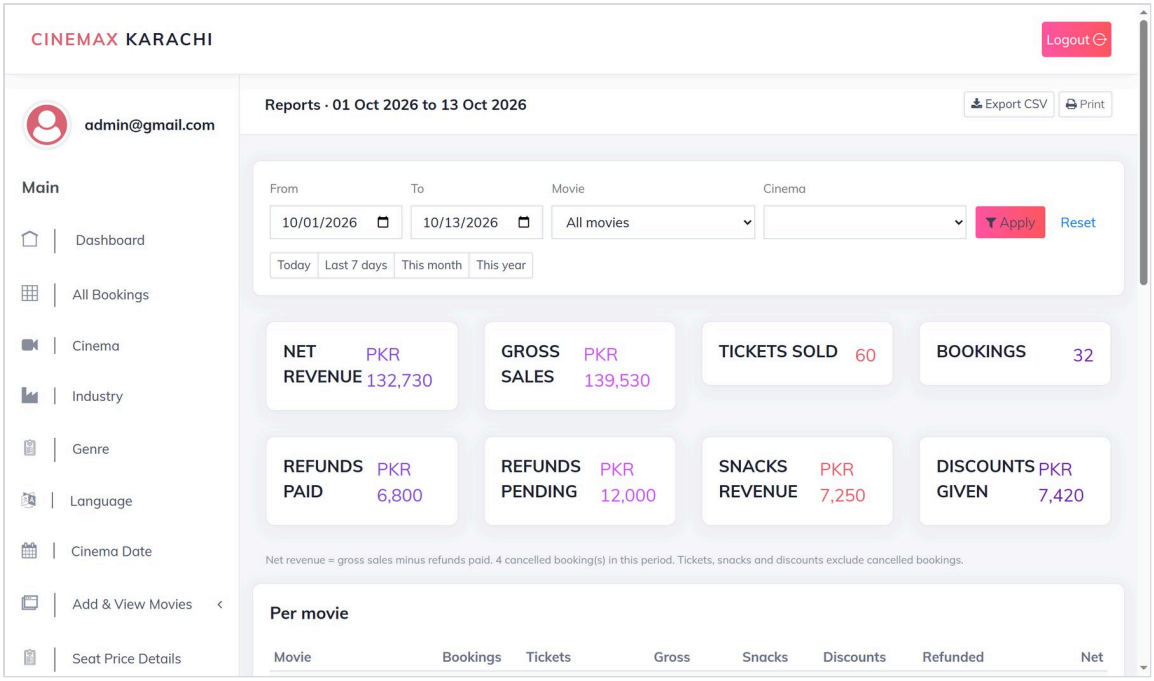


Figure 5.17: Reports. Date-range, film and cinema filters, revenue and refund totals, per-film and per-cinema breakdowns, CSV export and print (FR-46).

5.15 Customer account features

- **Registration** uses a column-list `INSERT` through a prepared statement, checks that the email is well formed and not already registered, and requires at least six password characters.
- **Profile** lets the customer change name and phone, upload a photo through `admin_upload_image()` into `uploads/profile/`, and change password. Customers without a photo get an avatar with their initials.
- **Reviews.** Each customer may have one review per film. The page looks up the customer's existing review and updates it instead of inserting a second one; the review can also be deleted. The average shown on the film page is computed as `SELECT COUNT(*), AVG(rating) FROM reviews WHERE movie_id = ? AND status = 'Visible'`, and rendered as full, half and empty stars.
- **Favourites** use `INSERT IGNORE` with the `UNIQUE (user_id, movie_id)` key, so a double click cannot create two rows, and the heart button toggles by deleting the row if it exists.

- **Live search.** `ajax/search.php` escapes the LIKE wildcards `%`, `_` and `\` in the query, matches titles containing the text, ranks titles that start with it first, and returns at most six results. The movie browser renders them as poster suggestions navigable with the arrow keys and Enter.
- **Filters.** The movie browser builds its `WHERE` clause from whitelisted integer parameters (`genre`, `language`, `cinema`) and binds them, and computes each film's next show time in a correlated subquery.

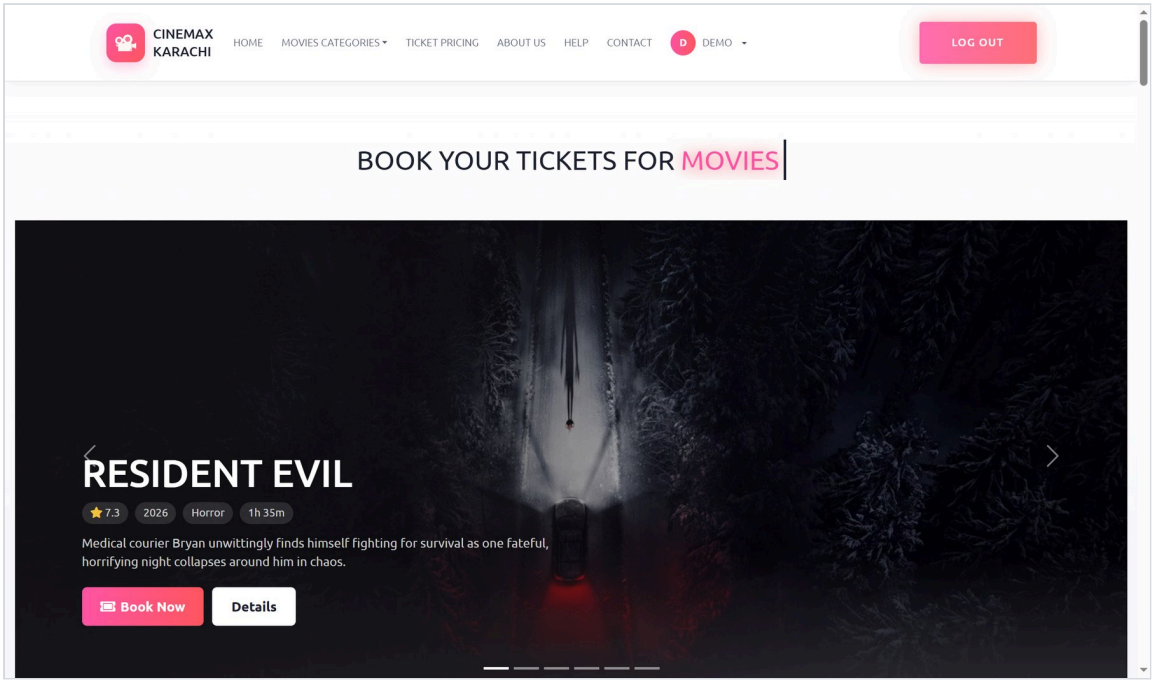


Figure 5.18: Home page. The slider shows the most popular current films from TMDB with rating, year, genre, runtime and a Book Now button (FR-01, FR-07).

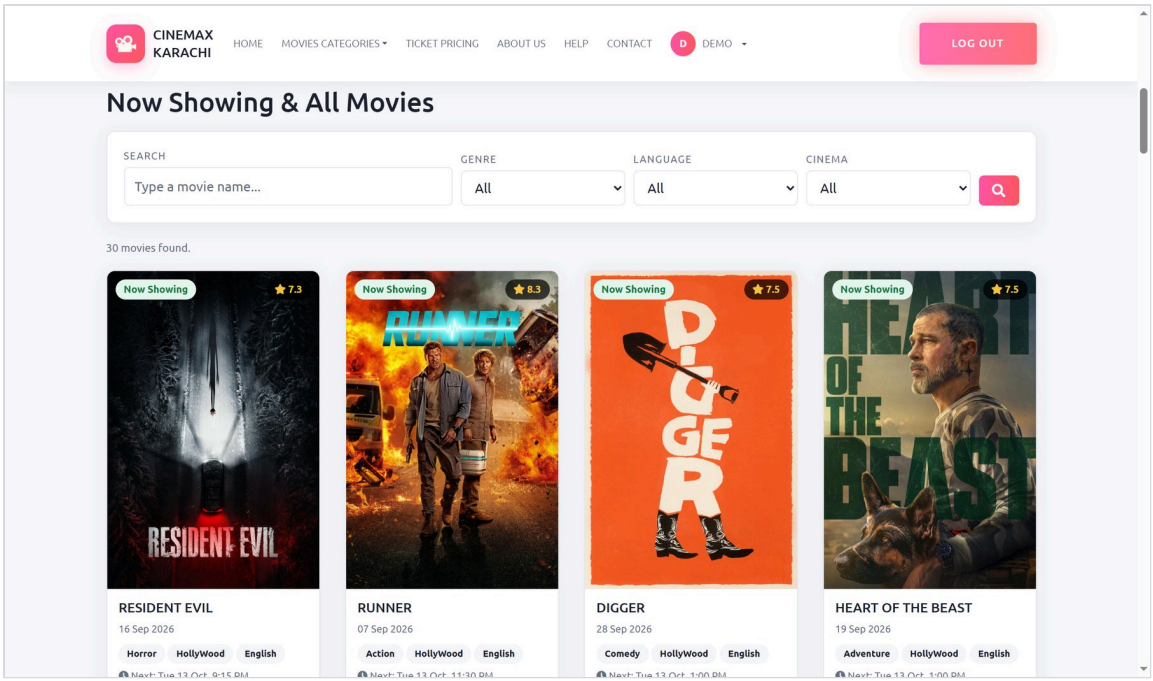


Figure 5.19: Movie browser. Live search, genre, language and cinema filters, and a card per film with its next show time (FR-01 to FR-04).

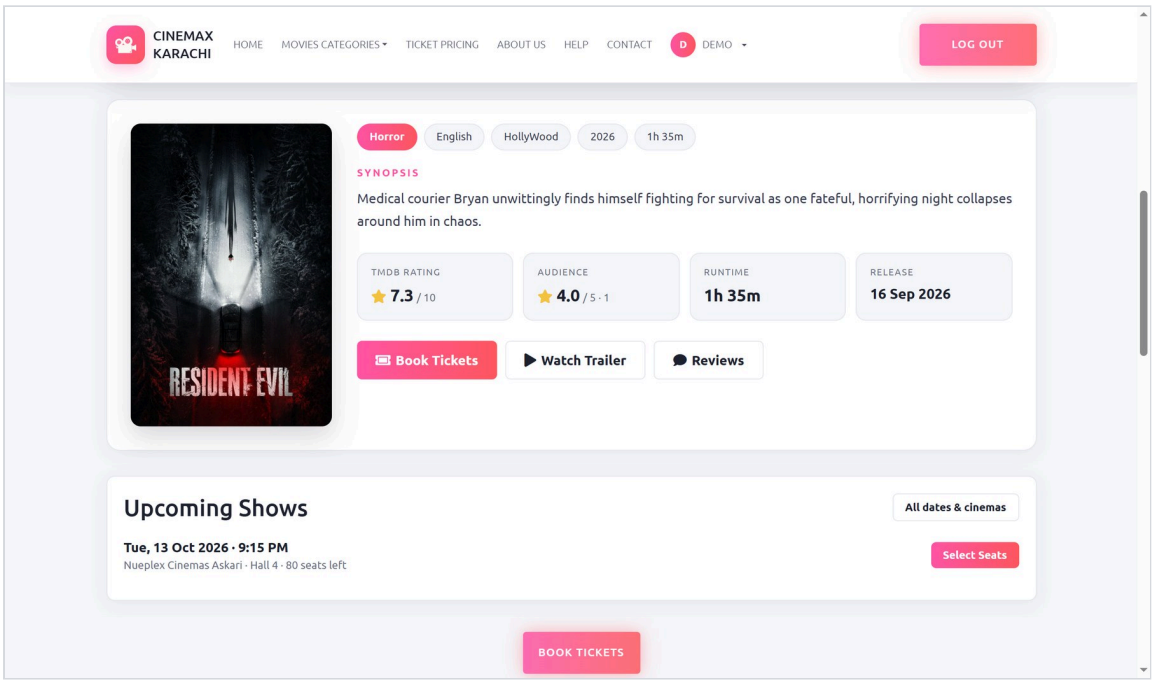


Figure 5.20: Movie details. Synopsis, TMDB and audience ratings, runtime, release date and the upcoming shows for the film, with the favourite button and reviews further down the page (FR-05, FR-12, FR-34, FR-35).

5.16 TMDB integration

5.16.1 The API client

`includes/tmdb.php` is a small cURL-based client. `tmdb_key()` reads the key from the `TMDB_API_KEY` environment variable or, failing that, from `includes/secrets.php`. `tmdb_get()` detects the credential type: a v4 read-access token (a JWT beginning `eyJ`) is sent as an `Authorization: Bearer` header, a v3 key as an `api_key` query parameter. It retries up to three times on HTTP 429, any 5xx status or a network failure, sleeping 1 and then 2 seconds between attempts (linear back-off), and raises an exception with TMDB's `status_message` on other errors.

`tmdb_trailer_key()` chooses the best YouTube video for a film by scoring each candidate: 4 points for type Trailer, 2 for Teaser, plus 1 if TMDB marks it official. `tmdb_download()` fetches an image and rejects responses that are not HTTP 200 or are smaller than 1 KB, a heuristic for error pages.

5.16.2 The seeding algorithm

`database/seed_tmdb.php` replaces the catalogue with current data. It runs only from the command line (it returns 404 over HTTP) and does nothing without the `--yes` flag except print instructions and a backup command.

```
1. Preconditions: upload directory writable; migration 005 applied; at least one
cinema.
2. Fetch TMDB's genre list.
3. Collect candidate films, skipping adult titles, titles without posters and
duplicates:
     Hollywood: /movie/now_playing (region US), English, released in the last 120
days   -> 16
     Bollywood: /discover/movie, original language Hindi, last 150 days, by
popularity   -> 8
     Lollywood: /discover/movie, origin country PK, last 3 years, by popularity
-> 6
     Coming soon: /movie/upcoming (region US), release date after today
-> 8
4. For each film, fetch details with videos; download poster (w500) and backdrop
(w1280).
   Skip films whose poster download fails.
5. If fewer than 3 films were downloaded, stop: the database is untouched.
6. Sort films by popularity. BEGIN TRANSACTION.
7. Delete holds, booking seats, booking snacks, bookings, reviews, favourites,
shows,
   prices, coming soon and films. Reset promo usage and loyalty points.
8. For each film (rank r):
```

```
     map original language and origin country to an industry (PK -> LollyWood,
       en -> HollyWood, hi -> BollyWood, ...);
     look up or create genre, language and release-date rows;
     insert the film with TMDB id, backdrop, rating, runtime, popularity, trailer
URL;
     price tier by rank: r < 5 -> 1500/2100/2900; r < 15 -> 1200/1700/2400; else
1000/1400/2000;
     create 4 shows (r < 10) or 3 shows over the next 7 days at rotating times
       (13:00, 15:45, 18:30, 21:15, 23:30), cinemas and halls, never within the next
hour.
9. Insert coming-soon rows. Delete genres, languages and dates no film uses.
10. Log the seeding in activity_log. COMMIT (or ROLLBACK on any exception).
11. Reset auto-increment counters of the booking tables; delete stale tmdb_* images.
```

Three properties make the seeder safe to run before a demonstration. All network work happens *before* the transaction, so a TMDB outage leaves the database untouched. The database work happens in one transaction, so a failure part-way through rolls back completely. And the seeder refuses to proceed with a nearly empty result, so a bad API response cannot wipe the catalogue.

The popularity-based price tiers mean the most popular new releases cost more, a simple model of demand-based pricing. The resulting catalogue in our environment held 30 films (16 Hollywood, 8 Bollywood, 6 Lollywood), 8 coming-soon titles and 112 shows across 9 cinemas.

5.16.3 Demo data

`database/seed_demo.php`, also CLI-only, adds a realistic operating history on top of the catalogue so the admin panel is not empty during a demonstration: eight customers under `@demo.example.com` (one of them blocked), a few past shows, about two dozen bookings across the previous week in every state (checked in, upcoming, cancelled with pending, refunded and rejected refunds), snack sales, promo usage, reviews and favourites. It builds bookings the same way `finalize_booking()` does and, when re-run, replaces only its own rows.

5.17 Deployment

The repository is its own deployment unit.

```
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - .:/var/www/html
    environment:
```

```
      DB_HOST: db
    depends_on:
      - db
  db:
    image: mariadb:10.4
    command: --sql-mode=""
    environment:
      MARIADB_ROOT_PASSWORD: ""
      MARIADB_ALLOW_EMPTY_ROOT_PASSWORD: "1"
      MARIADB_DATABASE: dbmo
    volumes:
      - ./dbmo.sql:/docker-entrypoint-initdb.d/dbmo.sql:ro
      - dbdata:/var/lib/mysql
volumes:
  dbdata:
```

The `Dockerfile` is two lines: `FROM php:7.4-apache` and `RUN docker-php-ext-install mysqli`. `config.php` reads the database host from `DB_HOST`, defaulting to `localhost`, so the same code runs in Docker (host `db`) and in XAMPP. The database is initialised from `dbmo.sql` on the first start; migrations are then applied with `docker compose exec -T db mysql -uroot dbmo < database/migrations/<file>`. The `--sql-mode=""` option reproduces the permissive SQL mode of a default XAMPP installation, which the legacy pages were written against. Appendix A gives the full installation procedure for both environments.

CHAPTER 6

Risk Management and Mitigation

The highest-impact risk in this project, selling one seat twice, was mitigated by design and verified by test; the largest residual risks are plain-text password storage and the simulated payment step, both of which are documented and scheduled as future work. This chapter describes how risks were identified and scored, then covers project, technical and security risks in turn, and ends with the residual-risk register.

6.1 Approach

Risks were identified at three points: during the audit of the baseline system, at the start of each increment ("what could this change break?"), and during the end-to-end test passes. Each risk was scored on a 1 to 5 scale for likelihood (L) and impact (I); the product L × I gives an exposure from 1 to 25. Exposures of 15 or more were treated as high and had to be mitigated before the increment was merged; 8 to 14 as medium, mitigated or explicitly accepted; below 8 as low, monitored.

Mitigations were recorded where developers would see them: in `TEAM_NOTES.md` (ownership, requests, known bugs), in `database/SCHEMA_CONTRACT.md` (rules), and in `VIVA_PREP.md` (limitations, stated honestly).

6.2 Project management risks

ID	Risk	L	I	Exposure	Mitigation	Status
P1	Developers overwrite each other's changes in shared files	4	4	16 High	File ownership in `TEAM_NOTES.md`; change requests instead of cross-edits; small commits with `pull --rebase` before push	Mitigated; no lost work recorded

ID	Risk	L	I	Exposure	Mitigation	Status
P2	Schema drift: each developer's database differs, code works on one machine only	4	4	16 High	Original dump never edited; numbered, re-runnable migrations; schema contract that overrides code	Mitigated
P3	One developer's schema change breaks the other's pages	3	4	12 Medium	Contract review before migrations; announced as URGENT requests	Occurred once: migration 001 broke sign-up and Add Movie (INSERTs without column lists); fixed the same day
P4	Scope creep delays the core booking flow	3	4	12 Medium	Core flow (shows, seats, payment, tickets, refunds) delivered first in Phases 1 to 4; extras (reviews, favourites, TMDB) after	Mitigated
P5	Demonstration fails on the day (expired shows, empty dashboard, no network)	4	5	20 High	Seeders regenerate shows for the next 7 days and a week of bookings; written demo scripts; Docker environment reproducible offline once images are pulled	Mitigated
P6	Uneven contribution or unclear ownership at assessment	2	3	6 Low	Ownership table and commit history document who built what	Monitored

6.3 Technical risks

6.3.1 Double booking under concurrency (high, mitigated)

This was the defining technical risk (L 4, I 5, exposure 20). On a release night, many customers select the same central seats within seconds of each other. A check-then-insert implementation would sell some seats twice.

Mitigation. Three independent layers, described in Sections 4.6 and 5.6: unique keys on `(show_id, seat_label)` in both `seat_holds` and `booking_seats`; all-or-nothing hold transactions; and a booking transaction that re-reads holds `FOR UPDATE`. The layers are independent: removing any one still leaves the unique key on `booking_seats`, which on its own prevents a duplicate sale.

Verification. Section 7.5 reports a test in which ten authenticated users requested the same seat simultaneously, repeated for 21 seats (1 + 20 rounds, 210 requests). Every round produced exactly one successful hold and nine refusals.

6.3.2 Abandoned checkouts locking seats (medium, mitigated)

If holds never expired, a customer who closed the browser would block seats indefinitely (L 5, I 3, exposure 15). **Mitigation:** holds are leases with a five-minute expiry; expired holds are deleted on every read and write of seat state; a customer has at most one active selection, because a new hold replaces the previous one; and the checkout countdown returns the customer to the seat map at expiry.

6.3.3 Partial writes (high, mitigated)

A booking touches six tables. A failure halfway through could leave a paid booking without seats, or seats without a booking (L 2, I 5, exposure 10). **Mitigation:** one InnoDB transaction with a single failure path (`$fail`) that rolls back. The TMDB seeder applies the same rule to its catalogue replacement, and performs all network calls before opening its transaction.

6.3.4 Time-zone errors (medium, mitigated)

The Docker host runs on UTC while the business operates in Pakistan Standard Time (UTC+5). Hold expiry, show visibility and the 24-hour refund rule would be wrong by five hours if PHP and MySQL disagreed (L 4, I 3, exposure 12). **Mitigation:** the helper layer sets both the PHP time zone and the MySQL session time zone on every request. Times are compared in SQL where possible, using `TIMESTAMP(show_date, show_time) > NOW()`.

6.3.5 Legacy data compatibility (medium, mitigated)

Old bookings have no show, seats or totals. Reports and the dashboard could crash or miscount (L 4, I 2, exposure 8). **Mitigation:** migration 001 backfills totals, references and user ids; the dashboard's `TICKETS_SQL` expression counts tickets correctly for both formats; ticket pages fall back to the snapshot text when no show exists.

6.3.6 External API dependency (medium, mitigated)

TMDB could be unreachable, rate-limit the seeder, or return unexpected data (L 3, I 3, exposure 9). **Mitigation:** the website never calls TMDB at request time; all images are downloaded and served locally. The seeder retries on 429 and 5xx, refuses to proceed with fewer than three films, and rolls back on any error.

6.3.7 PHP version differences (medium, partially mitigated)

The team ran Docker with PHP 7.4 and XAMPP with PHP 8.2. From PHP 8.1, `mysqli` throws `mysqli_sql_exception` by default instead of returning `false` (L 4, I 3, exposure 12). Under PHP 8.2, a duplicate-key failure in `hold_seats()` therefore raises an uncaught exception, and the customer sees a PHP error page instead of "Sorry, C9 was just taken". The double booking is still prevented, because the unique key rejected the insert and the transaction is never committed, but the user experience is wrong.

Status: reported in `TEAM_NOTES.md` with a one-line fix, not yet applied. The fix is either to call `mysqli_report(MYSQLI_REPORT_OFF)` once in the helper layer, or to wrap `mysqli_stmt_execute()` in `db_query()` in a `try ... catch (mysqli_sql_exception $e)` that returns `false`. Our Docker tests ran on PHP 7.4, where the friendly message appears (Section 7.5).

6.3.8 Performance under load (low, accepted)

Pages run a modest number of queries, and the hottest endpoint, seat-status polling, runs three indexed queries. Section 7.6 shows comfortable throughput on one host. One known inefficiency, the legacy admin film list issuing five lookup queries per film (an N+1 pattern), affects only an administrator page and is accepted.

6.4 Security risks

6.4.1 Threat model

We considered four classes of attacker: an anonymous Internet user; a registered customer trying to access other customers' data or obtain free or cheaper tickets; a malicious website trying to act through a signed-in user's browser; and an attacker who obtains a copy of the database. Table 6.2 maps the main threats to controls, using the STRIDE categories (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege).

Table 6.2: Threats and controls

Threat	STRIDE	Control in CineMax Karachi	Evidence
SQL injection through login, search, filters, booking parameters	Tampering, Elevation	Prepared statements in all new and modified code (223 helper calls); integer casting of ids; LIKE wildcards escaped in search	Test S03 (Chapter 7)
Stored XSS through reviews, names, contact messages	Tampering	`e()` on every dynamic output	Unit check U17; test S04

Threat	STRIDE	Control in CineMax Karachi	Evidence
CSRF on cancellation, admin actions, refunds, scanner	Tampering	Synchroniser token checked with `hash_equals()`	Tests S06, A04
Opening admin pages or delete handlers without logging in	Elevation	`auth.php` first line of every admin page	Test A01: 8 of 8 pages redirect
Viewing or cancelling another customer's ticket (IDOR)	Information disclosure, Tampering	`owns_booking()` on ticket, success and cancel pages; identical "not found" message	Tests S01, S07
Price tampering (editing seat list or totals in the form)	Tampering	Seats come from server-side holds; totals recomputed in the transaction	Code review, Section 5.9
Replaying a ticket (screenshot shared)	Spoofing	Conditional Booked → Used update; second scan returns ALREADY_USED; QR links carry an HMAC signature, so forged or altered codes are refused and logged	Test A05
Uploading a script disguised as an image	Elevation	Extension allow-list, `getimagesize()` type match, 2 MB limit, random names, `.htaccess` denies execution	Code review, Section 5.13.4
Running the seeders over HTTP to wipe data	Tampering, Denial of service	`PHP_SAPI !== 'cli'` returns 404	Test A02
Open redirect after login	Spoofing	`after_login` accepted only if it is a local path	Code review, Section 5.4.3
CSV formula injection in exported reports	Tampering	`csv_cell()` prefixes formula characters	Code review, Section 5.14
Theft of card data from the database	Information disclosure	Only last four digits stored; CVV never stored	Test E01 (`user_card = 4242`)
Unattributed administrative changes	Repudiation	Activity log of every admin add, edit, delete, refund decision and check-in	Code review
Blocked user continues to act	Elevation	Sign-in refuses blocked users; `customer()` signs them out on next page	Test S05

6.4.2 Residual security risks

The following weaknesses remain. We list them explicitly because a system that will hold customer data must not hide them.

1. **Plain-text passwords (high).** Customer and administrator passwords are stored and compared as plain text, inherited from the baseline. Anyone with read access to the database, or to a backup, learns every password, and users who reuse passwords are exposed elsewhere. *Remedy:* store `password_hash($password, PASSWORD_DEFAULT)` (bcrypt) and check with `password_verify()`; migrate each existing user transparently at their next successful login; then remove the plain-text values.

2. **No CSRF token on the checkout and payment steps (medium).** `checkout.php`, `payment.php` and the AJAX endpoints `seat_hold.php`, `promo.php` and `payment_otp.php` accept POSTs without a token. Because payment is simulated, a forged request could at worst create holds or a booking on the victim's own account using details the attacker would have to supply, but the gap is inconsistent with the rest of the system. *Remedy:* add `csrf_field()` to the forms and send the token in a request header from the `fetch()` calls.

3. **Customer session fixation (medium).** Administrator login calls `session_regenerate_id(true)`, but customer sign-in does not. An attacker who can plant a session identifier in a victim's browser could share the victim's session after login. *Remedy:* one call to `session_regenerate_id(true)` after successful customer sign-in, plus `HttpOnly`, `Secure` and `SameSite=Lax` cookie flags in production.

4. **No login throttling (medium).** Neither login form limits failed attempts. *Remedy:* count failures per account and per IP address with a short lock-out, or add a CAPTCHA after several failures.

5. **Demonstration OTP returned to the browser (accepted for demo).** The OTP endpoint returns the code so the demonstration can proceed without an SMS gateway. In production the code must be sent only by SMS.

6. **API key in the private repository (accepted, controlled).** The TMDB key is committed in `includes/secrets.php` so that each developer can seed a database; the repository is private, an environment variable overrides the file, and the team notes require deleting the file from history and regenerating the key before the repository is ever made public.

7. **Development database configuration (accepted for development).** The database root account has an empty password and the SQL mode is permissive. These settings exist only for local development and must be changed for any deployment.

6.5 Data protection and privacy

The system processes personal data: names, email addresses, phone numbers, profile photos and booking history. Pakistan's draft Personal Data Protection Bill and general good practice suggest the following controls, several of which are already in place: data minimisation (only the last four digits of payment accounts; the QR code holds only the booking reference and its signature, and the public verification page masks the customer's name); purpose limitation (customer data is used only for bookings and the customer's own account); access control (customers see only their own bookings; administrators require authentication); and accountability (the activity log). Password hashing (6.4.2 item 1) and HTTPS in production are the most important remaining controls.

6.6 Residual-risk register

ID	Residual risk	Exposure after mitigation	Owner	Next action
R1	Plain-text passwords	15 High	Both	Implement bcrypt hashing with login-time migration
R2	PHP 8.1+ shows error page on seat conflict	8 Medium	Abdul Rafay	Catch `mysqli_sql_exception` in `db_query()`
R3	Missing CSRF on checkout, payment and AJAX endpoints	8 Medium	Abdul Rafay	Token in forms and `fetch()` headers
R4	Customer session fixation	8 Medium	Abdul Rafay	`session_regenerate_id(true)` on sign-in
R5	No login throttling	8 Medium	Both	Attempt counter with lock-out
R6	Simulated payment	Accepted for FYP	Operator	Integrate a gateway with server-to-server confirmation
R7	`movie_price` linked by film name	4 Low	Both	Add `movie_id` foreign key
R8	Double booking	2 Low (residual)	Abdul Rafay	Keep unique keys; re-run concurrency test after changes

The register shows that the residual risks are concentrated in authentication and payment, not in the booking engine. Each has a small, well-understood remedy, listed again as future work in Chapter 8.

CHAPTER 7

Testing, Evaluation and Results

All 22 unit checks, all 21 concurrency rounds and all 20 integration, security and end-to-end cases ran on 12 October 2026 against the project's Docker environment and behaved as specified, with one exception: a weak XSS probe, discussed below. Public pages sustained 130 to 176 requests per second at 10 concurrent clients with 95th-percentile latency at or below 112 ms. This chapter describes the test strategy and environment, then reports each level of testing, the performance benchmarks, and an evaluation against the objectives of Chapter 1.

7.1 Test strategy

The project has no automated test suite in the repository; the codebase is framework-less and the team prioritised features. Testing was therefore organised in five levels, combining scripted checks written for this evaluation with the team's manual end-to-end passes.

Level	What it checks	How
Unit	Pure business rules: seat classes, seat-label validation, totals, discounts, refunds, loyalty, references, escaping, CSRF	22 assertions run with the PHP CLI inside the web container against `includes/booking.php`
Integration	AJAX endpoints and page controllers with a real database and real sessions	cURL scripts with per-user cookie jars against `http://127.0.0.1:8080`
Concurrency	Double-booking prevention under simultaneous requests	10 signed-in users firing hold requests for the same seat in parallel
Security	Injection, XSS, CSRF, access control, ownership, blocked users, CLI-only tools	cURL scripts; code review
System / acceptance	The full customer and admin flows in a browser	The team's scripted end-to-end passes (`DEMO_SCRIPT.md` , `DEMO_SCRIPT_TEAM.md`) on XAMPP with PHP 8.2

Level	What it checks	How
Performance	Throughput and latency of key pages	ApacheBench (`ab`), 200 requests per page at concurrency 1 and 10

To keep the tests repeatable and harmless, the database was dumped before the scripted tests and restored from the dump afterwards; we confirmed after restoring that the highest booking id, the number of active holds and the promo usage counts matched their pre-test values.

7.2 Test environment

Component	Version or value
Host	Windows 10 Pro 22H2 (build 19045), Docker 29.8.1; container sees 12 CPUs and 15.6 GB RAM
Web container	`php:7.4-apache` , PHP 7.4.33, Apache with `mod_php` , `mysqli`
Database container	MariaDB 10.4.34, InnoDB, empty SQL mode
Data at test time	30 films, 112 shows across 9 cinemas, 26 bookings (14 Booked, 8 Used, 4 Cancelled), 13 users, 6 snacks, 3 promo codes
Test show	Show 189, "Drishyam: The Conclusion", 12 October 2026 at 18:30 (less than 24 hours ahead, so cancellations quote 50%); prices Economy 1,000, Business 1,400, First 2,000
Test accounts	10 active customers, 1 blocked customer, 1 administrator

7.3 Unit testing of business rules

The unit checks load `includes/booking.php` in the PHP CLI and assert the results of its pure functions. Two checks (U18 to U20) read the `promo_codes` table but do not write to it.

ID	Check	Result
U01	`seat_class` maps rows A and C to Economy	Pass
U02	`seat_class` maps rows D and F to Business	Pass
U03	`seat_class` maps rows G and H to First	Pass
U04	`valid_seat_label` accepts A1 and H12 in an 8 × 12 hall	Pass
U05	`valid_seat_label` rejects I1, A13, A0, 1A and the empty string	Pass
U06	`generate_booking_ref(12, 2026)` is `CNX-2026-00012`	Pass

ID	Check	Result
U07	`money(1100)` is `PKR 1,100`; `money(0)` is `PKR 0`	Pass
U08	Loyalty: 5,799 gives 57 points; 99 gives 0	Pass
U09	Tickets for A1, E6, G3 at 1,000 / 1,400 / 2,000 total 4,400	Pass
U10	EID20 on 4,400 gives discount 880 and total 3,520	Pass
U11	A 33% discount on 1,000 is floored to 330 (total 670)	Pass
U12	Class summary reads "Economy x1, Business x1, First x1"	Pass
U13	Cancelling 48 hours ahead refunds 100% (3,520 of 3,520)	Pass
U14	Cancelling 1 hour ahead refunds 50%, floored (1,760 of 3,521)	Pass
U15	No refund quote after the show has started	Pass
U16	No refund quote for Used or Cancelled bookings	Pass
U17	`e()` escapes `<`, `>`, `"` and `'`	Pass
U18	`find_promo('eid20')` finds `EID20` (case-insensitive)	Pass
U19	`find_promo('BADCODE')` returns "This promo code is not valid."	Pass
U20	`find_promo('')` returns "Please enter a promo code."	Pass
U21	`row_letters` clamps to between 1 and 26 rows	Pass
U22	`csrf_ok` rejects a wrong token and accepts the right one	Pass

Result: 22 of 22 passed.

7.4 Integration and end-to-end testing

7.4.1 Hold endpoint validation

ID	Request	Expected	Observed
I01	Hold seat `Z99`	Refused: seat does not exist	`{"ok":false,"message":"Seat Z99 does not exist in this hall."}`
I02	Hold 11 seats (A1 to A11)	Refused: limit is 10	"You can book up to 10 seats at a time."
I03	Hold without a session	HTTP 401	401, "Your session has ended. Please sign in again."

ID	Request	Expected	Observed
I04	Hold on show 999999	HTTP 404	404, "This show is no longer available for booking."

7.4.2 E01: complete booking

The Demo customer held E6, E7 and G3 on show 189, opened checkout (HTTP 200), applied promo code `eid20` through the AJAX endpoint (response: "EID20 applied: 20% off your order."), added one Soft Drink, submitted checkout (302 to `payment.php`), and paid by card `4242 4242 4242 4242`, expiry 12/28, CVV 123 (302 to `booking_success.php?ref=CNX-2026-00051`). The database then held:

Field	Value	Expected
`booking_ref`	CNX-2026-00051	Reference derived from booking id 51
`user_seat`	E6, E7, G3	Seats from the holds, naturally sorted
`user_price`	Business x2, First x1	Class summary
`snacks_amount`	250	One Soft Drink
`discount_amount`	1,010	floor((4,800 + 250) × 20%)
`total_amount`	4,040	5,050 − 1,010
`payment_method`, `user_card`	Card, 4242	Last four digits only
`status`	Booked	
`booking_seats` rows	E6 Business 1,400; E7 Business 1,400; G3 First 2,000	One row per seat with price at sale
Customer's remaining holds	0	Holds deleted on commit

Result: pass. The totals match the hand calculation in Section 5.7.2.

7.4.3 Ticket scanning and cancellation

ID	Scenario	Expected	Observed
A03	Administrator login with valid credentials	Redirect to dashboard	302 to `Admin/dashboard.php`

ID	Scenario	Expected	Observed
A04	Scan CNX-2026-00051 without CSRF token	Refused	HTTP 403
A05	Scan CNX-2026-00051 with token, then again; scan CNX-0000-00000	VALID, then ALREADY_USED; NOT_FOUND	VALID; ALREADY_USED; NOT_FOUND
E02	Book seat B5 (Economy, 1,000), then cancel with token	Cancelled, refund Pending, 50% = 500, seat released	Cancelled / Pending / 500 of 1,000; 0 `booking_seats` rows remain
S06	Cancel CNX-2026-00052 without CSRF token	No change	Redirected; status still Booked
S07	Cancel CNX-2026-00052 from another customer's session	No change	Redirected; status still Booked (refused at the token check; the ownership check on this path was verified by code review)
A06	Cancel the already-scanned ticket CNX-2026-00051	No change	Redirected; status still Used

7.4.4 Acceptance passes by the team

Independently of the scripted tests, the team ran the complete demonstration scripts in a browser on XAMPP with PHP 8.2. According to `TEAM_NOTES.md`, every flow worked end to end: browse, search and filter; seat map and holds; checkout with snacks and EID20; card and JazzCash payment; e-ticket and QR; My Bookings; cancellation at 100% and 50%; refund approval and rejection; scanner results VALID, ALREADY_USED, CANCELLED and NOT_FOUND; dashboard; reports and CSV. All 25 of the account and administration features in the second developer's scope passed. These passes also found the PHP 8.1+ exception issue described in Section 6.3.7.

7.5 Concurrency testing

This test evaluates objective O3 and requirement NFR-01 directly.

Method. Ten different customers signed in, each with their own session. For a given seat, all ten sent `POST ajax/seat_hold.php` with `show_id=189` at the same moment, as ten background cURL processes released together. Afterwards, every customer released their holds so the next

round started clean. The first run targeted seat H12; a second run repeated the experiment for 20 further seats (C1 to C10 and D1 to D10).

Results.

Run	Rounds	Requests	Rounds with exactly one winner	HTTP 200 (held)	HTTP 409 (refused)	Hold rows for the seat afterwards
C01: seat H12	1	10	1 of 1	1	9	1
C02: seats C1 to C10, D1 to D10	20	200	20 of 20	20	180	(released after each round)
Total	**21**	**210**	**21 of 21**	**21**	**189**	

Every refused customer received HTTP 409 with a message naming the seat, for example "Sorry, E6 was just taken by someone else. Please choose other seats." In test S02, a second customer tried to hold E6 after it had been sold in E01, and was refused in the same way. After the run, `seat_holds` contained no rows for show 189.

Interpretation. The application performed no locking of its own before the insert; the requests genuinely raced through `hold_seats()`. The unique key `uq_hold_seat` decided every race, which is the property the design relies on. Because the guarantee is enforced by the database, it does not depend on request timing, the number of web workers, or the PHP version: on PHP 8.1+ the losers would see an error page instead of the 409 message (Section 6.3.7), but they would still not obtain the seat.

7.6 Security testing

ID	Test	Expected	Observed	Result
A01	Request 8 admin pages without logging in (dashboard, bookings, shows, refunds, reports, scanner, users, `cin_del.php`)	Redirect to login	8 of 8 returned 302 to `Admin/login.php`	Pass
A02	Request `database/seed_tmdb.php` over HTTP	Not runnable	HTTP 404	Pass

ID	Test	Expected	Observed	Result
S01	Second customer opens Demo's ticket by reference	Not shown	"Ticket not found"; the reference does not appear in the page	Pass
S03	Customer login with email `' OR 1=1 --`	Login fails	"Invalid Email Or Password!"	Pass
S03b	Admin login with `' OR '1'='1` in both fields	Login fails	HTTP 200 with the login form, no redirect	Pass
S04	Live search for `<script>alert(1)</script>`	Not executed	JSON response with an empty result list; nothing reflected	Inconclusive (no reflection path; output escaping covered by U17 and code review)
S05	Blocked customer signs in	Refused	"Your account has been blocked..."	Pass
S06, S07, A04	Requests without CSRF token	Refused	See Section 7.4.3	Pass

7.6.1 Ticket authenticity tests

After the signed QR code was introduced (Section 5.10.1), a further 27 checks were run on 13 October 2026 against the same Docker environment, on a database backup that was restored afterwards: 10 unit checks of the signing functions and 17 checks through the running website. All 27 passed.

ID	Check	Result
Q01 to Q10	Signature is 24 hexadecimal characters; independent of the reference's letter case; different for different references; rejects empty, missing and altered signatures; verification link built correctly; links and plain references parsed correctly; names masked	Pass (10 of 10)
Q11	A new booking's ticket QR holds `verify.php?ref=...&sig=...` with the correct signature	Pass
Q12	Opening the link shows "Genuine ticket" with seats D5 and D6 and the masked name; the customer's email is not on the page	Pass
Q13	Altered signature, missing signature, or another booking's reference with this signature: "Not a valid CineMax ticket", and no booking details are revealed	Pass

ID	Check	Result
Q14	Scanning the link at the door: VALID, with the signature reported as verified, and hall and seats listed	Pass
Q15	Second scan: ALREADY_USED; the verification page then shows "Already used"	Pass
Q16	Forged code for another booking: FORGED; that booking's status is unchanged; the attempt appears in the activity log	Pass
Q17	Genuine, unused ticket for an earlier date: WRONG_DAY; the ticket stays Booked; the verification page shows "Show date has passed"	Pass
Q18	Cancelled booking: "Cancelled ticket" on the verification page and CANCELLED at the door	Pass
Q19	A reference typed in lower case at the door still checks in, flagged as not signature-checked	Pass
Q20	Scanner without an administrator session is redirected to the login page	Pass

7.7 Performance evaluation

Method. ApacheBench ran inside the web container against `http://127.0.0.1/`, removing network effects from the host. Each page received 200 requests, first with one client at a time (latency baseline) and then with ten concurrent clients. Pages requiring a login used an existing session cookie.

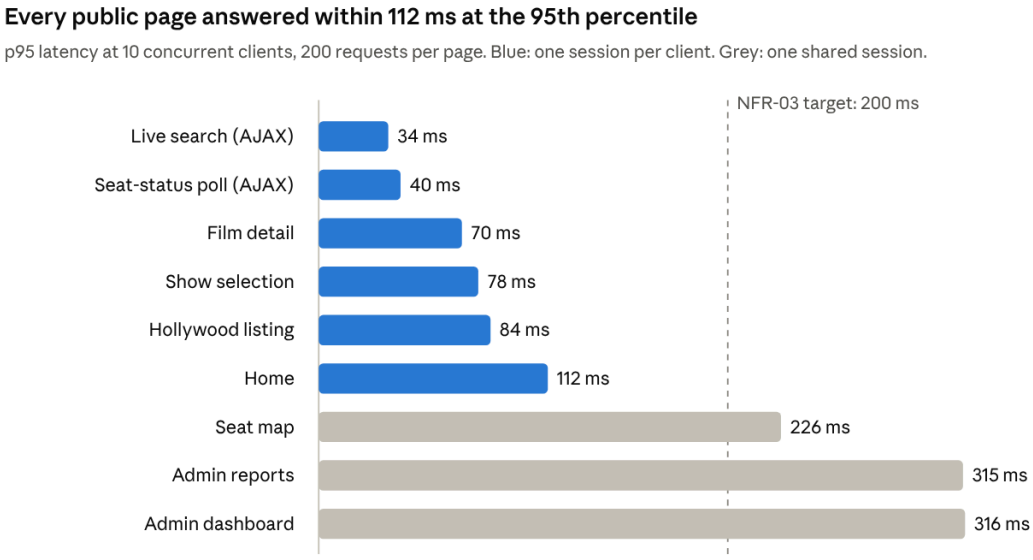


Figure 7.1: 95th-percentile latency at ten concurrent clients. All six public pages stay well under the 200 ms target; the three pages over it were tested with one shared session cookie, which PHP's session lock serialises (analysis point 3 below).

Results.

Page	Size (bytes)	req/s at c=1	Mean ms at c=1	req/s at c=10	Mean ms at c=10	p95 ms at c=10
Home (`index.php`)	76,099	23.7	42.2	130.8	76.4	112
Hollywood listing	41,100	32.4	30.9	175.9	56.9	84
Film detail	20,367	26.6	37.7	156.1	64.1	70
Show selection	12,539	33.3	30.1	161.1	62.1	78
Live search (AJAX)	746	65.2	15.3	360.3	27.8	34
Seat-status poll (AJAX)	57	49.5	20.2	298.1	33.5	40
Seat map (one session)	~63,000	30.4	32.9	51.8	193.0	226
Admin dashboard (one session)	18,889	25.7	38.9	34.9	286.8	316
Admin reports (one session)	27,467	27.6	36.2	35.8	279.7	315

No request failed. ApacheBench flagged the seat map's responses at c=1 as length mismatches because the page embeds per-request values, not because of errors.

Analysis.

1. **Public pages scale with concurrency.** Throughput rose about 5.5× from c=1 to c=10 for the listing and detail pages, and every public page met NFR-03 (95th percentile under 200 ms at ten concurrent clients).
2. **The polling endpoint is cheap.** At 298 requests per second, one host could serve the ten-second seat-status poll for roughly 3,000 customers looking at seat maps simultaneously (298 × 10), before counting other traffic.
3. **Single-session pages serialise.** The seat map and admin pages, tested with one shared session cookie, gained little from concurrency, and their latency rose to 190 to 320 ms. PHP's default file-based session handler locks the session file for the whole request, so concurrent requests carrying the *same* session wait for each other. Real customers each have their own session, so this is an artefact of the test setup rather than a capacity limit; it would matter only for one user opening many tabs. Calling `session_write_close()` early on read-only pages would remove it.
4. **The dashboard is the heaviest query load**, as expected for aggregate SQL over all bookings, yet it still answers in under 40 ms for one administrator.

These figures are from a development laptop with a small data set (26 bookings, 112 shows), so they show relative costs and headroom rather than production capacity. Of the queries that grow with data, the seats-left subquery and seat-state reads use the `uq_show_seat` and `uq_hold_seat` keys, and booking lookups use the indexes on `show_id`, `user_id`, `status` and `booking_ref`. The dashboard and report filters on `booked_at` are not indexed; an index on that column is the first optimisation to apply as booking volume grows.

7.8 Usability and accessibility review

We reviewed the main flow against the usability heuristics of Section 2.8.

- **Steps.** Film page → show selection → seat map → checkout → payment → e-ticket is five transitions, meeting NFR-08.
- **Status visibility.** The seat map shows a live total and seat chips; checkout and payment show a hold countdown that turns red in the last minute; the payment step shows a processing overlay.

- **Error prevention and recovery.** Taken seats are disabled before a click; selection is limited to ten in the browser; card and wallet fields are formatted as the user types; conflicts and expired holds produce specific messages and return the user to a sensible page.
- **Accessibility.** Seats are buttons with `aria-pressed` state and the countdown is an ARIA live region (`role="timer"`, `aria-live="polite"`). Two gaps remain: seat state is conveyed mainly by colour, and seat buttons show only the number, not the row; adding `aria-label` values such as "Seat E6, Business, PKR 1,400, available" would make the map usable with a screen reader.

7.9 Evaluation against objectives

Objective	Evidence	Assessment
O1 Show model with clash detection	`shows` table; admin shows page with clash query; 112 shows generated	Met
O2 Seat-level booking with classes	Seat map; U01 to U05; E01 class prices	Met
O3 Double-booking prevention	Unique keys; C01 and C02: 21 of 21 rounds with exactly one winner	Met
O4 Checkout and payment	E01: snacks, EID20, card payment; only last four digits stored	Met (payment simulated by design)
O5 Verifiable tickets	A05: VALID then ALREADY_USED; Section 7.6.1: signed QR verified by any phone, forged and wrong-day codes refused	Met
O6 Cancellation and refunds	U13 to U16; E02: 50% quote, seat released; admin approve and reject	Met
O7 Administration and analytics	A01; dashboard and report pages benchmarked; activity log	Met
O8 Real catalogue data	30 TMDB films, 8 coming-soon titles	Met
O9 Security baseline	Prepared statements, escaping, CSRF on admin and cancellation, upload hardening; S01 to S07	Largely met; gaps listed in Section 6.4.2
O10 Reproducible deployment	`docker compose up -d --build` used for every test in this chapter	Met

Nine of the ten objectives were fully met. Objective O9 was met for the threats the project prioritised (injection, XSS, access control, CSRF on administrative and cancellation actions), while password hashing, CSRF on the checkout endpoints, session-id regeneration for customers and login throttling remain outstanding.

CHAPTER 8

Future Scope and Conclusion

The system is ready to demonstrate and its booking engine is sound; making it ready for paying customers requires four security fixes and a real payment gateway, after which it can grow toward multiple cinemas, mobile clients and higher load. This chapter sets out that path in priority order and then concludes the report.

8.1 Immediate improvements (before any production use)

These changes close the residual risks of Section 6.6. Each is small and local.

1. **Password hashing.** Replace plain-text storage with `password_hash()` and `password_verify()`. A transparent migration avoids forcing resets: at sign-in, if the stored value is not a hash and matches the typed password, re-store it as a hash.

```
$user = db_one('SELECT id, name, email, password, status FROM user_reg_login WHERE
email = ? LIMIT 1', 's', [$email]);
$ok = $user && (password_verify($password, $user['password'])
    || (strpos($user['password'], '$2y$') !== 0 && hash_equals($user['password'],
$password)));
if ($ok && password_needs_rehash($user['password'], PASSWORD_DEFAULT)) {
    db_query('UPDATE user_reg_login SET password = ? WHERE id = ?', 'si',
[password_hash($password, PASSWORD_DEFAULT), $user['id']]);
}
```

2. **Exception-safe data access.** Make `db_query()` return `false` on `mysqli_sql_exception` so that PHP 8.1+ shows the intended "seat was just taken" message.
3. **CSRF everywhere.** Add tokens to `checkout.php` and `payment.php`, and send the token in an `X-CSRF-Token` header from every `fetch()` call, checked by the AJAX endpoints.
4. **Session hardening.** Regenerate the session id at customer sign-in; set `HttpOnly`, `Secure` and `SameSite=Lax` on the session cookie; serve the site only over HTTPS.
5. **Login throttling.** Lock an account or IP address for a few minutes after repeated failures.

6. **Production configuration.** A non-root database user with a strong password and only the privileges the application needs; strict SQL mode once the legacy pages are verified against it; error display off; secrets only in environment variables.

8.2 Functional extensions

Extension	Description	Approach
Real payment gateway	Accept money through JazzCash, Easypaisa or a card processor	Create the booking as "Awaiting payment" inside the hold window; confirm it only on the gateway's signed server-to-server callback; keep the existing transaction for the confirmation step; make the callback idempotent by gateway transaction id
Email and SMS	Send the e-ticket and OTPs; password reset	PHPMailer or a transactional email API; an SMS provider for OTPs; time-limited reset tokens stored hashed
Automatic refunds	Pay approved refunds back through the gateway	Gateway refund API triggered on approval, with the result recorded
Recurring schedules	"Every day at 19:25 for two weeks"	A schedule form that generates `shows` rows, reusing the clash check
Flexible hall layouts	Missing seats, wheelchair spaces, couple seats, curved rows	A `hall_layouts` table with a per-seat grid (class, type, blocked) referenced by shows; the unique-key design is unchanged because seats are still identified by label
Dynamic pricing	Weekday matinee discounts, premium evening prices	A price-rule table evaluated in `checkout_totals()`; the booking still stores the price at sale
Roles	Manager, box office, door staff	A `role` column on `admin_login` and a permission check in `auth.php`
Box-office sales	Staff sell tickets at the counter	An admin seat map that books for walk-in customers, sharing the same holds and transaction
Group bookings and gift cards	Larger parties and prepaid value	Raise the seat limit per role; a stored-value table debited inside the booking transaction
Recommendations	"Because you watched..."	Genre and favourites already exist; start with co-booking counts, then collaborative filtering
Multilingual interface	Urdu and English	Externalise strings; right-to-left stylesheet for Urdu

8.3 Data-model clean-up

- Link `movie_price` to films by `movie_id` with a foreign key, and store prices as integers.
- Add foreign keys from `booking_seats`, `booking_snacks`, `seat_holds`, `reviews` and `favorites` to their parents, where the snapshot design allows, and a unique key on `reviews (movie_id, user_id)` to make "one review per film" a database rule, as the code already enforces it.
- Add a unique key on `user_reg_login.email`; registration already checks for duplicates, but the database should guarantee it.
- Index `booking_details.booked_at` for date-range reports.
- Store show start as a single `DATETIME` column instead of a `DATE` and a `HH:MM` string, which would let the database index show-time comparisons.

8.4 Scalability

The current deployment is a single web container and a single database. Section 7.7 suggests this already serves several thousand simultaneous seat-map viewers. Growth beyond that follows a well-known path, and the design choices made so far keep it open:

1. **Stateless web tier.** Move PHP sessions from local files to Redis or the database. The web containers then become interchangeable and can run behind a load balancer. The holds already live in the database, not in sessions, so they work across servers without change.
2. **Cache read-heavy pages.** Film listings and detail pages change only when an administrator edits the catalogue; they can be cached for seconds to minutes. Seat state must not be cached, but its endpoint is already cheap.
3. **Push instead of poll.** Replace the ten-second poll with server-sent events or WebSockets fed by a lightweight pub/sub channel, reducing request volume and making seat changes visible within a second.
4. **Database scaling.** Read replicas can serve listings and reports. The booking transaction must stay on the primary, and because each show's seats are independent, the write load can later be partitioned by show or by cinema if a single primary is ever insufficient.
5. **Background work.** Move report generation, emails and the TMDB seeding to a job queue.
6. **Observability.** Structured logs, request timing and alerts on booking failures and payment callback errors.

The central invariant does not change at any step: whatever serves the request, the unique key on `(show_id, seat_label)` in the primary database decides who gets the seat.

8.5 Mobile and API

The AJAX endpoints already form a small JSON API for seats, holds, search and promotions. Adding endpoints for authentication (with tokens instead of cookies), shows, checkout and tickets would let a mobile app or a progressive web app reuse the same booking engine. A PWA could store tickets offline so that the QR code is available at the cinema entrance without a network connection.

8.6 Engineering practice

- **Automated tests.** Turn the 22 unit checks and the integration and concurrency scripts of Chapter 7 into a PHPUnit suite and a CI job that runs them against a containerised database on every push.
- **Static analysis.** PHPStan or Psalm at a low level would catch type errors and undefined variables in legacy pages.
- **Finish the migration to prepared statements** in the 22 remaining legacy `mysqli_query()` calls, and remove the N+1 queries in the admin film list with joins.
- **Accessibility.** Descriptive `aria-label`s on seats and non-colour cues for seat state.

8.7 Lessons learned

1. **Put invariants in the database.** The most valuable line in the project is a unique key. Application checks can be bypassed by a new code path, a race or a PHP upgrade; a constraint cannot.
2. **Design for the abandoned customer.** Most checkouts that start do not finish. Expiring holds made abandonment a non-event instead of a support problem.
3. **Small helpers go a long way.** A 145-line helper file supplied most of what a framework would have provided for safety (prepared statements, escaping, CSRF, flash messages, redirects), without rewriting the legacy pages.

4. **Migrations and contracts make teamwork possible.** Re-runnable migrations and a written schema contract let two developers change the same database in parallel. The one incident, migration 001 breaking column-less `INSERT`s, was caught and fixed within a day because the request process made it visible.
5. **Measure, then claim.** The concurrency test turned "we use a unique key" into "21 of 21 races produced exactly one winner", and the performance test revealed the session-locking effect, which code review alone would have missed.
6. **Be honest about limits.** Recording known weaknesses (plain-text passwords, demo payment) in the team's own documents kept them visible and made this report's risk register straightforward to write.

8.8 Conclusion

This project set out to turn a minimal, insecure academic booking site into a seat-level cinema ticketing system for Karachi that can be trusted not to sell a seat twice, using only conventional, low-cost technology: plain PHP, MariaDB and server-rendered pages.

The resulting system, CineMax Karachi, covers the full ticket lifecycle. Customers browse a catalogue of real current films from TMDB, choose a show, pick exact seats on a live map, add snacks and promotions, pay through a simulated card or mobile-wallet step, and receive a QR e-ticket, which they can cancel under a clear refund policy. Staff schedule shows with clash detection, manage prices, snacks, promotions and users, approve refunds, admit customers with a camera scanner, and follow sales on a dashboard with exportable reports.

The core technical contribution is the concurrency design: expiring holds stored in the database, unique keys on `(show_id, seat_label)` for both holds and sold seats, and a single transaction for each booking. Evaluation confirmed it: in 21 rounds of ten simultaneous requests for the same seat, exactly one request won every time, while all 22 business-rule checks and the end-to-end, scanning, cancellation and access-control tests behaved as specified, and public pages responded within 112 ms at the 95th percentile under ten concurrent clients.

The work also shows how an inherited codebase can be improved safely: by leaving the original dump untouched, delivering every schema change as a re-runnable migration, introducing a small shared helper layer, and dividing ownership clearly between developers.

The system is not yet ready to take real money. Passwords must be hashed, CSRF protection extended to the checkout endpoints, sessions hardened, and the simulated payment replaced with a gateway confirmed server to server. These are well-understood, bounded tasks. With them

completed, CineMax Karachi would offer a small Karachi cinema what has so far been available mainly to large chains: online, seat-level booking that customers can rely on, on infrastructure the cinema can afford.

References

Barth, A., Jackson, C. and Mitchell, J.C. (2008) 'Robust defenses for cross-site request forgery', in *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS '08)*. New York: ACM, pp. 75–88.

Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E. and O'Neil, P. (1995) 'A critique of ANSI SQL isolation levels', in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. New York: ACM, pp. 1–10.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Hoboken, NJ: Pearson.

Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD dissertation. University of California, Irvine.

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.

Gray, C.G. and Cheriton, D.R. (1989) 'Leases: an efficient fault-tolerant mechanism for distributed file cache consistency', in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*. New York: ACM, pp. 202–210.

Gray, J. and Reuter, A. (1992) *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann.

Halfond, W.G.J., Viegas, J. and Orso, A. (2006) 'A classification of SQL-injection attacks and countermeasures', in *Proceedings of the IEEE International Symposium on Secure Software Engineering*. Arlington, VA: IEEE.

ISO/IEC (2011) *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization.

ISO/IEC (2015) *ISO/IEC 18004:2015 Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*. Geneva: International Organization for Standardization.

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

MariaDB Foundation (2024) *MariaDB Server Documentation: InnoDB Lock Modes; ALTER TABLE; SELECT ... FOR UPDATE*. Available at: https://mariadb.com/kb/ (Accessed: 12 October 2026).

Merkel, D. (2014) 'Docker: lightweight Linux containers for consistent development and deployment', *Linux Journal*, 2014(239), article 2.

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94)*. New York: ACM, pp. 152–158.

OWASP Foundation (2021) *OWASP Top 10:2021*. Available at: https://owasp.org/Top10/ (Accessed: 12 October 2026).

OWASP Foundation (n.d.) *Cross-Site Request Forgery Prevention Cheat Sheet*; *File Upload Cheat Sheet*; *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/ (Accessed: 12 October 2026).

PHP Documentation Group (n.d.) *PHP Manual: mysqli prepared statements; password_hash; random_bytes; hash_equals; htmlspecialchars; session_regenerate_id*. Available at: https://www.php.net/manual/ (Accessed: 12 October 2026).

Shostack, A. (2014) *Threat Modeling: Designing for Security*. Indianapolis, IN: Wiley.

Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database System Concepts*. 7th edn. New York: McGraw-Hill.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

The Movie Database (n.d.) *TMDB API Documentation (v3): Now Playing, Upcoming, Discover, Movie Details, Videos, Images; Rate Limiting*. Available at: https://developer.themoviedb.org/docs (Accessed: 12 October 2026).

Software and libraries used: Apache HTTP Server; PHP 7.4 and 8.2; MariaDB 10.4; Docker and Docker Compose; Bootstrap 4.1.3 and 5.1.3; jQuery 3.3.1 and 3.6.0; Chart.js 2.7.2; SweetAlert2 11; Font Awesome 5.15.3; qrcode.js 1.0.0; html5-qrcode 2.3.8; FlixGo and Bootstrapious

dashboard templates. This product uses the TMDB API but is not endorsed or certified by TMDB.

APPENDIX A

Installation and Configuration

A.1 Docker (recommended)

Prerequisites: Docker Desktop (Windows or macOS) or Docker Engine with the Compose plugin (Linux).

```
docker compose up -d --build
docker compose exec -T db mysql -uroot dbmo <
database/migrations/001_shared_schema.sql
docker compose exec -T db mysql -uroot dbmo <
database/migrations/002_rafay_refund_reason.sql
docker compose exec -T db mysql -uroot dbmo <
database/migrations/003_rafay_admin_email.sql
docker compose exec -T db mysql -uroot dbmo <
database/migrations/004_umair_sample_data.sql
docker compose exec -T db mysql -uroot dbmo < database/migrations/005_rafay_tmdb.sql
docker compose exec -T db mysql -uroot dbmo <
database/migrations/005_umair_demo_data.sql
```

The site is then available at `http://127.0.0.1:8080` and the admin panel at `http://127.0.0.1:8080/Admin/login.php`. On Windows with Git Bash, prefix commands that pass container paths with `MSYS_NO_PATHCONV=1`.

To load a fresh catalogue from TMDB (this replaces films, shows, prices and bookings):

```
docker compose exec -T db mysqldump -uroot dbmo > backup.sql
docker compose exec web php database/seed_tmdb.php --yes
docker compose exec web php database/seed_demo.php
```

To stop: `docker compose down`. To reset the database to the original dump: `docker compose down -v` followed by `docker compose up -d` and the migrations.

A.2 XAMPP

1. Install XAMPP and start Apache and MySQL.
2. Place the project in `C:\xampp\htdocs\fyp_pro` (or create a junction with `mklink /J`).
3. In phpMyAdmin, create the database `dbmo`, import `dbmo.sql`, then import each migration in `database/migrations/` in number order.
4. Open `http://localhost/fyp_pro/`.

A.3 Configuration

Setting	Location	Default
Database host	`DB_HOST` environment variable, read by `config.php` and `Admin/config.php`	`localhost` (`db` under Docker Compose)
Database credentials	`config.php`, `Admin/config.php`	`root`, empty password, database `dbmo`
TMDB key	`TMDB_API_KEY` environment variable, else `includes/secrets.php`	none
Hold duration, seat limit	`HOLD_MINUTES`, `MAX_SEATS` in `includes/booking.php`	5 minutes, 10 seats
Site name and contact details	Constants in `customer_functions.php`	CineMax Karachi

A.4 Demonstration credentials

Role	Login	Password
Administrator	admin@gmail.com	12345
Customer	Demo@outlook.com	123
Demo customers (`seed_demo.php`)	`*@demo.example.com`	123

Demo payment: card 4242 4242 4242 4242, any future expiry, CVV 123; or JazzCash or Easypaisa with 03001234567 and the OTP shown on screen. Promo codes: EID20 (20%), FYP10 (10%), WELCOME50 (50%, 5 uses).

APPENDIX B

AJAX Endpoint Reference

Endpoint	Method	Parameters	Success response	Error responses
`ajax/seat_hold.php`	POST	`action=hold`, `show_id`, `seats` (comma-separated labels)	`{"ok":true,"expires_in":300,"redirect":"checkout.php?show_id=N"}`	401 not signed in; 405 not POST; 404 show unavailable; 422 no prices, no seats, more than 10 seats, invalid label; 409 `{"ok":false,"conflicts":[...],"message":...}`
`ajax/seat_hold.php`	POST	`action=release`	`{"ok":true}`	401
`ajax/seat_status.php`	GET	`show_id`	`{"ok":true,"booked":[...],"held":[...],"mine":[...]}`	404 show not found
`ajax/search.php`	GET	`q` (at least 2 characters)	`{"ok":true,"results":[{"id","name","poster","meta"}]}`, at most 6	Empty `results` for short queries
`ajax/promo.php`	POST	`code`	`{"ok":true,"code":"EID20","percent":20,"message":...}`	401; `{"ok":false,"message":...}` for invalid, expired or exhausted codes
`ajax/payment_otp.php`	POST	`mobile` (03XXXXXXXXX)	`{"ok":true,"otp":"######","message":...}` (OTP returned in demo build only)	401; invalid number

Endpoint	Method	Parameters	Success response	Error responses
`Admin/scan_ticket.php`	POST	`csrf`, `ref`	`{"result":"VALID"|"ALREADY_USED"|"CANCELLED"|"WRONG_DAY"|"FORGED"|"NOT_FOUND","ref","booking":{...}}`	403 without a valid token; redirect to login without an admin session

The public verification page is a normal page rather than a JSON endpoint: `GET verify.php?ref=<booking reference>&sig=<24 hex characters>` returns an HTML page with one of the verdicts Genuine ticket, Already used, Cancelled ticket, Show date has passed, Ticket not found or Not a valid CineMax ticket. It needs no sign-in, never changes a booking, and shows booking details only when the signature is valid.

APPENDIX C

Requirement-to-Test Traceability

Requirement	Test cases	Outcome
FR-14, FR-15 seat map and classes	U01 to U05, E01	Pass
FR-17 seat limit	I02	Pass
FR-18, FR-19 holds and conflicts	C01, C02, S02, I01	Pass
FR-21 promo validation	U18 to U20, E01	Pass
FR-22 totals	U09 to U12, E01	Pass
FR-24 last four digits only	E01	Pass
FR-25, FR-26 atomic booking	E01 (holds cleared, seats written); code review of rollback paths	Pass
FR-27 reference format	U06, E01	Pass
FR-31 own tickets only	S01	Pass
FR-32, FR-33 cancellation and release	U13 to U16, E02, S06, A06	Pass
FR-10 blocked users	S05	Pass
FR-37 admin authentication	A01	Pass
FR-44 ticket scanning	A04, A05	Pass
FR-50 seeder CLI-only	A02	Pass
NFR-01 no double sale	C01, C02 (21 of 21 rounds)	Pass
NFR-03 latency	Section 7.7	Pass for public pages

Requirement	Test cases	Outcome
NFR-05 injection and escaping	S03, S03b, U17; S04 inconclusive	Pass
NFR-06 CSRF	U22, S06, A04	Pass for admin and cancellation; not implemented on checkout and payment (R3)
NFR-12 one-command setup	Every test in Chapter 7 ran on `docker compose up`	Pass

APPENDIX D

Demonstration Script (Summary)

1. **Browse:** home slider, Now Showing grid, live search (type "res", arrow keys, Enter), genre and language filters, cinema filter, Coming Soon, industry pages.
2. **Pick a show:** date tabs, show times with hall and seats left; sign in when prompted and return to the same page.
3. **Seat map:** classes and prices; select E6, E7, G3; try an eleventh seat; Continue to hold for 5 minutes.
4. **Double-booking demonstration:** a second customer in a private window sees the seats on hold within ten seconds; racing for the same free seat gives one customer the seat and the other "Sorry, X was just taken".
5. **Checkout:** countdown, snacks, `BADCODE` refused, `EID20` applied.
6. **Payment:** card with auto-formatting, or JazzCash with OTP; processing overlay; confirmation.
7. **Ticket:** reference, QR, print or save as PDF.
8. **My Bookings and cancellation:** 100% quote more than 24 hours ahead, 50% within 24 hours; refund Pending.
9. **Admin:** dashboard tiles and charts; refunds approve and reject with reason; scanner VALID, ALREADY USED, CANCELLED, NOT FOUND; shows with occupancy and clash check; bookings filters; reports with CSV and print; blocked users.

APPENDIX E

Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability: the guarantees of a database transaction
CSRF	Cross-site request forgery: a forged request sent through a victim's signed-in browser
Hold	A temporary, expiring reservation of a seat for one customer during checkout
IDOR	Insecure direct object reference: accessing another user's record by changing an identifier
InnoDB	MariaDB/MySQL's transactional storage engine
Lease	A time-limited grant of exclusive access that lapses unless renewed
Lollywood	The Pakistani (Lahore-based) film industry
Migration	A numbered, re-runnable script that changes the database schema or data
OTP	One-time password
PRG	Post/Redirect/Get: answering a form POST with a redirect to avoid resubmission
QR code	Quick Response two-dimensional barcode (ISO/IEC 18004)
SSR	Server-side rendering of HTML pages
TMDB	The Movie Database, the source of film data
XSS	Cross-site scripting: injection of script into pages viewed by other users

APPENDIX F

Additional Administration Screens

The remaining administration pages, captured on 13 October 2026 from the same deployment as the screenshots in Chapter 5.

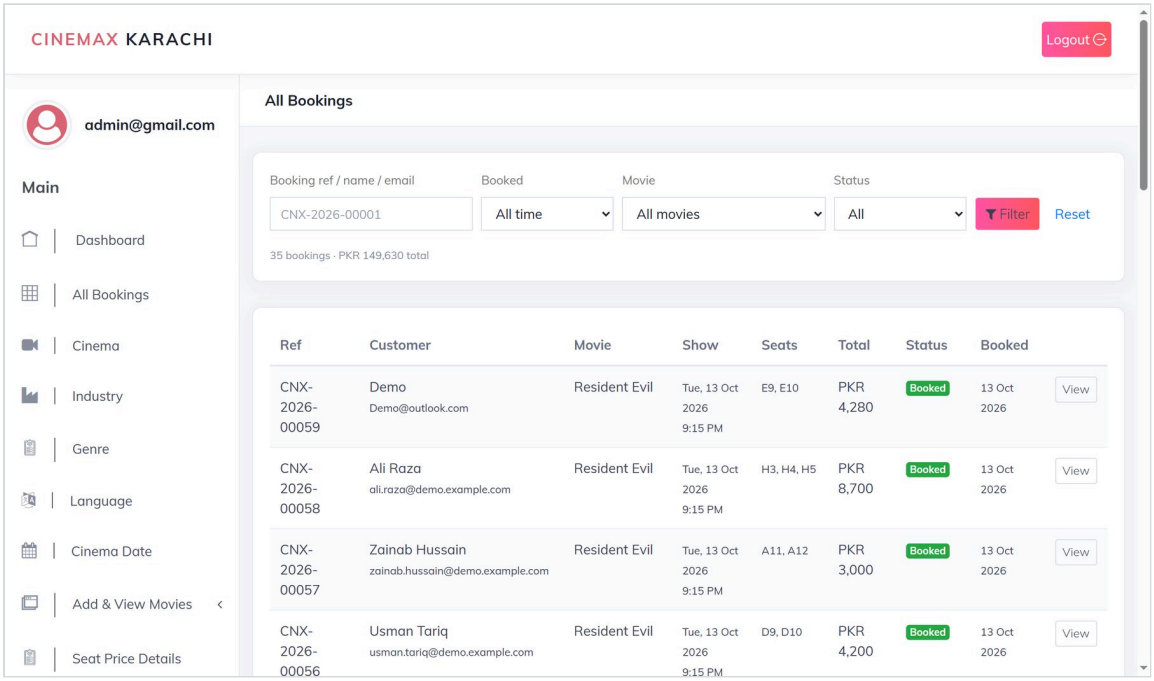


Figure F.1: All bookings. Search by reference, name or email, and filter by booking date, film and status.

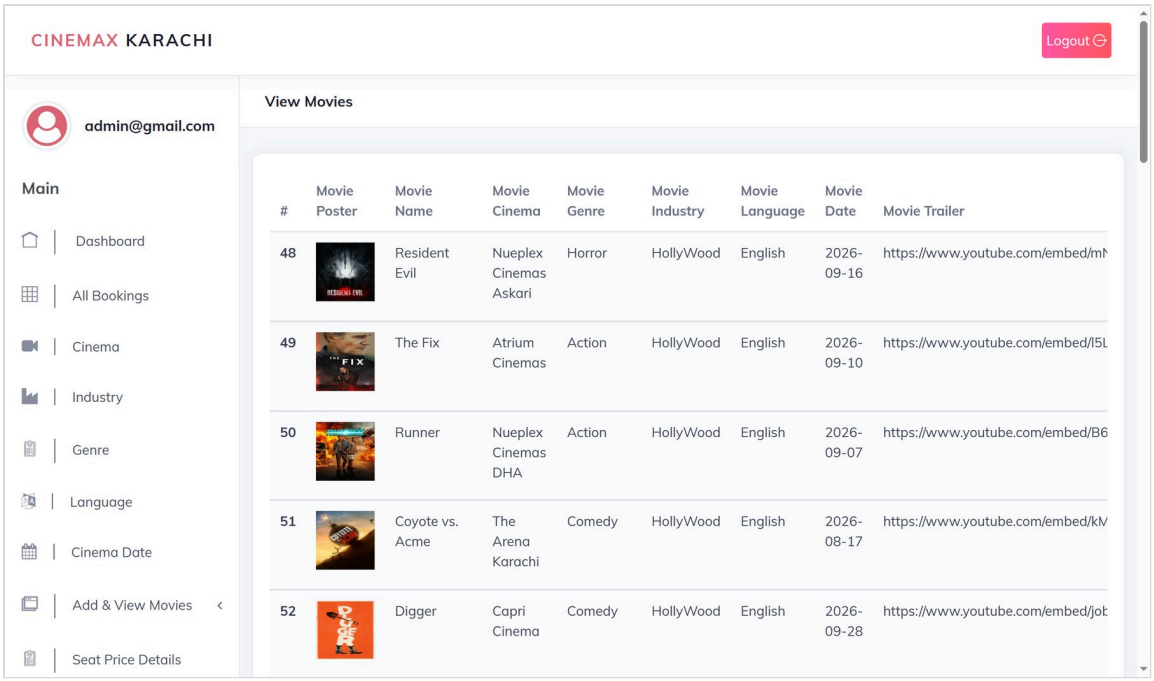


Figure F.2: Movies. The catalogue with poster, cinema, genre, industry, language, release date and trailer link.

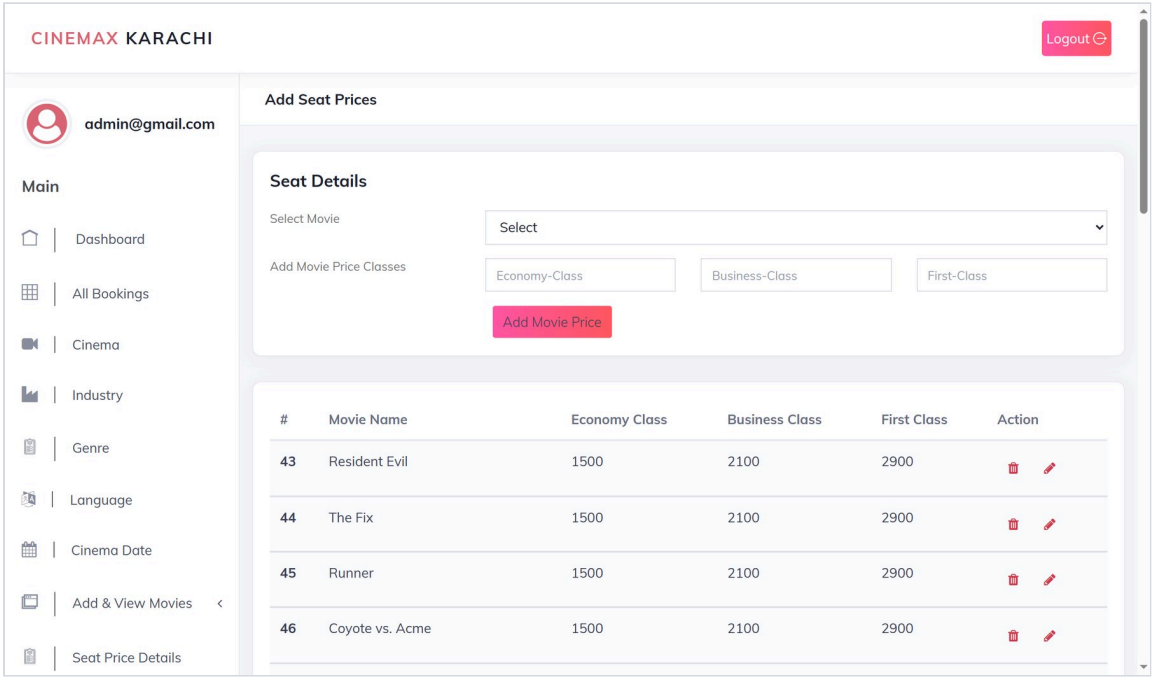


Figure F.3: Seat prices. Economy, Business and First prices per film.

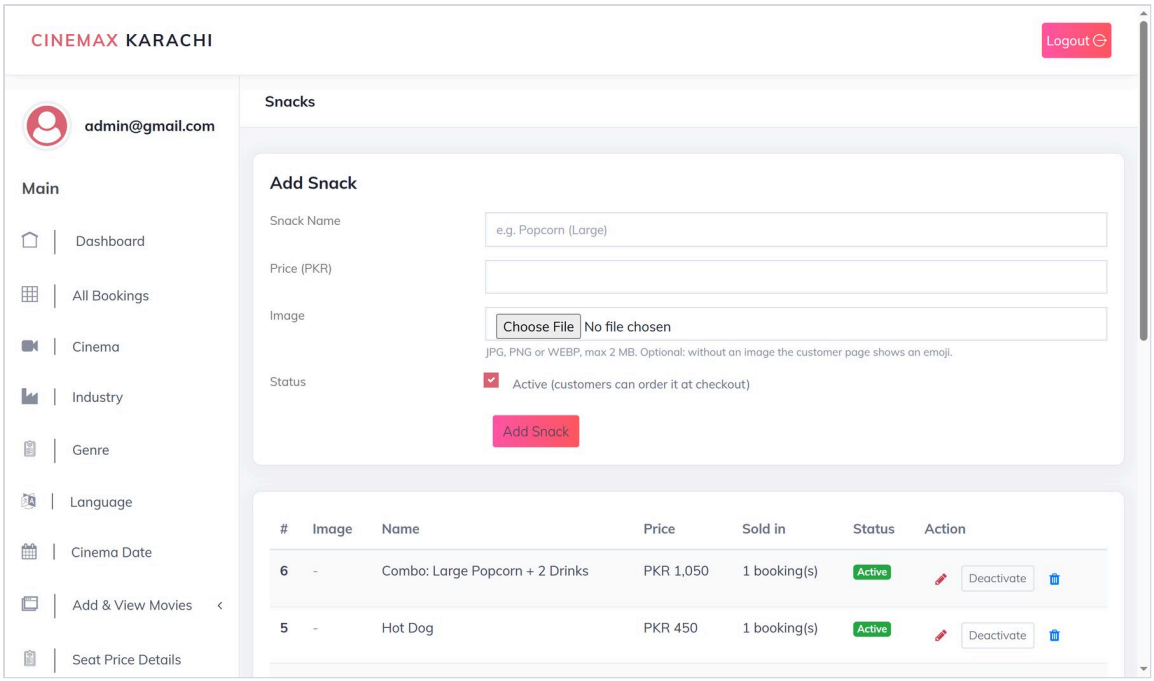


Figure F.4: Snacks. Prices, sales count and active status; sold snacks are deactivated rather than deleted.

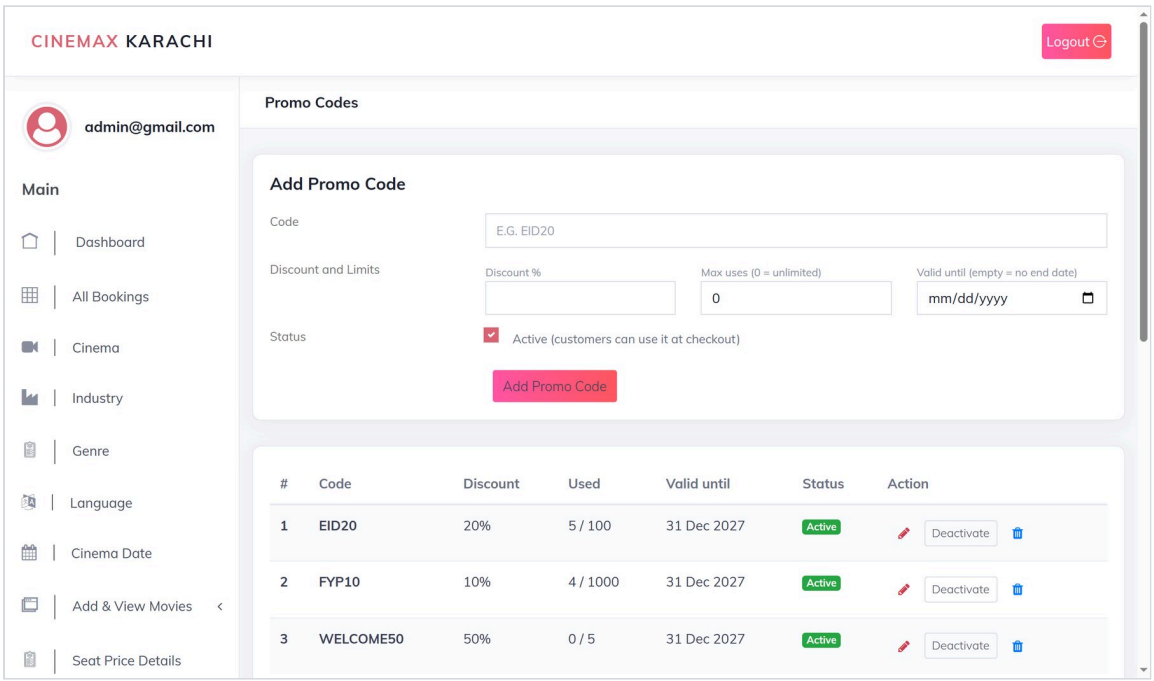


Figure F.5: Promo codes. Discount, usage against the limit, expiry and status.

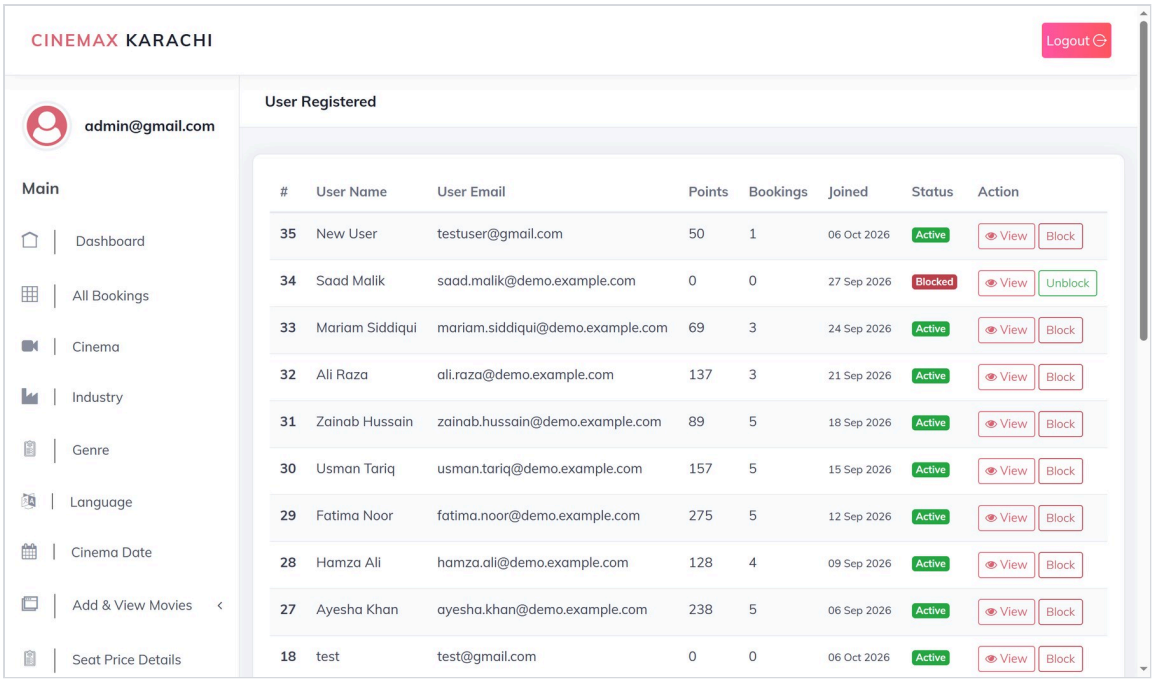


Figure F.6: Users. Loyalty points, booking count, join date and status, with block and unblock actions.

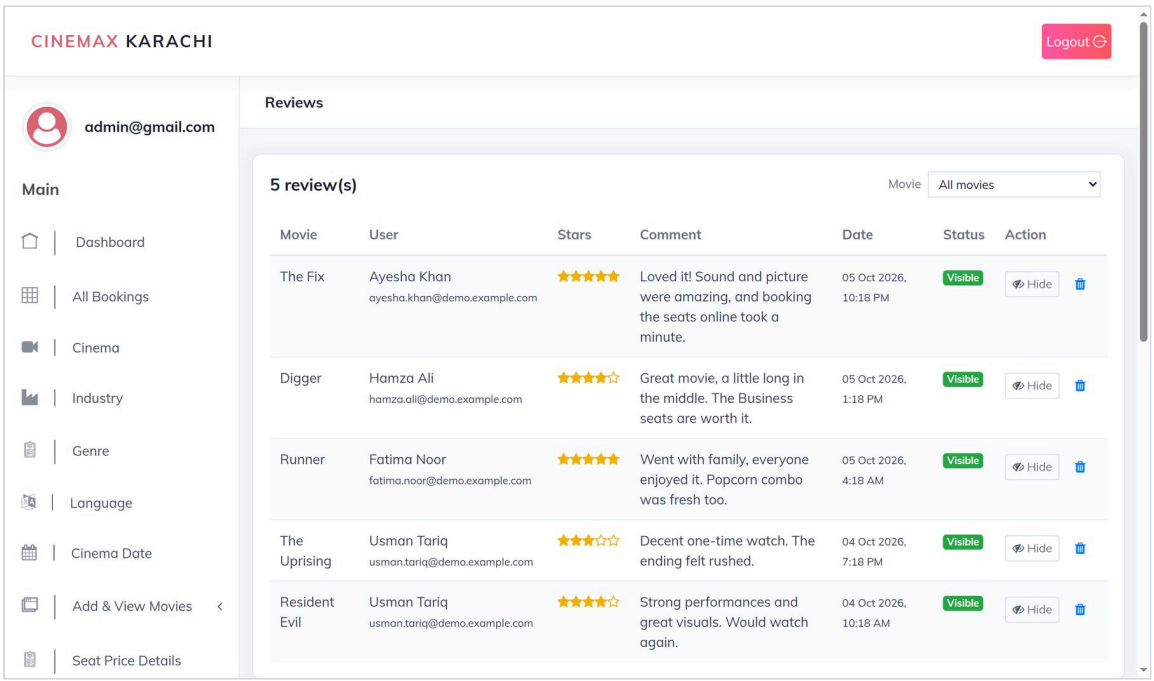


Figure F.7: Reviews. Star ratings and comments with hide and delete actions.

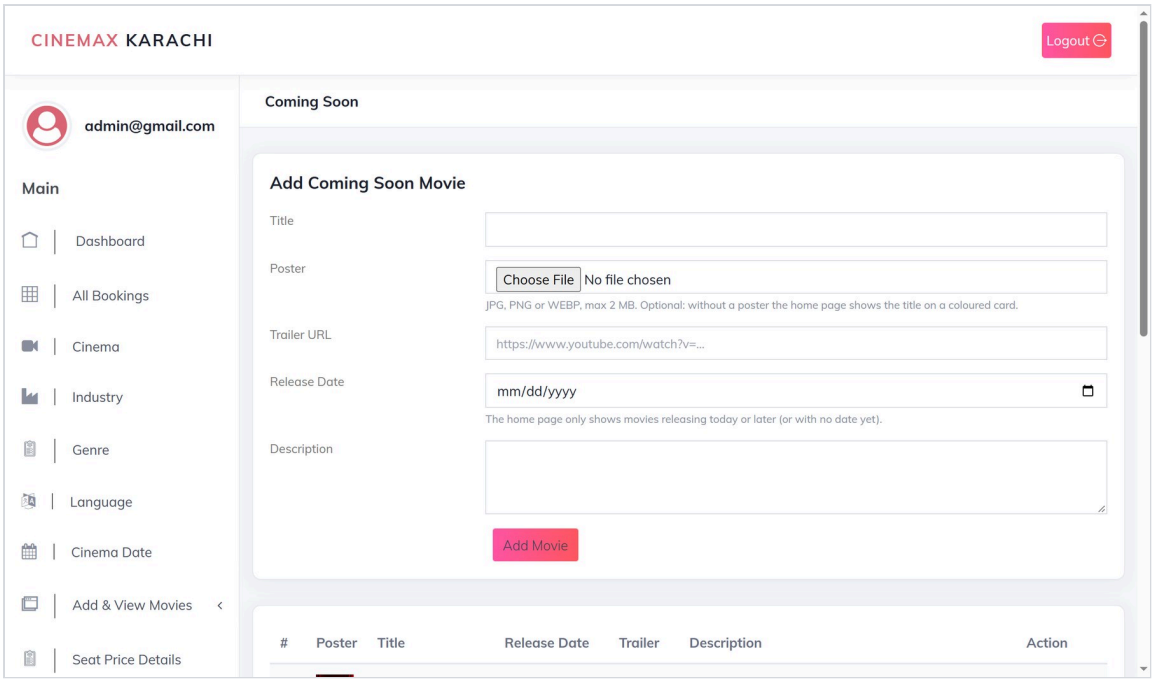


Figure F.8: Coming soon. Upcoming releases from TMDB with release dates and trailers.

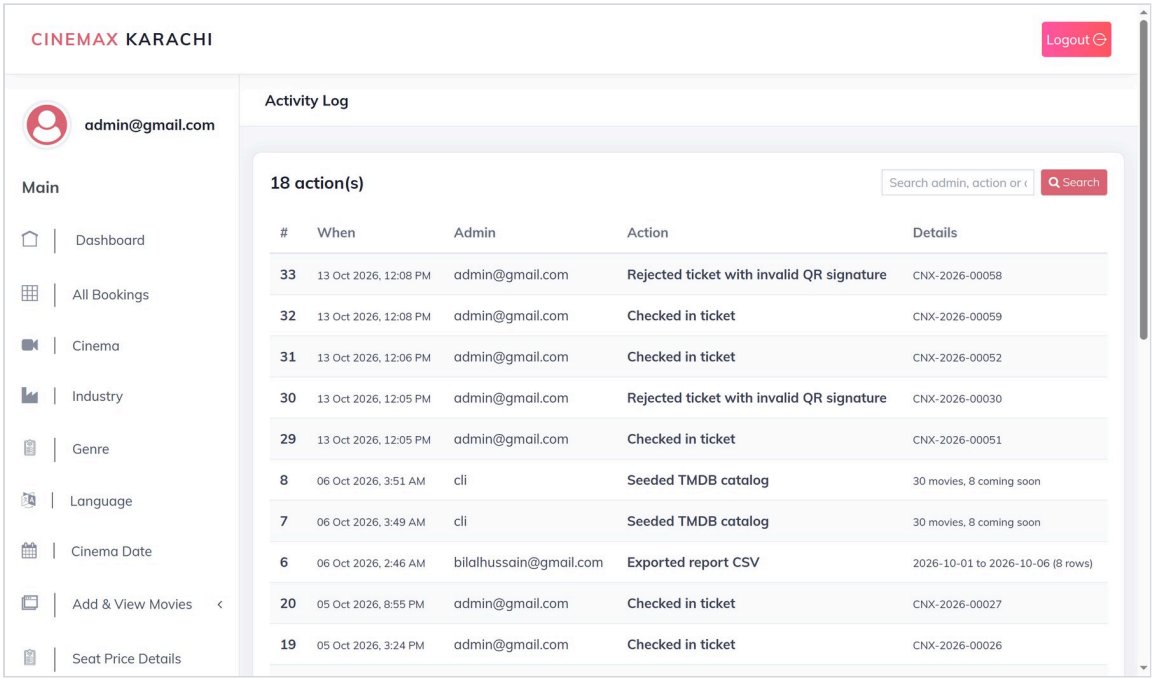


Figure F.9: Activity log. Every administrative change, check-in and rejected forged ticket, newest first.